

Vibe Engineering — Acople QuipuNet ↔ PrinterServices

1. Objetivo

Replicar la **orquestación de impresión** (estrategias, workflows, lógica de decisión) en el **Backend (QuipuNetX)**, copiando las clases necesarias desde el **Front (QuipuNet.exe)** sin eliminarlas del Front. Cuando el feature flag `USAR_PRINTER_SERVICE` esté activo y la máquina sea **SERVIDOR**, el backend ejecuta sus propias versiones de las estrategias de impresión, delegando la impresión física a **PrinterServices.exe** vía HTTP. Cuando el flag esté **OFF** o la máquina sea **CLIENTE**, el Front sigue usando sus clases originales con el flujo actual (`PrintUtil` → `ESCPOSPrinterEthernet` → `TCP:9100`).

1.1 REGLA CRÍTICA: Componente de monitoreo obligatorio

Toda acción relacionada con impresión que pase por PrinterServices **DEBE** mostrar el componente de monitoreo de impresión al usuario. Esto incluye pero no se limita a:

- **Envío de pedidos** (`addLista` → `verificarPrintJobsPorComandas`)
- **Reimpresión de pedidos** (`reImprimirPedido` → `verificarPrintJobsPorComandas`)
- **Cualquier acción futura** que genere `PrintJobResults` en la Respuesta

Flujo obligatorio cuando `USAR_PRINTER_SERVICE` está ON:

1. El **Backend** envía comandas a PrinterServices y retorna `PrintJobResults` en la Respuesta
2. El **Frontend (Presenter)** detecta `ImpresionList` vacía + flag ON → llama `verificarPrintJobsPorComandas`
3. Si hay `PrintJobResults` → `view.showImprimiendoPrinterService(onSuccess, onFail)` muestra modal de progreso
4. `PSProgressService` escucha callbacks reales de PrinterServices en tiempo real
5. Al completar → `onSuccess` (éxito) o `onFail` → `showFailedPrinterService` (modal de error con reintento)
6. Si **NO** hay `PrintJobResults` → mostrar error: *"No se ha podido imprimir, porque el servicio de impresión no responde"*

Razón: Sin el componente de monitoreo, el usuario no tiene feedback visual de que la impresión se está procesando, falló o completó. Esto genera confusión y pedidos sin imprimir sin notificación.

1.2 Principio rector: Camino A

Copiar las estrategias al backend, no interceptar en un punto intermedio. El Front **conserva intactas** sus clases originales para el flujo con flag OFF. Esto garantiza que:

- El backend orquesta TODO el flujo de impresión cuando FLAG ON (comprobantes, comandas, sorteo, encuesta, motorizado)
- El Front queda "tonto" cuando el flag está activo (solo envía pedido/venta, no imprime nada)
- El Front sigue funcionando normalmente cuando FLAG OFF o en máquinas CLIENTE
- `PrinterServices.exe` es solo el ejecutor físico (recibe jobs, imprime, notifica)
- Las estrategias del backend son adaptaciones de las del Front (Iterator→Controller, `PrintUtil`→`PrinterServiceClient`, sin UI)

1.3 Contexto: Arquitectura Cliente/Servidor

IMPORTANTE: Antes de continuar, es fundamental entender la arquitectura cliente/servidor de QuipuNet.

📖 Ver documento: [vibe_engineering_arquitectura_cliente_servidor.md](#)

Resumen clave:

- Una terminal **SIEMPRE** tiene `QuipuNet.exe` + `QuipuNetX.dll` (ambos componentes)
- El **Router** decide si ejecutar localmente (servidor) o hacer REST (cliente)
- El **Backend (Controller)** es quien decide si delegar a PrinterServices (solo servidor con flag ON)
- El **Front** NO verifica el flag — solo confía en lo que el backend le envía (`ImpresionList` o vacía)
- Un **cliente** recibe la decisión del servidor vía HTTP (`ImpresionList` vacía si servidor tiene flag ON)

1.4 Tabla de decisión: Los 4 escenarios posibles

Esta tabla resume **TODOS** los escenarios posibles de operación con el feature flag `USAR_PRINTER_SERVICE` :

Escenario	Terminal	Flag Servidor	Backend ejecuta	Retorna al Front	Front imprime
1	SERVIDOR	OFF	Controller local	<code>ImpresionList</code> CON comandas	✅ Sí (flujo antiguo)
2	SERVIDOR	ON	Controller local → PrinterServices	<code>ImpresionList</code> VACÍA	❌ No (delegó)
3	CLIENTE	OFF (servidor remoto)	REST → servidor Controller	<code>ImpresionList</code> CON comandas vía HTTP	✅ Sí (flujo antiguo)
4	CLIENTE	ON (servidor remoto)	REST → servidor Controller → PrinterServices	<code>ImpresionList</code> VACÍA vía HTTP	❌ No (servidor delegó)

Conclusiones clave:

1. **El SERVIDOR decide:** Solo el servidor (local o remoto) verifica `esModoServidor()` && `USAR_PRINTER_SERVICE` para decidir si delegar a `PrinterServices`
2. **El CLIENTE espera:** El cliente hace petición REST y espera la respuesta del servidor (`ImpresionList` con datos o vacía)
3. **El FRONT confía:** El Front (servidor o cliente) NO verifica el flag — solo verifica si `ImpresionList.Count > 0`
4. **ImpresionList vacía = delegado:** Si el Front recibe `ImpresionList` vacía, significa que el servidor ya delegó a `PrinterServices`
5. **ImpresionList con datos = imprimir:** Si el Front recibe `ImpresionList` con comandas, imprime localmente (flujo antiguo)

Flujos detallados:

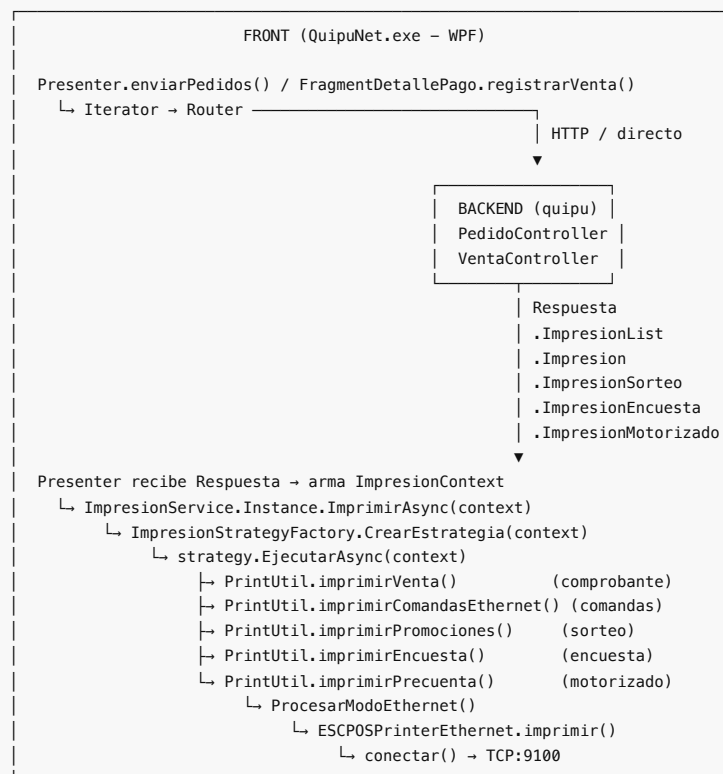
Escenario 1 y 2 (Terminal SERVIDOR):

```
Front → Iterator → Router [detecta esModoServidor() = TRUE]
└─ Controller.addLista() LOCAL
    └─ Genera ImpresionList
    └─ Verifica: esModoServidor() && USAR_PRINTER_SERVICE
    └─ Si OFF: retorna ImpresionList con comandas
    └─ Si ON: envía a PrinterServices, vacía ImpresionList, retorna vacía
```

Escenario 3 y 4 (Terminal CLIENTE):

```
Front → Iterator → Router [detecta esModoServidor() = FALSE]
└─ Client.addLista() REST al servidor
    └─ HTTP POST → Servidor recibe
        └─ Controller.addLista() en servidor
            └─ Verifica: esModoServidor() && USAR_PRINTER_SERVICE
            └─ Si OFF: retorna ImpresionList con comandas
            └─ Si ON: envía a PrinterServices, retorna vacía
        └─ HTTP Response → Cliente recibe ImpresionList
```

2. Arquitectura actual (SIN PrinterServices)



2.1 Flujo detallado de imprimirComandasEthernet (PrintUtil.cs:1060)

```
foreach Impresion in impresionComandasList:
    1. Impresora impresoraObj = Impresora.findById(impresion.Impresora_id) // ← BD local
    2. Setear propiedades desde impresoraObj:
        - Impresora_tamanoqr, Abregaveta, Cashdrawer, Printermodel, Tipoimpresora
```

```

3. switch(impresion.Impresora_modoimpresion):
- ETHERNET → ProcesarModoEthernet(impresion, impresoraObj, respuesta)
- SERVICIO → ProcesarModoServicio(impresion, impresoraObj, respuesta)
- SERIAL → ProcesarModoSerial(impresion, respuesta)

```

2.2 ProcesarModoEthernet (PrintUtil.cs:1178)

```

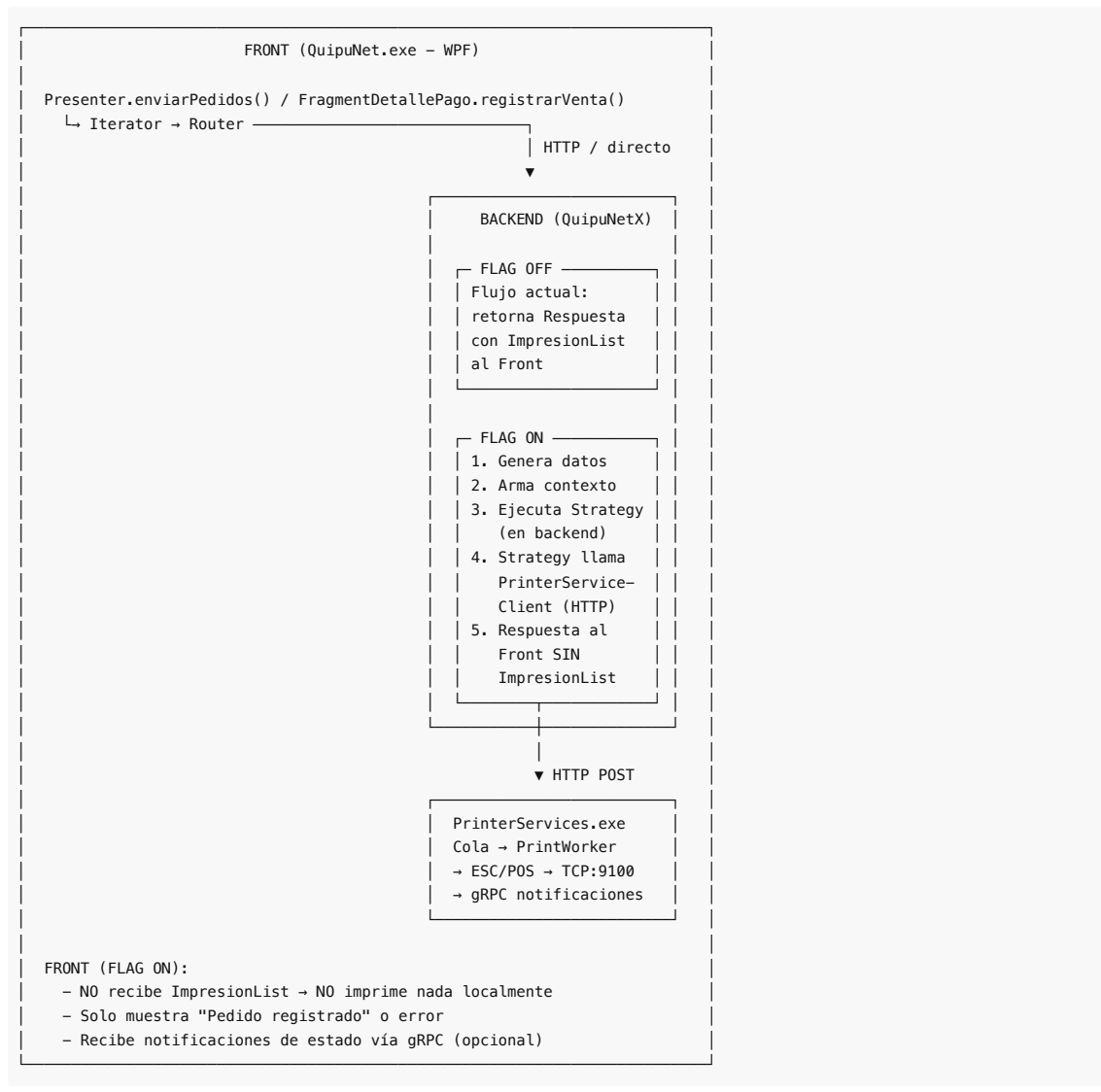
var printer = getPrinterObjectByModel(printermodel);
printer.init(impresion);
printer.setLetterSize(impresoraObj.Impresora_tamano);

if (NUEVO_FORMATO_COMANDA_ETHERNET_MEJORADA):
    htmlInput = CadenaHTML ?? Cadena
    Bitmap bmp = HtmlBitmapRenderer.RenderSimpleHtmlAsBitmap(htmlInput)
    Bitmap resized = ResizeIfNeeded(bmp, 576)
    printer.PrintBitmap(resized) // ← BITMAP PATH
else:
    printer.printString(Cadena) // ← TEXT PATH

printer.feedAndCut(2)
printer.imprimir() → conectar(commands.ByteArray, 9100, true) → TCP socket

```

3. Arquitectura nueva (CON PrinterServices) — Camino A



4. Estrategias de impresión — Inventario y replicación al backend

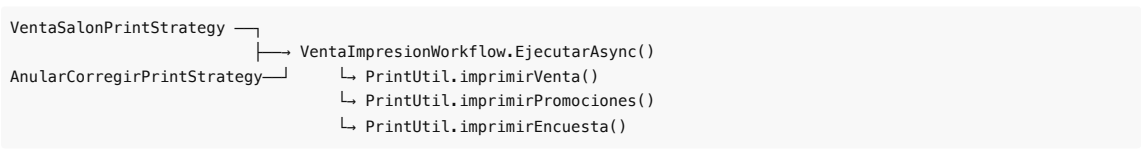
4.1 Todas las IPrintStrategy actuales (en Front)

#	Estrategia	Archivo Front	Llama a PrintUtil.*	Usa Iterators
1	ComandasPrintStrategy	Strategies/ComandasPrintStrategy.cs	imprimirComandasEthernet()	No
2	VentaRapidaPrintStrategy	Strategies/VentaRapidaPrintStrategy.cs	imprimirVenta(), imprimirComandasEthernet(), imprimirPromociones(), imprimirEncuesta(), imprimirPrecuenta()	VentaIterator, AjusteConfiguracionIterator
3	VentaSalonPrintStrategy	Strategies/VentaSalonPrintStrategy.cs	Delega a VentaImpresionWorkflow	VentaIterator, AjusteConfiguracionIterator
4	DeliveryPrintStrategy	Strategies/DeliveryPrintStrategy.cs	imprimirVenta(), imprimirPromociones(), imprimirEncuesta(), imprimirPrecuenta()	AjusteConfiguracionIterator, VentaIterator
5	VentaProveedorPrintStrategy	Strategies/VentaProveedorPrintStrategy.cs	imprimirVenta()	AjusteConfiguracionIterator, VentaIterator
6	AnularCorregirPrintStrategy	Strategies/AnularCorregirPrintStrategy.cs	Delega a VentaImpresionWorkflow + ReimprimirAsync()	Indirecto vía Workflow

4.2 Métodos de PrintUtil afectados

Método	Estrategias que lo llaman	Tipo
imprimirComandasEthernet()	Comandas, VentaRapida	Comandas
imprimirVenta()	VentaRapida, VentaSalon, Delivery, VentaProveedor, AnularCorregir	Comprobantes
imprimirPromociones()	VentaRapida, VentaSalon, Delivery	Sorteo
imprimirEncuesta()	VentaRapida, VentaSalon, Delivery	Encuesta
imprimirPrecuenta()	VentaRapida, Delivery	Motorizado

4.3 Workflow compartido



4.4 Qué pasa con cada estrategia en backend (Camino A)

En la versión backend, cada estrategia:

- Donde llamaba `PrintUtil.imprimirX()` → llama `PrinterServiceClient.EnviaX()`
- Donde llamaba `VentaIterator.reImprimir()` → llama `VentaController.reImprimir()` directo
- Donde llamaba `AjusteConfiguracionIterator.traerAjuste()` → llama `ConfiguracionLocalpos.findById()` directo
- Donde usaba callbacks UI → simplemente retorna resultado (sin UI)
- Donde llamaba `Util.setImpresion()` → no aplica (PrinterServices maneja reintentos)

5. Mapa de dependencias: Front → Backend

5.1 Hallazgo clave: Los Iterators son wrappers triviales

Los Routers en QuipuNetX ya tienen lógica dual:

```
// VentaRouter.reImprimir() - YA EXISTE EN QuipuNetX
if (Util.esModoServidor())
    return VentaController.reImprimir(venta_id, copiaSimple); // ← DIRECTO
else
    VentaClient.reImprimir(...); // ← HTTP al servidor (modo cliente)
```

Los Iterators del Front (`VentaIterator` , `AjusteConfiguracionIterator`) son wrappers que llaman a Routers que llaman a Controllers. En el backend, llamamos a los Controllers directamente.

5.2 Tabla completa de dependencias

Componente en Front	Vive en	Equivalente directo en Backend	Estado
<code>VentaIterator.reImprimir()</code>	Front → Router → Controller	<code>VentaController.reImprimir()</code>	✔ Ya existe
<code>VentaIterator.ImprimirVentaRapida()</code>	Front → Router → Controller	<code>VentaController.reImprimir(id, PARAM_NO)</code>	✔ Ya existe
<code>AjusteConfiguracionIterator.traerAjuste()</code>	Front → Router → Controller	<code>ConfiguracionLocalpos.findById()</code>	✔ Ya existe
<code>ConfiguracionImpresionAdapter</code>	Front (llama a <code>Util.estaConfiguracionActivadaPOS()</code>)	Copiar a QuipuNetX (deps ya están ahí)	📋 Copiar (se mantiene en Front)
<code>ImpresionContext</code>	Front, data class con entities QuipuNetX	Copiar a QuipuNetX	📋 Copiar (se mantiene en Front)
<code>PrintResultDto + PrintErrorType</code>	Front, data class pura	Copiar a QuipuNetX	📋 Copiar (se mantiene en Front)
<code>IPrintStrategy</code>	Front, interfaz	Copiar a QuipuNetX	📋 Copiar (se mantiene en Front)
<code>ImpresionStrategyFactory</code>	Front, factory simple	Copiar a QuipuNetX	📋 Copiar (se mantiene en Front)
<code>VentaImpresionWorkflow</code>	Front, lógica de workflow	Copiar a QuipuNetX (adaptar)	📋 Copiar (se mantiene en Front)
<code>PrinterQueueManager</code>	QuipuNetX.PrinterManager	Ya está en backend	✔ Ya existe
<code>Util.setImpresion()</code>	QuipuNetX.util.Util	Ya está en backend	✔ Ya existe
<code>RestaurantpeApplication.Instance</code>	QuipuNetX.applications	Ya está en backend	✔ Ya existe
<code>NLog</code>	Front	Usar logging de QuipuNetX	🔄 Adaptar
<code>Application.Current.Dispatcher (UI)</code>	Front WPF	NO APLICA — backend no tiene UI	✖ Eliminar
Popups de confirmación	Front WPF	NO APLICA — backend no pide confirmación	✖ Eliminar

5.3 Conclusión

El 90% de las dependencias ya están en QuipuNetX. Solo hay que:

- 1. **Copiar** 7 clases puras al backend (data classes + interfaz + factory + adapter + workflow) — las originales **se mantienen en el Front** para el flujo con FLAG OFF
- 2. Crear adaptaciones backend de 6 estrategias (Iterator→Controller, PrintUtil→PrinterServiceClient, sin UI)
- 3. Crear 1 clase nueva (PrinterServiceClient)

6. Feature Flag — Diseño dual (Front + Backend)

6.1 ¿Por qué en AMBOS lados?

- 1. **Backend (QuipuNetX):** Si `USAR_PRINTER_SERVICE=true` :
 - `PedidoController.addLista()` → ejecuta estrategia de comandas en backend → envía a `PrinterServices` → **NO retorna `ImpresionList`**
 - `VentaController` (al cobrar) → ejecuta estrategia de comprobante en backend → envía a `PrinterServices` → **NO retorna objetos de impresión**
- 2. **Front (QuipuNet.exe):** Si `USAR_PRINTER_SERVICE=true` :
 - `ListaPedidosTemporalesPresenter.enviarPedidos()` → **NO espera `ImpresionList`** , no llama `imprimirComandas()`
 - `FragmentDetallePago` (cobro) → **NO ejecuta `ImpresionService`** , no imprime nada

6.2 Mecanismo

Usa `FeatureFlagConfigReader` existente en `QuipuNetX/SQLite/FeatureFlagConfigReader.cs` :

- Archivo: `%LOCALAPPDATA%\QuipuNet\config fla.cfg`
- Formato: `USAR_PRINTER_SERVICE=true`
- Lectura: `FeatureFlagConfigReader.IsEnabled("USAR_PRINTER_SERVICE")`
- Solo servidor: `Util.esModoServidor() && FeatureFlagConfigReader.IsEnabled("USAR_PRINTER_SERVICE")`

6.3 Agregar a flags definidos

```
// En FeatureFlagConfigReader.GetAllDefinedFlags()
{ "USAR_PRINTER_SERVICE", "Delega impresión al servicio PrinterServices.exe (solo servidor)" }
```

7. Clases a copiar al Backend (QuipuNetX) — Front las conserva

7.1 Namespace nuevo: `QuipuNetX.Services.Print`

Crear carpeta `QuipuNetX/Services/Print/` con la siguiente estructura. Las clases originales se mantienen intactas en el Front (namespace `QuipuNet.Services.Print`) para el flujo con FLAG OFF.

```
QuipuNetX/Services/Print/
├─ ImpresionContext.cs           ← copiar desde Front (cambiar namespace)
├─ PrintResultDto.cs            ← copiar desde Front (cambiar namespace)
├─ ConfiguracionImpresionAdapter.cs ← copiar desde Front (cambiar namespace, deps ya en QuipuNetX)
├─ IPrintStrategy.cs            ← copiar desde Front (cambiar namespace)
├─ ImpresionStrategyFactory.cs   ← copiar desde Front (cambiar namespace)
├─ PrinterServiceClient.cs       ← NUEVO: cliente HTTP a PrinterServices
├─ ServerImpresionService.cs     ← NUEVO: orquestador en backend
├─ Strategies/
│   ├── ServerComandasStrategy.cs ← adaptación de ComandasPrintStrategy
│   ├── ServerVentaRapidaStrategy.cs ← adaptación de VentaRapidaPrintStrategy
│   ├── ServerVentaSalonStrategy.cs ← adaptación de VentaSalonPrintStrategy
│   ├── ServerDeliveryStrategy.cs ← adaptación de DeliveryPrintStrategy
│   ├── ServerVentaProveedorStrategy.cs ← adaptación de VentaProveedorPrintStrategy
│   └─ ServerAnularCorregirStrategy.cs ← adaptación de AnularCorregirPrintStrategy
└─ Workflows/
    └─ ServerVentaImpresionWorkflow.cs ← adaptación de VentaImpresionWorkflow
```

7.2 Clases que se copian al backend (cambio de namespace, Front las conserva)

IMPORTANTE: Estas clases se **COPIAN** al backend, NO se eliminan del Front. El Front las sigue usando cuando `USAR_PRINTER_SERVICE=false` o en máquinas CLIENTE. Existirán dos versiones: `QuipuNet.Services.Print.*` (Front) y `QuipuNetX.Services.Print.*` (Backend).

Clase	Motivo de copia	Cambio en la copia
ImpresionContext	Data class pura, solo usa entities de QuipuNetX	Solo cambio de namespace
PrintResultDto + PrintErrorType	Data class pura, sin dependencias externas	Solo cambio de namespace
IPrintStrategy	Interfaz con 2 miembros (NombreEstrategia, EjecutarAsync)	Solo cambio de namespace
ConfiguracionImpresionAdapter	Singleton, todas sus llamadas ya están en QuipuNetX	Solo cambio de namespace
ImpresionStrategyFactory	Factory switch-case, sin dependencias externas	Adaptar para crear Server*Strategy en vez de las del Front

7.3 Clases que se crean NUEVAS

Clase	Responsabilidad
PrinterServiceClient	Singleton HTTP client que envía jobs a PrinterServices
ServerImpresionService	Orquestador: crea contexto + ejecuta estrategia (equivalente a ImpresionService del Front pero sin UI)

8. API REST de PrinterServices — Endpoints disponibles

PrinterServices expone una API HTTP REST en el puerto `8090` (configurable). Todos los endpoints están definidos en `ApiRouter.cs` .

8.1 Endpoints de impresión

Método	Endpoint	Descripción	Body	Response
POST	/api/print/comanda	Imprime una sola comanda	JSON objeto	{ status: "OK", jobs: [{job_id, pedido_ids}] }
POST	/api/print/comandas	Imprime múltiples comandas	JSON array	{ status: "OK", jobs: [{job_id, pedido_ids}, ...] }
POST	/api/print/venta	Imprime comprobante de venta	JSON objeto	{ status: "OK", jobs: [{job_id, pedido_ids}] }
POST	/api/print/precuenta	Imprime precuenta	JSON objeto	{ status: "OK", jobs: [{job_id, pedido_ids}] }

Usado por QuipuNetX: POST /api/print/comandas (línea 70 de PrinterServiceClient.cs)

8.2 Endpoints de estado de impresoras

Método	Endpoint	Descripción	Response
GET	/api/printer/status	Lista todas las impresoras registradas	JSON array
GET	/api/printer/status/{id}	Estado de una impresora específica	JSON objeto
POST	/api/printer/register	Registra una nueva impresora	{ impresora_id, impresora_ip, ... }

8.3 Endpoints de trabajos de impresión

Método	Endpoint	Descripción	Response
GET	/api/jobs/pending	Lista trabajos pendientes	JSON array
GET	/api/job/{jobId}	Detalle de un trabajo específico	JSON objeto
POST	/api/job/{jobId}/retry	Reintentar trabajo fallido	{ status: "OK" }

8.4 Endpoints de configuración

Método	Endpoint	Descripción
GET	/api/config	Todas las configuraciones
GET	/api/config/{key}	Configuración específica
GET	/api/config/category/{category}	Configuraciones por categoría
PUT	/api/config	Actualizar una configuración
PUT	/api/config/batch	Actualizar múltiples configuraciones
POST	/api/config/{key}/reset	Resetear configuración a default
POST	/api/config/reset-all	Resetear todas a defaults

8.5 Endpoints de salud

Método	Endpoint	Descripción	Response
GET	/api/health	Health check del servicio	{ status: "OK", timestamp, db_path }

8.6 Formato del body para /api/print/comandas

```
[
{
  "impresora_id": "IMP001",
  "impresora_ip": "192.168.1.100",
  "impresora": "Cocina Principal",
  "impresora_nombre": "Cocina Principal",
  "printermodel": "TM-T20II",
  "impresora_modoopresion": "ethernet",
  "cadena": "linea1\nlinea2\n...",
  "cadenaHTML": "<html>...</html>",
  "tipo": "comanda",
  "device_id_origen": "POS-001",
  "ip_origen": "192.168.1.50",
  "uniqueid": "CMD-2024-001",
  "areaproduccion_numerocopias": "2",
```

```
        "impresora_tamanolettra": "normal",
        "impresora_tipogeneracion": "1",
        "codigo corte": "29,86",
        "abregaveta": "0",
        "cash_drawer_code": "",
        "pedido_ids": ["PED-001", "PED-002"],
        "lineas imprimir": [
            { "texto": "Mesa 5", "estilo": "bold" },
            { "texto": "2x Lomo Saltado", "estilo": "normal" }
        ]
    }
}
```

Campos OBLIGATORIOS (PrinterServices rechaza job si falta):

- impresora_ip : IP de la impresora (ej: "192.168.1.100") — PrintController.cs línea 78-82 valida que NO esté vacío

Campos importantes (búsqueda en PrintController.cs):

- impresora o impresora_nombre : Línea 129 busca **PRIMERO** "impresora" , luego "impresora_nombre"
- impresora_ip o impresoraIP : Línea 128 busca ambas variantes
- tipo o tipoimpresion : Línea 134 busca ambas variantes

IMPORTANTE:

- PrinterServiceClient.EnviarComandasAsync() transforma IList<Impresion> a este formato JSON array antes de enviarlo
- EnriquecerImpresion() valida que impresora_ip NO esté vacío antes de agregar al array (línea 218-223)
- Si falta impresora_ip , el job se omite y se loguea con Util.Capture()

9. Adaptación detallada de cada estrategia

8.1 Patrones de adaptación (aplican a TODAS)

Patrón 1: Iterator → Controller directo

```
FRONT:  VentaIterator.reImprimir(ventaId, callback)
        → VentaRouter.reImprimir()
        → VentaController.reImprimir()

BACKEND: Respuesta resp = VentaController.reImprimir(ventaId, copiaSimple) // DIRECTO
```

Patrón 2: AjusteConfiguracionIterator → BD directa

```
FRONT:  AjusteConfiguracionIterator.traerAjuste(CONFIG_ID, callback)
        → ConfiguracionlocalposRouter.getConfiguracionlocalpos()
        → ConfiguracionlocalposController

BACKEND: var config = Configuracionlocalpos.findById(CONFIG_ID) // DIRECTO
```

Patrón 3: PrintUtil → PrinterServiceClient

```
FRONT:  await PrintUtil.imprimirComandasEthernet(impresionList, callback)
        → ProcesarModoEthernet() → ESCPOSPrinterEthernet → TCP:9100

BACKEND: var resultado = await PrinterServiceClient.Instance
        .EnviarComandasAsync(impresionList) // HTTP POST a PrinterServices
```

Patrón 4: Sin UI

```
FRONT:  tcs.TrySetResult(PrintResultDto.CreateError(...))
        Util.setImpresion(impresion) // guardar para reintento manual UI

BACKEND: return PrintResultDto.CreateError(...) // retornar directo
        // PrinterServices maneja reintentos automáticos (cola SQLite)
```

8.2 ServerComandasStrategy

Complejidad: 🌟 Baja — la más simple

Cambio	De (Front)	A (Backend)
Imprimir directo	PrintUtil.imprimirComandasEthernet()	PrinterServiceClient.EnviarComandasAsync()

Cola impresión	PrinterQueueManager.Enqueue()	PrinterServiceClient.EnviarComandasAsync() (PS tiene cola propia)
Logging	NLog	QuipuNetX logging

8.3 ServerVentaRapidaStrategy

Complejidad: ☆☆☆ Alta — la más compleja (5 tipos de impresión)

Cambio	De (Front)	A (Backend)
Comprobante	PrintUtil.imprimirVenta()	PrinterServiceClient.EnviarVentaAsync()
Comandas	PrintUtil.imprimirComandasEthernet()	PrinterServiceClient.EnviarComandasAsync()
Sorteo	PrintUtil.imprimirPromociones()	PrinterServiceClient.EnviarPromocionAsync()
Encuesta	PrintUtil.imprimirEncuesta()	PrinterServiceClient.EnviarEncuestaAsync()
Motorizado	PrintUtil.imprimirPrecuenta()	PrinterServiceClient.EnviarPrecuentaAsync()
Delivery reprint	VentaIterator.ImprimirVentaRapida()	VentaController.reImprimir(id, PARAM_NO)
Doble comprobante	AjusteConfiguracionIterator.traerAjuste()	ConfiguracionLocalpos.findById()
Reimpresión	VentaIterator.reImprimir()	VentaController.reImprimir()
Reintento UI	Util.setImpresion()	No aplica

8.4 ServerVentaSalonStrategy

Complejidad: ☆☆ Baja — delega a ServerVentImpresionWorkflow

8.5 ServerDeliveryStrategy

Complejidad: ☆☆☆ Media — similar a VentaRapida pero sin comandas

Cambio	De (Front)	A (Backend)
Comprobante	PrintUtil.imprimirVenta()	PrinterServiceClient.EnviarVentaAsync()
Sorteo	PrintUtil.imprimirPromociones()	PrinterServiceClient.EnviarPromocionAsync()
Encuesta	PrintUtil.imprimirEncuesta()	PrinterServiceClient.EnviarEncuestaAsync()
Motorizado	PrintUtil.imprimirPrecuenta()	PrinterServiceClient.EnviarPrecuentaAsync()
Doble comprobante	AjusteConfiguracionIterator	ConfiguracionLocalpos.findById()
Reimpresión	VentaIterator.reImprimir()	VentaController.reImprimir()

8.6 ServerVentaProveedorStrategy

Complejidad: ☆☆☆ Media — solo comprobante con CUFE/UUID

Cambio	De (Front)	A (Backend)
Comprobante	PrintUtil.imprimirVenta()	PrinterServiceClient.EnviarVentaAsync()
Doble comprobante	AjusteConfiguracionIterator	ConfiguracionLocalpos.findById()
Reimpresión	VentaIterator.reImprimir()	VentaController.reImprimir()

8.7 ServerAnularCorregirStrategy

Complejidad: ☆☆☆ Baja — delega a ServerVentImpresionWorkflow + reimpresión

8.8 ServerVentImpresionWorkflow

Complejidad: ☆☆☆ Media — workflow compartido por Salón y AnularCorregir

Cambio	De (Front)	A (Backend)
Comprobante	PrintUtil.imprimirVenta()	PrinterServiceClient.EnviarVentaAsync()
Sorteo	PrintUtil.imprimirPromociones()	PrinterServiceClient.EnviarPromocionAsync()
Encuesta	PrintUtil.imprimirEncuesta()	PrinterServiceClient.EnviarEncuestaAsync()

Doble comprobante	AjusteConfiguracionIterator	Configuracionlocalpos.findById()
Reimpresión	VentaIterator.reImprimir()	VentaController.reImprimir()
TaskCompletionSource	callbacks async	llamadas directas con await

9. PrinterServiceClient — Cliente HTTP (NUEVO en QuipuNetX)

9.1 Ubicación

QuipuNetX/Services/Print/PrinterServiceClient.cs

9.2 Rol simplificado

El **único rol** de `PrinterServiceClient` es **enviar** las `Impresion` a `PrinterServices.exe` vía HTTP. **NO** orquesta impresión, **NO** maneja reintentos, **NO** espera a que se imprima.

`PrinterServices.exe` **solo responde** si logró registrar el Job en su base de datos SQLite, devolviendo los `job_id` asociados a los `pedido_ids`.

9.3 Responsabilidades concretas

- Singleton
- HTTP POST a `PrinterServices` (varios endpoints)
- Descubre IP vía `config fla. cfg` o UDP Discovery o `localhost:8090`
- Enriquece cada `Impresion` con datos de `Impresora` (BD local) antes de enviar
- Serializa `IList<Impresion>` → JSON compatible con `PrintController.ParsePrintJob()`
- Envía y **recibe respuesta** con `job_id ↔ pedido_ids` (ver §9.6)
- Timeout: 10s
- Si `PrinterServices` no responde → retorna `Respuesta` tipo ERROR

9.4 Métodos públicos

Método	Endpoint <code>PrinterServices</code>	Equivale a <code>PrintUtil.*</code>
<code>EnviarComandasAsync(IList<Impresion>)</code>	POST <code>/api/print/comandas</code>	<code>imprimirComandasEthernet()</code>
<code>EnviarVentaAsync(Impresion)</code>	POST <code>/api/print/venta</code>	<code>imprimirVenta()</code>
<code>EnviarPromocionAsync(Impresion)</code>	POST <code>/api/print/comanda</code>	<code>imprimirPromociones()</code>
<code>EnviarEncuestaAsync(Impresion)</code>	POST <code>/api/print/comanda</code>	<code>imprimirEncuesta()</code>
<code>EnviarPrecuentaAsync(Impresion)</code>	POST <code>/api/print/precuenta</code>	<code>imprimirPrecuenta()</code>
<code>IsAvailableAsync()</code>	GET <code>/api/health</code>	Health check

9.5 Enriquecimiento de Impresion

Antes de serializar cada `Impresion`, enriquecer con datos de `Impresora`:

```
private void EnriquecerImpresion(Impresion imp)
{
    Impresora obj = Impresora.findById(imp.Impresora_id);
    if (obj == null) return;
    imp.Impresora_tamanoqr = obj.Impresora_tamanoqr;
    imp.Abregaveta = obj.Impresora_abregaveta;
    imp.Cashdrawer = obj.Impresora_cashdrawer;
    imp.Printermodel = obj.Impresora_printermodel;
    imp.Tipoimpresora = obj.Impresora_tipo;
    imp.Impresora_tamanoletra = obj.Impresora_tamano;
}
```

9.6 Contrato de respuesta: `job_id ↔ pedido_ids`

Nueva propiedad en `Impresion.cs` (`QuipuNetX`)

```
// Impresion.cs – NUEVA propiedad (en implementación)
public string Pedidos_ids_printer_asociados { get; set; }
// Formato: "1772,1773,1774,1780" – IDs de pedidos que viajan en esta Impresión
```

Cada objeto `Impresion` contiene N pedidos internamente (una comanda puede tener varios pedidos). Al enviar a `PrinterServices`, **1 Impresion = 1 Job**.

Ejemplo concreto

```
QuipuNetX envía 2 objetos Impresion a PrinterServices:

Impresion1 (impresora Epson cocina):
  Pedidos_ids_printer_asociados = "1772,1773"
  → PrinterServices crea Job en SQLite → job_id = 714

Impresion2 (impresora Epson bar):
  Pedidos_ids_printer_asociados = "1771,1774"
  → PrinterServices crea Job en SQLite → job_id = 715
```

Respuesta HTTP de PrinterServices

PrinterServices **solo confirma** que registró los Jobs en SQLite. NO espera a que se impriman:

```
// POST /api/print/comandas → Response 200
{
  "status": "OK",
  "jobs": [
    { "job_id": "714", "pedido_ids": ["1772", "1773"] },
    { "job_id": "715", "pedido_ids": ["1771", "1774"] }
  ]
}
```

Mapeo HTTP Response → Respuesta.cs (QuipuNetX)

PrinterServiceClient parsea el JSON de PrinterServices y construye un objeto Respuesta usando sus propiedades reales (ver QuipuNetX/Util/Respuesta.cs):

Caso ÉXITO (HTTP 200 + status "OK"):

```
// PrinterServiceClient – ver implementación completa en §13.2:
var respuesta = new Respuesta();
respuesta.Tipo = Util.SUCCESS;
respuesta.Mensajes = new List<string> { "Imprimiendo " + tipoImpresion + "..."; // ver §13.3
respuesta.Data = jobsParsed; // List<PrintJobResult> con {JobId, PedidoIds[]}
respuesta.ImpresionList = stickersParaFront; // solo stickers (ver §14), vacío si no hay
// NOTA: SUCCESS = "se ha empezado a imprimir", NO "ya se imprimió" (ver §13.4)
```

Caso ERROR (HTTP 500, timeout, o status "ERROR"):

```
var respuesta = new Respuesta();
respuesta.Tipo = Util.ERROR;
respuesta.CodigoError = "PRINTER_SERVICE_UNAVAILABLE"; // o "PRINTER_SERVICE_REGISTER_FAILED"
respuesta.Mensajes = new List<string> { "No se pudo registrar la impresión en PrinterServices" };
respuesta.Data = null;
respuesta.ImpresionList = null; // sin impresiones para el Front
```

Propiedades de Respuesta.cs relevantes:

Propiedad Respuesta.cs	Tipo	Uso en este contexto
Tipo	string	Util.SUCCESS o Util.ERROR
CodigoError	string	"PRINTER_SERVICE_UNAVAILABLE" / "PRINTER_SERVICE_REGISTER_FAILED"
Mensajes	IList<string>	Mensajes informativos o de error
Data	object	List<PrintJobResult> CON {JobId, PedidoIds[]}
ImpresionList	IList<Impresion>	Solo stickers Jaltech para flujo local (ver §14)
ImpresionErrorList	IList<Impresion>	Impresiones que no se pudieron registrar
PedidoList	IList<Pedido>	No se usa en este flujo
Flag	bool	No se usa en este flujo

DTO auxiliar nuevo: PrintJobResult

```
// Clase nueva en QuipuNetX.Services.Print
public class PrintJobResult
```

```
{
    public string JobId { get; set; }
    public List<string> PedidoIds { get; set; }
}
```

Qué hace QuipuNetX con la Respuesta exitosa

```
// En ServerComandasStrategy, después de recibir Respuesta SUCCESS:
var jobs = respuesta.Data as List<PrintJobResult>;
foreach (var job in jobs)
{
    foreach (var pedidoId in job.PedidoIds)
    {
        PedidoPrintJobTracker.Asociar(pedidoId, job.JobId);
    }
}
// Retorna la Respuesta al caller (PedidoController → Front)
```

Si PrinterServices NO puede registrar el Job

```
// Response 500 o timeout
{
    "status": "ERROR",
    "message": "No se pudo registrar el job en la base de datos"
}
```

PrinterServiceClient construye `Respuesta` con `Tipo = Util.ERROR` y propaga al Front.

9.7 Mapeo Impresion → JSON

Propiedad Impresion (QuipuNetX)	Campo JSON (PrinterServices)
Impresora_id	impresora_id
Impresora_ip	impresora_ip
Impresora (nombre)	impresora_nombre
Printermodel	printermodel
Impresora_modoiimpresion	impresora_modoiimpresion
Cadena	cadena
cadenaHTML	cadenaHTML
Uniqueid	uniqueid
Tipo	tipo
Abregaveta	abregaveta
Cashdrawer	cashdrawer
Tipoimpresora	tipoimpresora
Impresora_tamanoiletra	impresora_tamanoiletra
Impresora_tamanoiqr	impresora_tamanoiqr
Areaproduccion_numerocopias	areaproduccion_numerocopias
lineasimprimir	lineasimprimir
impresora_tipogeneracion	impresora_tipogeneracion
Codigocorte	codigocorte
qrData	qrData
qrEncuesta	qrEncuesta
Pedidos_ids_printer_asociados	pedido_ids

9.8 Descubrimiento de IP

Prioridad:

1. config_fla.cfg → PRINTER_SERVICE_URL=http://192.168.1.10:8090
2. UDP Discovery (QUIPU_PRINTER_DISCOVERY → respuesta con IP)
3. Fallback: http://localhost:8090

10. Trazabilidad: job_id ↔ pedidos (notificación de fallos)

10.1 Flujo completo de trazabilidad

```
1. QuipuNetX envía Impresion[] a PrinterServices
   Cada Impresion tiene Pedidos_ids_printer_asociados = "1772,1773"

2. PrinterServices registra Jobs en SQLite
   Job 714 → pedido_ids: [1772, 1773]
   Job 715 → pedido_ids: [1771, 1774]

3. PrinterServices responde con job_id ↔ pedido_ids
   QuipuNetX almacena: pedido 1772 → job_id 714
                      pedido 1773 → job_id 714
                      pedido 1771 → job_id 715
                      pedido 1774 → job_id 715

4. PrinterServices IMPRIME (asíncrono, en su cola)
   Job 714 → ✅ SUCCESS → gRPC: NotifyPrintSuccess(job_id=714)
   Job 715 → ❌ FAILED  → gRPC: NotifyPrintFailed(job_id=715)

5. QuipuNet.exe recibe gRPC NotifyPrintFailed(job_id=715)
   → Busca pedidos asociados a job 715: [1771, 1774]
   → Marca pedidos 1771 y 1774 como "impresión fallida"
   → Muestra al usuario qué pedidos fallaron
```

10.2 Cambios necesarios en PrinterServices

PrintJob.cs — Nuevo campo

```
public string PedidoIds { get; set; } // "1772,1773" — viene del Impresion
```

PrintJobEntity.cs — Nueva columna

```
public string pedido_ids { get; set; } // TEXT en SQLite
```

PrintController.ParsePrintJob() — Leer pedido_ids

```
job.PedidoIds = GetString(json, "pedido_ids");
```

Respuesta del endpoint /api/print/comandas

Actualmente `PrintController` solo responde "OK". Debe cambiar para retornar los `job_id` generados:

```
// Después de Enqueue() cada job:
var jobResults = new List<object>();
foreach (var job in jobs)
{
    _jobManager.Enqueue(job);
    jobResults.Add(new {
        job_id = job.Id.ToString(),
        pedido_ids = (job.PedidoIds ?? "").Split(',')
    });
}
return JsonResponse(200, new { status = "OK", jobs = jobResults });
```

gRPC NotifyPrintFailed — Incluir job_id y pedido_ids

En `PrinterNotificationServiceImpl`, cuando se notifica un fallo, incluir:

- `job_id` (ya existe en la notificación)

- pedido_ids (nuevo: leer del Job para que QuipuNet sepa qué pedidos fallaron)

10.3 Cambios necesarios en QuipuNetX

Almacenar job_id por pedido

Después de recibir la respuesta de PrinterServices:

```
// Pseudocódigo en ServerComandasStrategy o PrinterServiceClient:
foreach (var jobResult in response.Jobs)
{
    foreach (var pedidoId in jobResult.PedidoIds)
    {
        // Guardar relación: pedido_id → job_id
        // Para que cuando llegue gRPC NotifyPrintFailed(job_id)
        // podamos saber qué pedidos fallaron
        PedidoPrintJobTracker.Asociar(pedidoId, jobResult.JobId);
    }
}
```

Recibir notificación gRPC de fallo

```
// En el listener gRPC del Front (QuipuNet.exe):
void OnPrintFailed(string jobId, string pedidoIds)
{
    // Marcar pedidos como "impresión fallida"
    foreach (var pedidoId in pedidoIds.Split(','))
    {
        MarcarPedidoComoFalloImpresion(pedidoId);
    }
}
```

10.4 Importante: PrinterServices NO espera a imprimir

La respuesta de POST /api/print/comandas se envía **inmediatamente después de registrar los Jobs en SQLite**. La impresión física ocurre **después**, en el `PrintWorker` (cola asíncrona). El resultado de la impresión llega vía **gRPC push** (NotifyPrintSuccess / NotifyPrintFailed).

11. Renderizado HTML → Bitmap (en PrinterServices)

11.1 Problema

`PrintUtil.ProcesarModoEthernet()` líneas 1206-1211 renderiza HTML a bitmap en QuipuNetX:

```
string htmlInput = preimpresion.CadenaHTML ?? preimpresion.Cadena;
using (Bitmap bmp = HtmlBitmapRenderer.RenderSimpleHtmlAsBitmap(htmlInput))
using (Bitmap resized = ResizeIfNeeded(bmp, 576))
    printer.PrintBitmap(resized);
```

11.2 Solución: Renderizar DENTRO de PrinterServices

El renderizado HTML → Bitmap se hace **dentro de PrinterServices**, NO en el backend QuipuNetX. El backend solo envía `cadenaHTML` como texto plano; PrinterServices renderiza localmente.

Ventajas:

- No se transmite base64 pesado en el JSON HTTP
- PrinterServices tiene control total del renderizado
- El backend se mantiene liviano (solo serializa texto)

11.3 Análisis de dependencias de HtmlBitmapRenderer.cs

La clase `HtmlBitmapRenderer` (QuipuNetX/Util/print/HtmlBitmapRenderer.cs) es **100% auto-contenida**:

Dependencia	Tipo	¿Disponible en PrinterServices?
System	.NET Framework	✅ Sí
System.Collections.Generic	.NET Framework	✅ Sí
System.Drawing	GDI+ (.NET Framework)	✅ Sí
System.Text.RegularExpressions	.NET Framework	✅ Sí

QuipuNetX.*	—	✗ NO se usa
Variables globales	—	✗ NO se usa
Util.*	—	✗ NO se usa

Resultado: Se puede copiar directamente sin modificaciones.

11.4 Archivos a crear en PrinterServices

Rendering/HtmlBitmapRenderrer.cs

Copia exacta de QuipuNetX/Util/print/HtmlBitmapRenderrer.cs cambiando solo el namespace:

```
using System;
using System.Collections.Generic;
using System.Drawing;
using System.Text.RegularExpressions;

namespace PrinterServices.Rendering
{
    public static class HtmlBitmapRenderrer
    {
        private class HtmlRun
        {
            public string Text;
            public int FontSize;
            public FontStyle Style;
            public HtmlRun(string text, int fontSize, FontStyle style)
            { Text = text; FontSize = fontSize; Style = style; }
        }

        // --- Método principal: RenderSimpleHtmlAsBitmap() ---
        // Copia EXACTA del método en QuipuNetX (465 líneas)
        // Parsea HTML simple (<h1>--<h5>, <b>, <i>, <br>)
        // Renderiza con GDI+ (System.Drawing) a 192 DPI
        // Retorna Bitmap con ancho máximo 584px

        // --- ParseLine() ---
        // Parsea tags HTML con pilas de fontSizeStack y styleStack

        // --- SplitRunToFitWidth() ---
        // Corta runs largos en múltiples líneas si exceden maxWidthPx
    }
}
```

NOTA: Se copia la clase completa (465 líneas). Solo se cambia `namespace QuipuNetX.util.print` → `namespace PrinterServices.Rendering`.

Rendering/BitmapResizer.cs

Copia del método `ResizeIfNeeded` de `PrintUtil.cs`:

```
using System.Drawing;

namespace PrinterServices.Rendering
{
    public static class BitmapResizer
    {
        public static Bitmap ResizeIfNeeded(Bitmap bmp, int maxWidth)
        {
            if (bmp.Width <= maxWidth)
                return bmp;

            float scale = (float)maxWidth / bmp.Width;
            int newHeight = (int)(bmp.Height * scale);
            Bitmap resized = new Bitmap(maxWidth, newHeight);
            resized.SetResolution(192, 192);

            using (Graphics g = Graphics.FromImage(resized))
            {
                g.Clear(Color.White);
            }
        }
    }
}
```

```

        g.DrawImage(bmp, 0, 0, maxWidth, newHeight);
    }

    return resized;
}
}
}

```

11.5 Integración en PrintWorker

PrintWorker usa HtmlBitmapRenderer cuando el Job tiene cadenaHTML :

```

// PrintWorker.ProcessJob():
if (!string.IsNullOrEmpty(job.ContenidoHtml))
{
    // Renderizar HTML → Bitmap localmente en PrinterServices
    using (Bitmap bmp = HtmlBitmapRenderer.RenderSimpleHtmlAsBitmap(job.ContenidoHtml))
    using (Bitmap resized = BitmapResizer.ResizeIfNeeded(bmp, 576))
    {
        builder.AddBitmapFromImage(resized); // ESC/POS GS v 0
    }
}
else
{
    builder.AddText(job.Contenido);
}

builder.FeedAndCut(2);
transport.Send(builder.Build());

```

11.6 Consecuencia: NO se envía BitmapBase64 en el JSON

El campo `bitmap_base64` **NO existe** en el JSON que viaja de QuipuNetX → PrinterServices. Solo viaja `cadenaHTML` (texto plano). PrinterServices renderiza localmente.

12. Cambios en el Front (QuipuNet.exe)

12.1 Cambios mínimos — el Front solo deja de imprimir

Con Camino A, el Front **no necesita nuevas estrategias ni lógica**. Solo necesita:

ListaPedidosTemporalesPresenter.enviarPedidos() — Línea ~50

PSEUDOCÓDIGO:

```

// Callback de PedidoIterator.enviarPedidos():
if (respuesta.Tipo == SUCCESS):
    // ... código existente mesa, cache, etc. ...

    if (USAR_PRINTER_SERVICE && ES_MOD0_SERVIDOR):
        // Backend ya ejecutó estrategia y envió a PrinterServices
        // Respuesta NO tiene ImpresionList
        view.showSuccessRegister()
    else:
        // Flujo actual: imprimir localmente
        if (respuesta.ImpresionList != null && respuesta.ImpresionList.Count > 0):
            imprimirComandas(respuesta.ImpresionList)
            view.showSuccessRegister()

```

12.2 Otros presenters que reciben ImpresionList

Aplicar la misma lógica de feature flag en TODOS los presenters que reciben `ImpresionList` :

Presenter	Método	Línea ref
ListaPedidosTemporalesPresenter	enviarPedidos()	~50
ListaPedidosTemporalesPresenter	enviarPedidosOld()	~181
ListaPedidosTemporalesPresenter	enviarPedidosSelfService()	~232

Otros presenters que llamen <code>imprimirComandas()</code>	Buscar con grep	—
---	-----------------	---

12.3 Presenters de cobro (ventas)

Para comprobantes, el feature flag también debe interceptar en los presenters de cobro:

Presenter / Fragment	Método	Qué hace hoy
FragmentDetallePago	<code>registrarVenta()</code> callback	Recibe Respuesta → arma <code>ImpresionContext</code> → <code>ImpresionService.ImprimirAsync()</code>
Otros fragments de cobro	Similar	Similar

Con FLAG ON: el callback **NO ejecuta** `ImpresionService` porque el backend ya envió todo a `PrinterServices`.

12.4 El Front mantiene sus estrategias (FLAG OFF)

Las estrategias existentes en el Front **NO se eliminan**. Siguen funcionando cuando:

- FLAG OFF → flujo actual completo
- Máquina CLIENTE → siempre flujo actual (`PrinterServices` solo corre en servidor)

13. Manejo de errores y reintentos

13.1 Principio: CERO excepciones no controladas

Regla: `PrinterServiceClient` **NUNCA lanza excepciones**. Todos sus métodos públicos retornan `Respuesta`. El caller simplemente evalúa `respuesta.Tipo` — sin try/catch.

Esto evita:

- Caídas de software por excepciones no controladas
- Try/catch obligatorio en cada Strategy que llame al cliente
- Código inmanejable con catches anidados

13.2 Método interno: `PostConReintentoAsync` (retorna `Respuesta`, no `HttpResponseMessage`)

```
// PrinterServiceClient — método interno que NUNCA lanza excepciones:
private const int MAX_INTENTOS = 2;
private const int TIMEOUT_MS = 10000; // 10 segundos por intento
private const int DELAY_ENTRE_REINTENTOS = 1000; // 1 segundo entre reintentos

private async Task<Respuesta> PostConReintentoAsync(string url, StringContent jsonContent,
                                                    string tipoImpresion)
{
    string ultimoError = "";

    for (int intento = 1; intento <= MAX_INTENTOS; intento++)
    {
        try
        {
            var cts = new CancellationTokenSource(TIMEOUT_MS);
            var httpResponse = await _httpClient.PostAsync(url, jsonContent, cts.Token);

            if (httpResponse.IsSuccessStatusCode)
            {
                // ✅ PrinterServices registró los jobs
                var json = await httpResponse.Content.ReadAsStringAsync();
                var jobsParsed = ParseJobResults(json);

                var respuesta = new Respuesta();
                respuesta.Tipo = Util.SUCCESS;
                respuesta.Mensajes = new List<string> { "Imprimiendo " + tipoImpresion + "..."};
                respuesta.Data = jobsParsed;
                return respuesta;
            }
        }
        else
        {
            // HTTP 500, 400, etc. — no reintentar (error de lógica, no de red)
            var respuesta = new Respuesta();
            respuesta.Tipo = Util.ERROR;
            respuesta.CodigoError = "PRINTER_SERVICE_REGISTER_FAILED";
            respuesta.Mensajes = new List<string>
```

```

        {
            "No se pudo registrar la impresión de " + tipoImpresion
            + " (HTTP " + (int)httpResponse.StatusCode + ")"
        };
        return respuesta;
    }
}
catch (Exception ex)
{
    // Cualquier excepción de red/timeout → capturar y reintentar
    ultimoError = ex.Message;

    if (intento < MAX_INTENTOS)
    {
        await Task.Delay(DELAY_ENTRE_REINTENTOS);
        continue; // Reintentar
    }
    // Último intento falló → NO lanzar, retornar Respuesta ERROR
}
}

// Agotados los reintentos – retornar ERROR (NUNCA throw)
var respuestaFinal = new Respuesta();
respuestaFinal.Tipo = Util.ERROR;
respuestaFinal.CodigoError = "PRINTER_SERVICE_UNAVAILABLE";
respuestaFinal.Mensajes = new List<string>
{
    "No hay servicio de impresión activo (PrinterServices no responde después de "
    + MAX_INTENTOS + " intentos: " + ultimoError + ")"
};
return respuestaFinal;
}

```

13.3 Método público: EnviarComandasAsync — SIN try/catch

El caller **no necesita try/catch** porque `PostConReintentoAsync` siempre retorna `Respuesta` :

```

public async Task<Respuesta> EnviarComandasAsync(IList<Impresion> impresionList,
                                                string tipoImpresion = "comandas")
{
    // 1. Enriquecer impresiones con datos de Impresora
    foreach (var imp in impresionList)
        EnriquecerImpresion(imp);

    // 2. Serializar a JSON
    var jsonContent = SerializarImpresiones(impresionList);

    // 3. Enviar con reintento – SIEMPRE retorna Respuesta, NUNCA lanza
    var respuesta = await PostConReintentoAsync(
        _baseUrl + "/api/print/comandas",
        jsonContent,
        tipoImpresion
    );

    // 4. Si hay stickers, agregarlos al ImpresionList de la respuesta
    if (respuesta.Tipo == Util.SUCCESS)
        respuesta.ImpresionList = stickersParaFront; // solo stickers (ver §14)

    return respuesta;
}

```

13.4 Uso desde las Strategies — limpio, sin try/catch

```

// En ServerComandasStrategy.EjecutarAsync():
var respuesta = await PrinterServiceClient.Instance
    .EnviarComandasAsync(context.ImpresionComandas, "comandas");

// Sin try/catch – evaluar directo:
if (respuesta.Tipo == Util.SUCCESS)
{

```

```

    var jobs = respuesta.Data as List<PrintJobResult>;
    foreach (var job in jobs)
        foreach (var pedidoId in job.PedidoIds)
            PedidoPrintJobTracker.Asociar(pedidoId, job.JobId);
}

return respuesta; // Propagar al caller (SUCCESS o ERROR, nunca excepción)

```

13.5 Mensajes dinámicos por Strategy

Cada Strategy invoca al cliente con su `tipoImpresion` :

Strategy	tipoImpresion	Mensaje Respuesta.Mensajes
ServerComandasStrategy	"comandas"	"Imprimiendo comandas..."
ServerVentaRapidaStrategy	"comprobante de venta"	"Imprimiendo comprobante de venta..."
ServerVentaSalonStrategy	"comprobante de salón"	"Imprimiendo comprobante de salón..."
ServerDeliveryStrategy	"comprobante de delivery"	"Imprimiendo comprobante de delivery..."
ServerVentaProveedorStrategy	"factura electrónica"	"Imprimiendo factura electrónica..."
ServerAnularCorregirStrategy	"anulación/corrección"	"Imprimiendo anulación/corrección..."
(sorteo)	"sorteo"	"Imprimiendo sorteo..."
(encuesta)	"encuesta"	"Imprimiendo encuesta..."
(precuenta)	"precuenta delivery"	"Imprimiendo precuenta delivery..."

13.6 Propagación al Front — Semántica: "se ha empezado a imprimir"

IMPORTANTE: La Respuesta con Tipo = SUCCESS que llega al Front **NO significa que ya se imprimió**. Significa que **PrinterServices** recibió y registró los **jobs** — la impresión física es asíncrona.

Caso ÉXITO → "Se ha empezado a imprimir"

Respuesta al Front:

```

Tipo           = "SUCCESS"                // Respuesta.Tipo
Mensajes       = ["Imprimiendo comandas..."] // Respuesta.Mensajes
Data           = List<PrintJobResult>      // Respuesta.Data (job_id ↔ pedido_ids)
ImpresionList  = [stickers] o vacío         // Respuesta.ImpresionList
Flag           = false                    // Respuesta.Flag (default)

```

- El Front sabe que la impresión EMPEZÓ (jobs registrados en cola)
- El resultado FINAL llega por gRPC: `NotifyPrintSuccess` / `NotifyPrintFailed`

Caso ERROR → "No se pudo iniciar impresión"

Respuesta al Front:

```

Tipo           = "ERROR"                // Respuesta.Tipo
CodigoError    = "PRINTER_SERVICE_UNAVAILABLE" // Respuesta.CodigoError
Mensajes       = ["No hay servicio de impresión activo (PrinterServices no responde
                  después de 2 intentos)"] // Respuesta.Mensajes
Data           = null                  // Respuesta.Data
ImpresionList  = null                  // Respuesta.ImpresionList

```

- El Front muestra el error al usuario

13.5 NO hay fallback automático

Si **PrinterServices** está caído, **NO se imprime localmente como fallback**. Razones:

- Simplicidad: evita flujos mixtos difíciles de depurar
- Consistencia: el operador debe saber que **PrinterServices** no está corriendo
- **PrinterServices** se instala como Windows Service con arranque automático

14. Exclusiones — ImprimirStickerJaltech

`ImprimirStickerJaltech()` (`PrintUtil.cs`) se excluye de esta integración:

```
if (preimpresion.Tipoimpresora == Definitions.TIPOIMPRESORA_STICKER)
{
    var resultado = ImprimirStickerJaltech(impresoraObj, preimpresion);
}
```

Los stickers Jaltech usan un flujo diferente con driver propio. En el backend, antes de enviar a PrinterServices, se filtran:

```
foreach Impresion imp in impresionList:
    if (imp.Tipoimpresora == TIPOIMPRESORA_STICKER):
        stickersParaFront.Add(imp) // incluir en Respuesta.ImpresionList
    else:
        comandasParaServicio.Add(imp) // enviar a PrinterServices
```

Si hay stickers, la Respuesta INCLUYE un `ImpresionList` parcial (solo stickers) para que el Front los imprima localmente.

15. Campos NUEVOS en PrinterServices

15.1 PrintJob.cs — Campos adicionales

Campo	Tipo	Descripción
ImpresoraTamano	string	setLetterSize("1","2","3")
ImpresoraTamanoQr	string	Tamaño QR
CashDrawerCode	string	Código de cash drawer
PedidoIds	string	IDs de pedidos asociados "1772,1773" (ver §10)

NOTA: `BitmapBase64` NO es campo de `PrintJob`. `PrinterServices` renderiza el bitmap localmente a partir de `ContenidoHtml` usando `HtmlBitmapRenderer` (ver §11).

15.2 PrintWorker — Lógica de renderizado (actualizada con §11)

```
PSEUDOCÓDIGO – ProcessJob():

if (job.ContenidoHtml != null):
    // Renderizado HTML → Bitmap LOCAL en PrinterServices (ver §11.5)
    Bitmap bmp = HtmlBitmapRenderer.RenderSimpleHtmlAsBitmap(job.ContenidoHtml)
    Bitmap resized = BitmapResizer.ResizeIfNeeded(bmp, 576)
    builder.AddBitmapFromImage(resized)
    bmp.Dispose(); resized.Dispose()
else:
    builder.AddText(job.Contenido)

builder.FeedAndCut(2)
transport.Send(builder.Build())
```

16. Archivos a crear/modificar — Resumen completo

16.1 Backend (QuipuNetX) — Archivos NUEVOS

Archivo	Descripción
Services/Print/ImpresionContext.cs	Copiar desde Front (se mantiene en Front, cambiar namespace)
Services/Print/PrintResultDto.cs	Copiar desde Front (se mantiene en Front, cambiar namespace)
Services/Print/ConfiguracionImpresionAdapter.cs	Copiar desde Front (se mantiene en Front, cambiar namespace)
Services/Print/IPrintStrategy.cs	Copiar desde Front (se mantiene en Front, cambiar namespace)
Services/Print/ImpresionStrategyFactory.cs	Copiar desde Front (se mantiene en Front, adaptar para Server*)
Services/Print/PrinterServiceClient.cs	NUEVO: cliente HTTP a PrinterServices
Services/Print/PrintJobResult.cs	NUEVO: DTO {JobId, PedidoIds[]} para respuesta de PrinterServices
Services/Print/ServerImpresionService.cs	NUEVO: orquestador backend
Services/Print/Strategies/ServerComandasStrategy.cs	Adaptación

Services/Print/Strategies/ServerVentaRapidaStrategy.cs	Adaptación
Services/Print/Strategies/ServerVentaSalonStrategy.cs	Adaptación
Services/Print/Strategies/ServerDeliveryStrategy.cs	Adaptación
Services/Print/Strategies/ServerVentaProveedorStrategy.cs	Adaptación
Services/Print/Strategies/ServerAnularCorregirStrategy.cs	Adaptación
Services/Print/Workflows/ServerVentaImpresionWorkflow.cs	Adaptación

16.2 Backend (QuipuNetX) — Archivos MODIFICADOS

Archivo	Cambio
controller/PedidoController.cs	En addLista(): if FLAG → ejecutar ServerComandasStrategy
controller/VentaController.cs	En cobro: if FLAG → ejecutar Server*Strategy
SQLite/FeatureFlagConfigReader.cs	Agregar USAR_PRINTER_SERVICE a flags definidos

16.3 Front (QuipuNet.exe) — Archivos MODIFICADOS

Archivo	Cambio
ListaPedidosTemporalesPresenter.cs	enviarPedidos(): if FLAG → no esperar ImpresionList
FragmentDetallePago (y similares)	Si FLAG → no ejecutar ImpresionService

16.4 PrinterServices — Archivos NUEVOS

Archivo	Descripción
Rendering/HtmlBitmapRenderrer.cs	Copia de QupuNetX (465 líneas, solo cambio namespace) — ver §11.4
Rendering/BitmapResizer.cs	Método ResizeIfNeeded — ver §11.4

16.5 PrinterServices — Archivos MODIFICADOS

Archivo	Cambio
Queue/PrintJob.cs	Campos: PedidoIds, CashDrawerCode, etc. (sin BitmapBase64)
Data/Models/PrintJobEntity.cs	Columnas correspondientes
Api/Controllers/PrintController.cs	Parsear nuevos campos + respuesta con job_id ↔ pedido_ids (ver §10)
Workers/PrintWorker.cs	Renderizado HTML→Bitmap local (ver §11.5)
Drivers/EscPosCommandBuilder.cs	Método AddBitmapFromImage(Bitmap) → comandos GS v 0

17. Secuencia de implementación

Fase 7A: Comandas (MVP)

1.

✓

[PrinterServices] Copiar HtmlBitmapRenderrer.cs → Rendering/ (cambiar namespace)

2.

✓

[PrinterServices] Crear BitmapResizer.cs en Rendering/

3.

✓

[PrinterServices] Agregar campos nuevos a PrintJob, PrintJobEntity, ParsePrintJob (PedidoIds, CashDrawerCode – SIN BitmapBase64)

4.

✓

[PrinterServices] Implementar AddBitmapFromImage(Bitmap) en EscPosCommandBuilder

5.

✓

[PrinterServices] Actualizar PrintWorker: ContenidoHtml → HtmlBitmapRenderrer → Bitmap

6.

✓

[PrinterServices] Actualizar PrintController: respuesta con job_id ↔ pedido_ids

7.

✓

[PrinterServices] Build y verificar 0 errores – System.Drawing agregado al .csproj

8.

✓

[QuipuNetX] Copiar clases base al backend (se mantienen en Front): ImpresionContext, PrintResultDto, IPrintStrategy, ConfiguracionImpresionAdapter, ImpresionStrategyFactory (cambiar namespace a QupuNetX.Services.Print)

9.

✓

[QuipuNetX] Crear PrinterServiceClient.cs (con retry 2 intentos, mensajes dinámicos)

10.

✓

[QuipuNetX] Crear PrintJobResult.cs (DTO)

11.

✓

[QuipuNetX] Crear ServerComandasStrategy.cs

12.

✓

[QuipuNetX] Crear ServerImpresionService.cs (orquestador)

13.

✓

[QuipuNetX] Agregar USAR_PRINTER_SERVICE a FeatureFlagConfigReader

14. ☒ [QuipuNetX] Modificar PedidoController.addLista() con feature flag
15. ☒ [QuipuNetX] Build y verificar 0 errores
16. ☒ [QuipuNet.exe] Modificar ListaPedidosTemporalesPresenter.enviarPedidos()
(también enviarPedidosOld y enviarPedidosSelfService – código comentado, listo para activar)
17. ☒ [QuipuNet.exe] Build y verificar 0 errores nuevos (errores PrinterCore/TfhkaNet pre-existentes)
18. Testing end-to-end con flag OFF (sin regresión)
19. Testing end-to-end con flag ON (comandas vía PrinterServices)

Fase 7B: Comprobantes

20. [QuipuNetX] Crear ServerVentaRapidaStrategy, ServerVentaSalonStrategy, ServerDeliveryStrategy, ServerVentaProveedorStrategy, ServerAnularCorregirStrategy
21. [QuipuNetX] Crear ServerVentaImpresionWorkflow
22. [QuipuNetX] Modificar VentaController con feature flag
23. [QuipuNet.exe] Modificar FragmentDetallePago con feature flag
24. Testing comprobantes end-to-end

Fase 7C: Secundarios

25. Replicar sorteo, encuesta, delivery al backend (ya incluidos en estrategias de 7B)
26. Replicar reimpresión al backend
27. Testing completo

18. Verificación y Testing

18.1 Feature flag OFF (sin cambios)

- ☒ Flujo actual completo: pedido → ImpresionList → Front → ImpresionService → PrintUtil → TCP:9100
- ☒ No hay regresión en ningún tipo de impresión
- ☒ Estrategias del Front siguen funcionando normalmente

18.2 Feature flag ON + Servidor (con PrinterServices)

- ☒ Pedido → backend ejecuta ServerComandasStrategy → PrinterServiceClient → PrinterServices → TCP:9100
- ☒ Venta → backend ejecuta Server*Strategy → PrinterServiceClient → PrinterServices → TCP:9100
- ☒ Front NO intenta imprimir nada
- ☒ PrinterServices responde con job_id ↔ pedido_ids
- ☒ gRPC NotifyPrintFailed(job_id) → Front marca pedidos asociados como fallidos
- ☒ gRPC NotifyPrintSuccess(job_id) → Front confirma impresión de pedidos
- ☒ Stickers Jaltech siguen imprimiéndose por flujo actual (Front)

18.3 Feature flag ON + Cliente

- ☒ Máquina cliente ignora el flag – flujo actual siempre

18.4 PrinterServices caído + flag ON

- ☒ Backend retorna error al Front
- ☒ Front muestra error al usuario
- ☒ NO hay fallback local

19. Restricciones y reglas

1. **Solo servidor:** USAR_PRINTER_SERVICE solo aplica si Util.esModoServidor() == true
2. **No stickers:** ImprimirStickerJaltech() se excluye, sigue con flujo actual
3. **Sin fallback:** Si PrinterServices no responde (después de 2 intentos), error (no impresión local)
4. **Backward compatible:** Con flag OFF, todo funciona exactamente igual que antes
5. **Enriquecimiento en backend:** Impresora.findById() en QuipuNetX antes de enviar a PrinterServices
6. **Bitmap renderizado LOCAL en PrinterServices:** HtmlBitmapRenderer se copia a PrinterServices; NO se envía base64 en JSON
7. **Mismos NuGet:** No se agregan NuGet nuevos a QuipuNetX (HttpClient estándar .NET)
8. **Front intacto:** Las clases y estrategias del Front **NO se eliminan ni modifican**. Se COPIAN al backend — las originales se mantienen en el Front para el flujo con FLAG OFF o máquinas CLIENTE
9. **Coexistencia dual:** Existirán dos versiones de las clases base: `QuipuNet.Services.Print.*` (Front, FLAG OFF) y `QuipuNetX.Services.Print.*` (Backend, FLAG ON). Ambas coexisten sin conflicto
10. **Camino A:** Las estrategias Server* viven en el backend — el Front NO orquesta impresión con flag ON

11. **Reintentos:** PrinterServiceClient intenta 2 veces (1 + 1 reintento) con 1s entre intentos antes de declarar servicio no disponible
12. **Mensajes dinámicos:** Respuesta.Mensajes dice "Imprimiendo {tipo}..." según la Strategy que lo invoca
13. **SUCCESS ≠ impreso:** Respuesta.Tipo = SUCCESS significa "jobs registrados en cola", NO "ya se imprimió". El resultado final llega por gRPC

20. Sincronización Bidireccional: QuipuNetX ↔ PrinterServices

20.1 Principio de diseño

QuipuNetX es la fuente de verdad (master) para datos de impresoras.
PrinterServices sincroniza su catálogo desde QuipuNetX al inicio.
PrinterServices notifica a QuipuNetX cuando detecta cambios de IP por DHCP.
Ambos sistemas mantienen consistencia automática sin intervención manual.

20.2 Problema resuelto

ANTES:

- PrinterServices tenía su propia tabla `printers` desconectada de QuipuNetX
- Las impresoras se registraban/activaban/inactivaban en QuipuNetX pero PrinterServices no se enteraba
- Si una impresora cambiaba de IP (DHCP), PrinterServices la auto-resolvía pero QuipuNetX quedaba con IP obsoleta
- Administrador tenía que actualizar MANUALMENTE en ambos sistemas

AHORA:

- PrinterServices recibe lista de impresoras desde QuipuNetX al inicio (sincronización inicial)
- Cambios de IP detectados por PrinterServices se notifican a QuipuNetX automáticamente
- Sistema de retry con persistencia: si QuipuNetX está offline, la notificación se guarda y se reenvía cada 30s
- Consistencia bidireccional garantizada sin intervención manual

20.3 Arquitectura de 2 flujos

```
FLUJO 1: Sincronización inicial (QuipuNetX → PrinterServices)
Disparador: QuipuNetX inicia servidor web

QuipuNetX Servidor inicia (WebServer_New.cs)
↓
PrinterSyncService.SyncPrintersWithService() // Llamado después de Nancy.Start()
↓
Query BD local: SELECT * FROM IMPRESORA
                WHERE impresora_modoimpresion='SERVICIO'
                AND impresora_estado='ACTIVO'
↓
Mapear a DTO: List<PrinterSyncDto> {
    impresora_id,
    nombre,
    ip,
    puerto,
    mac_address,
    estado
}
↓
HTTP POST → http://localhost:8090/api/printers/sync
Body: [
    { "impresora_id": "IMP-001", "nombre": "Cocina", "ip": "192.168.68.193",
      "puerto": 9100, "mac_address": "AA:BB:CC:DD:EE:FF" },
    ...
]
↓
PrinterServices recibe en PrinterController.SyncPrinters()
↓
Para cada impresora en el array:
├─ Existe en BD (WHERE impresora_id = ?)
│   └─ SÍ → UPDATE (ip, nombre, mac, puerto)
│   └─ NO → INSERT nueva
↓
Response: { "success": true, "synchronized": 5, "inserted": 2, "updated": 3 }
↓
Log: "Sincronización inicial: 5 impresoras (2 nuevas, 3 actualizadas)"
```

```
FLUJO 2: Notificación de cambio de IP (PrinterServices → QuipuNetX) |
Disparador: ArpScanWorker detecta nueva IP para MAC
```

```
ArpScanWorker.ProcessScanAsync()
↓
PrinterIpResolver.TryResolveNewIp() → encuentra nueva IP
↓
UPDATE printers SET ip='192.168.68.150' WHERE mac_address='AA:BB:CC:DD:EE:FF'
↓
QuipuNetXNotifier.NotifyIpChangeAsync(mac, oldIp, newIp)
↓
Intenta POST → http://localhost:8081/api/rest/printers/update-ip
Body: {
  "mac_address": "AA:BB:CC:DD:EE:FF",
  "old_ip": "192.168.68.193",
  "new_ip": "192.168.68.150"
}
↓
✅ QuipuNetX ONLINE
QuipuNetX recibe en PrinterSyncModule
↓
UPDATE IMPRESORA
SET impresora_ipdeimpresora = '192.168.68.150'
WHERE impresora_macdeimpresora = 'AA:BB:CC:DD:EE:FF'
↓
Response: { "success": true }
↓
Log: "IP actualizada para MAC AA:BB:CC..."
↓
❌ QuipuNetX OFFLINE
Timeout / HttpRequestException
↓
INSERT INTO notificacionescambiosip
(mac_address, old_ip, new_ip, estado, intentos)
VALUES
('AA:BB:CC...', '192.168.68.193', '192.168.68.150',
'PENDIENTE', 0)
↓
Log: "QuipuNetX offline - notificación encolada (ID 42)"
```

20.4 Proceso de retry persistente

```
NotificationRetryWorker - Hilo dedicado, event-driven |
Intervalo: 30s (configurable)
```

```
while (true):
  await Task.Delay(30000) // Esperar 30 segundos
  ↓
  SELECT * FROM notificacionescambiosip WHERE estado='PENDIENTE'
  ↓
  Si hay notificaciones pendientes:
    ↓
    Para cada notificación:
      ↓
      Intenta POST → QuipuNetX /api/rest/printers/update-ip
      ↓
      ✅ SUCCESS
      UPDATE notificacionescambiosip
      SET estado='ENVIADO',
        fecha_envio=NOW()
      WHERE id=42
      ↓
      Log: "Notificación ID 42 enviada (retry)"
```



```

↓
❌ FAILURE
UPDATE notificacionescambiosip
SET intentos=intentos+1,
    ultimo_error='Timeout'
WHERE id=42
↓
Si intentos >= 10:
    UPDATE estado='FALLIDO'
    Log: "Notificación ID 42 FALLIDA"
Sino:
    Log: "Retry fallido (intento 3/10)"

```

Resultado: Cuando QuipuNetX vuelve a estar activo, todas las notificaciones pendientes se envían automáticamente en el próximo ciclo de 30s.

20.5 Modelo de datos

Tabla: notificacionescambiosip (PrinterServices)

Columna	Tipo	Descripción
id	INTEGER PK	ID autoincremental
mac_address	TEXT	MAC de la impresora (invariable)
old_ip	TEXT	IP anterior detectada
new_ip	TEXT	IP nueva detectada por ARP
estado	TEXT	PENDIENTE / ENVIADO / FALLIDO
fecha_creacion	DATETIME	Timestamp de creación
fecha_envio	DATETIME	Timestamp de envío exitoso (null si pendiente)
intentos	INTEGER	Contador de reintentos (max 10)
ultimo_error	TEXT	Último error capturado (para debugging)

Índices:

- `idx_notif_estado` en `estado` (optimiza query de PENDIENTES)
- `idx_notif_mac` en `mac_address` (evita duplicados)

Campos en tabla IMPRESORA (QuipuNetX)

Campo	Descripción	Uso
<code>impresora_macdeimpresora</code>	MAC address de la impresora física	Clave para sincronización
<code>impresora_ipdeimpresora</code>	IP actual de la impresora	Se actualiza cuando PrinterServices notifica cambio
<code>impresora_modoopresion</code>	Tipo: SERVICIO / ETHERNET / BLUETOOTH	Filtro: solo SERVICIO se sincroniza
<code>impresora_estado</code>	ACTIVO / INACTIVO	Filtro: solo ACTIVO se sincroniza

20.6 DTOs y contratos

PrinterSyncDto.cs (compartido conceptualmente)

```

// QuipuNetX → PrinterServices
{
    "impresora_id": "IMP-001",          // PK en ambas BDs
    "nombre": "Cocina Principal",
    "ip": "192.168.68.193",
    "puerto": 9100,
    "mac_address": "AA:BB:CC:DD:EE:FF",
    "estado": "ACTIVO"
}

```

IpChangeNotificationDto (PrinterServices → QuipuNetX)

```

// POST /api/rest/printers/update-ip
{

```

```
    "mac_address": "AA:BB:CC:DD:EE:FF",
    "old_ip": "192.168.68.193",
    "new_ip": "192.168.68.150"
}
```

20.7 Endpoints implementados

PrinterServices (recibe sincronización)

Método	Endpoint	Descripción	Body	Response
POST	/api/printers/sync	Sincroniza lista de impresoras desde QuipuNetX	Array de PrinterSyncDto	{ success: true, synchronized: N, inserted: X, updated: Y }

QuipuNetX (recibe notificaciones de cambio)

Método	Endpoint	Descripción	Body	Response
POST	/api/rest/printers/update-ip	Actualiza IP de impresora por MAC	IpChangeNotificationDto	{ success: true }

20.8 Configuración

PrinterServices (config_settings)

Key	Valor Default	Descripción
QuipuNetXUrl	http://localhost:8081	URL base de QuipuNetX servidor
QuipuNetXTimeoutMs	5000	Timeout para HTTP requests a QuipuNetX
NotificationRetryIntervalSeconds	30	Intervalo entre reintentos de notificaciones
NotificationMaxRetries	10	Máximo de reintentos antes de marcar como FALLIDO

QuipuNetX (WebServerConfig.json)

```
{
  "PrinterServiceUrl": "http://localhost:8090",
  "PrinterServiceSyncOnStartup": true
}
```

20.9 Integración en ciclo de vida

QuipuNetX (WebServer_New.cs)

```
public void Start()
{
    // ... después de Nancy.Start() ...

    // Sincronizar impresoras con PrinterServices
    if (WebServerConfig.PrinterServiceSyncOnStartup)
    {
        Task.Run(() => PrinterSyncService.SyncPrintersWithService());
    }
}
```

PrinterServices (PrinterServicesHost.cs)

```
public void Start()
{
    // ...

    // 6. Inicializar ArpScanWorker
    _arpWorker = new ArpScanWorker(_db);
    _arpWorker.Start();

    // 6.5. Inicializar NotificationRetryWorker
    _notifRetryWorker = new NotificationRetryWorker(_db);
    _notifRetryWorker.Start();
}
```

```
    // ...
}

public void Stop()
{
    // ...

    if (_notifRetryWorker != null)
        _notifRetryWorker.Stop();

    if (_arpWorker != null)
        _arpWorker.Stop();

    // ...
}
```

20.10 Casos edge manejados

Edge 1: QuipuNetX offline durante detección de cambio de IP

Escenario: PrinterServices detecta nueva IP pero QuipuNetX está cerrado.

Solución:

- Notificación se persiste en BD con estado PENDIENTE
- NotificationRetryWorker reintenta cada 30s
- Cuando QuipuNetX vuelve, notificación se envía automáticamente

Edge 2: Múltiples cambios de IP para misma MAC

Escenario: Impresora cambia de IP 2 veces antes de que QuipuNetX reciba notificación.

Solución:

- Solo se persiste la ÚLTIMA IP detectada (nueva notificación sobrescribe pendiente)
- QuipuNetX recibe solo la IP final más reciente

Edge 3: Notificación duplicada

Escenario: ArpScanWorker encuentra misma IP nueva 2 veces.

Solución:

- Query verifica si ya existe notificación PENDIENTE para esa MAC + nueva IP
- Si existe, no crea duplicado (retorna early)

Edge 4: PrinterServices inicia antes que QuipuNetX

Escenario: PrinterServices arranca pero QuipuNetX aún no está listo.

Solución:

- PrinterServices funciona con su catálogo actual (tabla `printers` poblada previamente)
- Cuando QuipuNetX arranque, enviará sincronización inicial
- PrinterServices actualiza su catálogo en ese momento

Edge 5: Impresora eliminada en QuipuNetX

Escenario: Administrador elimina impresora en QuipuNetX, pero PrinterServices la tiene registrada.

Solución:

- Sincronización inicial envía solo impresoras ACTIVAS
- PrinterServices mantiene impresoras inactivas (no las elimina)
- Jobs pendientes para impresora eliminada quedan en espera indefinida (visibles en `/api/jobs/pending`)

Edge 6: Exceder máximo de reintentos

Escenario: QuipuNetX permanece offline > 5 minutos (10 reintentos × 30s).

Solución:

- Notificación se marca como FALLIDO en BD
- Administrador puede consultar tabla `notificacionescambiosip` para ver notificaciones fallidas
- Opción futura: endpoint manual para reactivar notificación fallida

20.11 Ventajas de esta arquitectura

✅ **Consistencia automática:** Ambos sistemas mantienen datos sincronizados sin intervención manual

- ✅ **Robustez ante fallos:** Si QuipuNetX está offline, notificaciones se guardan y reintentan automáticamente
- ✅ **Visibilidad completa:** Tabla `notificacionescambiosip` registra historial de todos los cambios de IP
- ✅ **Recuperación automática:** Cuando QuipuNetX vuelve activo, todas las notificaciones pendientes se envían en < 30s
- ✅ **Sin pérdida de datos:** Persistencia en BD garantiza que ningún cambio de IP se pierda
- ✅ **Performance optimizado:** Worker de retry usa event-driven (CPU 0% idle entre ciclos)
- ✅ **Escalable:** Soporta múltiples cambios de IP pendientes sin saturar memoria

20.12 Logs y monitoreo

PrinterServices

```
[QUIPU-NOTIFIER] Notificando cambio IP: 192.168.68.193 → 192.168.68.150 (MAC: AA:BB:CC...)
[QUIPU-NOTIFIER] ✅ Notificación enviada exitosamente a QuipuNetX

[QUIPU-NOTIFIER] 🚫 QuipuNetX offline/inalcanzable: No connection could be made
[QUIPU-NOTIFIER] 📁 Notificación encolada para retry: MAC AA:BB:CC... → 192.168.68.150

[NOTIF-RETRY] Worker iniciado (intervalo: 30s, max reintentos: 10)
[NOTIF-RETRY] 📌 3 notificación(es) pendiente(s) – iniciando retry
[NOTIF-RETRY] ✅ Notificación ID 42 enviada exitosamente (MAC AA:BB:CC...)
[NOTIF-RETRY] ⚠️ Retry fallido para notificación ID 43 (3/10): Timeout
[NOTIF-RETRY] ❌ Notificación ID 44 marcada como FALLIDO tras 10 intentos
```

QuipuNetX

```
[PRINTER-SYNC] Iniciando sincronización con PrinterServices (http://localhost:8090)
[PRINTER-SYNC] ✅ Sincronización exitosa: 5 impresoras (2 nuevas, 3 actualizadas)

[PRINTER-SYNC-MODULE] IP actualizada: MAC AA:BB:CC... → 192.168.68.150
[PRINTER-SYNC-MODULE] Impresora 'Cocina Principal' ahora en 192.168.68.150
```

20.13 Tabla de decisión completa

Evento	PrinterServices	QuipuNetX	Resultado
QuipuNetX inicia	En ejecución	Arranca → sincroniza	PrinterServices actualiza catálogo
Cambio IP detectado	Notifica → QuipuNetX online	Recibe → actualiza BD	Consistencia inmediata ✅
Cambio IP detectado	Notifica → QuipuNetX offline	-	Notificación PENDIENTE en BD
QuipuNetX vuelve online	Retry worker reenvía	Recibe notificaciones	Consistencia recuperada ✅
Retry falla 10 veces	Marca FALLIDO	-	Requiere revisión manual
Impresora nueva en QuipuNetX	-	Próxima sincronización	PrinterServices la registra
Impresora inactivada	Mantiene registro	Solo envía ACTIVAS	No se elimina de PrinterServices

21. Fase 7A-bis: Tabla `pedidoprinterjob` + Verificación de JobIds en Front

21.1 Problema

Cuando el feature flag `USAR_PRINTER_SERVICE` está ON, `PedidoController.addLista()` envía las comandas a PrinterServices en modo **fire-and-forget** (`Task.Run` sin `await`). Esto significa:

- El **Front NUNCA sabe** si PrinterServices recibió los jobs
- Si PrinterServices está muerto, **no se muestra ningún error** al usuario
- El usuario cree que sus comandas se imprimirán, pero nunca llegarán a la impresora
- No hay forma de verificar si un pedido tiene un job de impresión asociado

21.2 Solución: Tabla dedicada `pedidoprinterjob`

¿Por qué NO usar `PedidoBase.Printerjobid` ?

La tabla `pedido` es **extremadamente concurrida** — cada pedido, modificador, combo, adicional genera registros. Agregar writes adicionales para el job de impresión impactaría el rendimiento. Además, la relación `pedido↔job` es un concepto de PrinterServices, no del dominio de pedidos.

Solución: Crear una tabla SQLite dedicada `pedidoprinterjob` que almacena la relación entre pedidos y jobs de PrinterServices, sin tocar la tabla `pedido`.

21.3 Esquema de la tabla

Tabla: pedidoprinterjob				
ID (PK)	PEDIDOID	JOBID	STATUS	FECHACREACION
auto	"1772"	"714"	"SENT"	"2026-02-23 21:30:00"
auto	"1773"	"714"	"SENT"	"2026-02-23 21:30:00"
auto	"1774"	"715"	"SENT"	"2026-02-23 21:30:00"

Campos:

- ID : Primary key autoincremental (heredado de SugarRecord)
- PEDIDOID : ID del pedido original (referencia lógica a tabla pedido)
- JOBID : ID del job asignado por PrinterServices al encolar la comanda
- STATUS : Estado del registro — SENT (enviado OK), ERROR (falló), PRINTED (futuro: confirmación física)
- FECHACREACION : Fecha/hora de generación del registro (auto-asignado al crear)

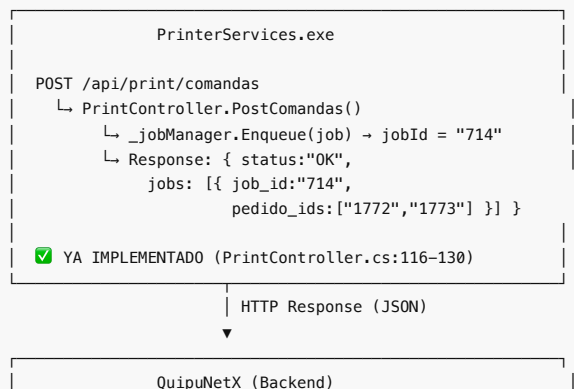
Relación con PrinterServices: Un job en PrinterServices puede tener N pedidos asociados (ej: comanda con 7 productos = 7 pedido_ids, 1 job_id). Por eso hay N registros en pedidoprinterjob con el mismo JOBID .

21.4 Herencia de entidades — Patrón del proyecto

```
SugarRecord (Sugar/Com/Orm/SugarRecord.cs)
├ Id (PK, AutoIncrement)
├ save(), delete(), FindWithQuery<T>(), Find<T>(), etc.
├
├ └ PedidoprinterjobBase (entitybase/PedidoprinterjobBase.cs) ← NUEVO
│   ├── Fields: pedidoid, jobid, status, fechacreacion
│   ├── [Column("...")] properties mapeadas a SQLite
│   ├── fromJSON(JObject), toJSON() — serialización estándar
│   ├── save(), saveSinCvs(), delete(), deleteSinCvs()
│   ├── findById(), findByJobid(), findByPedidoid()
│   └ listAll(), deleteAll(), EnsureTable()
├
├ └ Pedidoprinterjob (entity/Pedidoprinterjob.cs) ← NUEVO
│   ├── [Table("pedidoprinterjob")] — nombre de tabla SQLite
│   ├── Constructores: () y (JObject)
│   ├── ICloneable
│   ├── findByJobId(string) — buscar pedidos por job
│   ├── findByPedidoId(string) — buscar job de un pedido
│   └ todosConJobAsignado(IList<string>) — verificar batch
├
├ └ PedidoprinterjobController (controller/) ← NUEVO
│   ├── guardarJobsPorComandas(IList<PrintJobResult>) — batch insert
│   ├── getByJobId(string) — consultar por job
│   └ getByPedidoId(string) — consultar por pedido
```

Referencia de patrón existente: TransaccionordenpedidoBase → Transaccionordenpedido (entidad simple con el mismo patrón de herencia).

21.5 Flujo de datos completo — De punta a punta



```

PrinterServiceClient.EnviarComandasAsync()
└─ PostConReintentoAsync(url, body)
└─ ParseSuccessResponse(responseBody)
    └─ List<PrintJobResult> jobResults
        [{JobId="714", PedidoIds=["1772","1773"]}]}

```

✅ YA IMPLEMENTADO (PrinterServiceClient.cs:156-189)

```

PedidoController.addLista()
└─ ANTES: fire-and-forget (Task.Run sin await)
└─ DESPUÉS: await sincrono + guardar en tabla
    └─ respuestaPS = await EnviarComandasAsync()
        └─ SI SUCCESS + jobResults:
            └─ PedidoprinterjobController
                .guardarJobsPorComandas(jobResults)
            └─ respuesta.PrintJobResults = jobResults
        └─ SI ERROR:
            └─ mensajes.Add("Hubo un problema...")
            └─ respuesta.PrintJobResults = null

```

🔄 POR IMPLEMENTAR

Respuesta (objeto C#)

QuipuNet.exe (Front)

```

ListaPedidosTemporalesPresenter.enviarPedidos()
└─ callback(respuesta)
    └─ SI impresionList.Count > 0:
        └─ imprimirComandas() (FLAG OFF, local)
    └─ SI impresionList vacía + FLAG ON:
        └─ verificarPrintJobsPorComandas(respuesta)
            └─ SI PrintJobResults tiene datos:
                └─ showSuccessRegister() ✅
            └─ SI PrintJobResults vacío/null:
                └─ showErrorRegister(
                    "Hubo un problema en la
                    impresión: PrinterServiceLCA
                    no está activo") ⚠️

```

🔄 POR IMPLEMENTAR

21.6 Qué ya está implementado en PrinterServices

PrinterServices NO necesita cambios. El flujo de retorno de joblds ya funciona:

- PrintController.PostComandas()** (PrinterServices/Api/Controllers/PrintController.cs:68-141):
 - Recibe array JSON de comandas
 - Por cada comanda: `_jobManager.Enqueue(job)` → obtiene `jobId`
 - Construye respuesta: `{ status: "OK", jobs: [{ job_id, pedido_ids }] }`
 - Retorna `ApiResponse.Ok(json)` con HTTP 200
- PrintJobManager.Enqueue()** (PrinterServices/Queue/PrintJobManager.cs):
 - Persiste el job en SQLite (`print_jobs` tabla)
 - Retorna el ID auto-generado como `jobId`
 - El job queda encolado para que `PrintWorker` lo procese
- Formato de respuesta HTTP** (ya definido y funcional):

```

{
  "status": "OK",
  "jobs": [
    {
      "job_id": "714",
      "pedido_ids": ["1772", "1773", "1774", "1775", "1776", "1777", "1778"]
    },
    {

```

```

        "job_id": "715",
        "pedido_ids": ["1779"]
    }
}
]
}

```

Ejemplo real: Una comanda para BARRA con 1 chicha, 2 limonadas, 4 naranjadas = 7 pedido_ids internos, pero salen en una sola comanda → 1 job_id ("714") con 7 pedido_ids asociados.

21.7 Qué ya está implementado en QuipuNetX (cliente HTTP)

PrinterServiceClient NO necesita cambios. El parseo de la respuesta ya funciona:

1. **EnviarComandasAsync()** (QuipuNetX/Services/Print/PrinterServiceClient.cs:44-79):
 - Enriquece cada Impresion con datos de Impresora
 - Construye JSON array y hace POST a `/api/print/comandas`
 - Llama a `PostConReintentoAsync()` (con reintentos y timeout)
 - Retorna `Respuesta` con `Data = List<PrintJobResult>`
2. **ParseSuccessResponse()** (PrinterServiceClient.cs:156-195):
 - Parsea `{ status: "OK", jobs: [...] }`
 - Extrae `job_id` y `pedido_ids` de cada job
 - Crea `List<PrintJobResult>` y lo pone en `Respuesta.Data`
 - Retorna `Respuesta.Tipo = SUCCESS`
3. **PostConReintentoAsync()** (PrinterServiceClient.cs:82-149):
 - Hasta `MAX_REINTENTOS + 1` intentos (default: 2)
 - Timeout de `TIMEOUT_MS` (default: 10000ms)
 - Delay entre reintentos: `DELAY_ENTRE_REINTENTOS_MS` (default: 1000ms)
 - Si todos fallan: `Respuesta.Err("Servicio no disponible después de N intentos")`

21.8 Cambios necesarios en QuipuNetX (PedidoController)

Archivo: QuipuNetX/controller/PedidoController.cs (líneas 612-650)

ANTES (fire-and-forget — el Front nunca se entera si PrinterServices falló):

```

// Vaciar impresionList AHORA para que Front NO imprima localmente
impresionList = new List<Impresion>();

// Fire-and-forget seguro: enviar a PrinterServices (usa variable local capturada)
System.Threading.Tasks.Task.Run(async () =>
{
    try
    {
        await PrinterServiceClient.Instance.EnviarComandasAsync(
            comandasParaImprimir, pedidoIds, "Imprimiendo comandas...");
    }
    catch (Exception ex)
    {
        Util.Capture(ex, "PedidoController", "addLista.PrinterService", ex.StackTrace ?? "-");
    }
});

```

DESPUÉS (await síncrono + guardar en tabla + inyectar en respuesta):

```

// Vaciar impresionList AHORA para que Front NO imprima localmente
impresionList = new List<Impresion>();

// Await síncrono: obtener respuesta de PrinterServices con job_ids
// Task.Run evita deadlock al ejecutar en thread del pool (sin SynchronizationContext)
Respuesta respuestaPS = null;
try
{
    respuestaPS = System.Threading.Tasks.Task.Run(async () =>
    {
        // Enviar comandas a PrinterServices y esperar respuesta con job_ids
        return await PrinterServiceClient.Instance.EnviarComandasAsync(
            comandasParaImprimir, pedidoIds, "Imprimiendo comandas...");
    }).GetAwaiter().GetResult();
}

```

```

}
catch (Exception ex)
{
    // Capturar error de comunicación con PrinterServices
    Util.Capture(ex, "PedidoController", "addLista.PrinterService", ex.StackTrace ?? "-");
}

// Variable para almacenar jobResults si PrinterServices respondió OK
List<PrintJobResult> printJobResultsParaRespuesta = null;

// Evaluar respuesta de PrinterServices y guardar en tabla pedidoprinterjob
if (respuestaPS != null && respuestaPS.Tipo == Util.SUCCESS
    && respuestaPS.Data is List<PrintJobResult> jobResults && jobResults.Count > 0)
{
    // PrinterServices respondió OK – guardar relación pedido+job en tabla dedicada
    PedidoprinterjobController.guardarJobsPorComandas(jobResults);
    // Almacenar para inyectar en respuesta al Front
    printJobResultsParaRespuesta = jobResults;
}
else
{
    // PrinterServices no respondió o falló – agregar advertencia
    // El Front detectará que PrintJobResults es null y mostrará error
    mensajes.Add("Hubo un problema en la impresión: PrinterServiceLCA no está activo");
}

```

Y antes del `return` final de `addLista()` , inyectar los jobResults en la respuesta:

```

// Inyectar jobResults de PrinterServices en la respuesta (null si falló)
respuesta.PrintJobResults = printJobResultsParaRespuesta;

```

21.9 Cambios necesarios en Respuesta.cs

Archivo: QuipuNetX/Util/Respuesta.cs

Agregar propiedad para transportar los jobResults al Front:

```

/// <summary>
/// Lista de resultados de jobs de PrinterServices.
/// null → feature flag OFF o no aplica
/// vacío → PrinterServices no respondió (error)
/// con datos → PrinterServices registró los jobs exitosamente
/// Marcado con [Ignore] para que SQLite no intente mapearlo
/// </summary>
[Ignore]
public IList<PrintJobResult> PrintJobResults { get; set; }

```

21.10 Cambios necesarios en Front (ListaPedidosTemporalesPresenter)

Archivo: QuipuNet/MainApp/venta/Pedidos/ListaPedidosTemporalesPresenter.cs (líneas 118-138)

ANTES:

```

if (respuesta.ImpresionList != null && respuesta.ImpresionList.Count > 0)
{
    // Imprimir localmente (flag OFF)
    this.imprimirComandas(impresiones);
    view.showSuccessRegister();
}
else
{
    view.showSuccessRegister(); // ← Flag ON: asume todo OK (BUG)
}

```

DESPUÉS:

```

if (respuesta.ImpresionList != null && respuesta.ImpresionList.Count > 0)
{
    // Flag OFF: imprimir localmente (flujo antiguo sin cambios)
    // ... configuración de área ...
}

```



```

        this.imprimirComandas(impresiones);
        view.showSuccessRegister();
    }
    else if (FeatureFlagManager.Instance.IsEnabled("USAR_PRINTER_SERVICE"))
    {
        // Flag ON: verificar que PrinterServices recibió los jobs
        this.verificarPrintJobsPorComandas(respuesta, view);
    }
    else
    {
        // Sin impresiones y sin flag – flujo normal
        view.showSuccessRegister();
    }
}

```

Nuevo método verificarPrintJobsPorComandas :

```

/// <summary>
/// Verifica que PrinterServices haya recibido y registrado los jobs de impresión.
/// Se llama cuando ImpresionList está vacía Y feature flag USAR_PRINTER_SERVICE está ON.
///
/// Lógica:
/// - Si respuesta.PrintJobResults tiene datos → éxito (PrinterServices registró los jobs)
/// - Si respuesta.PrintJobResults está vacío/null → error (PrinterServices no respondió)
///
/// Patrón de error: Mismo que GlobalPrintErrorHandler para consistencia visual.
/// </summary>
private void verificarPrintJobsPorComandas(Respuesta respuesta, IListaPedidosTemporalesView view)
{
    try
    {
        // Verificar si la respuesta incluye PrintJobResults de PrinterServices
        bool tieneJobs = respuesta.PrintJobResults != null && respuesta.PrintJobResults.Count > 0;

        if (tieneJobs)
        {
            // PrinterServices recibió y persistió las comandas en jobs → éxito
            view.showSuccessRegister();
        }
        else
        {
            // PrinterServices no respondió o no generó job_ids → mostrar error
            view.showErrorRegister("Hubo un problema en la impresión: PrinterServiceLCA no está activo");
        }
    }
    catch (Exception ex)
    {
        NLog.LogManager.GetCurrentClassLogger().Error(ex, "Error verificando PrintJobResults");
        view.showErrorRegister("Hubo un problema en la impresión: PrinterServiceLCA no está activo");
    }
}

```

21.11 Registro de tabla en SugarContext

Archivo: QuipuNetX/Sugar/Com/Orm/SugarContext.cs

Agregar después de la última tabla (Cajaposintegracion):

```

// Fase 7A-bis: Tabla dedicada para relación pedido+job de PrinterServices
getSugarDb().CreateTable<Pedidoprinterjob>();

```

21.12 Registro en QuipuNetX.csproj

Agregar 3 archivos nuevos:

```

<Compile Include="entitybase\PedidoprinterjobBase.cs" />
<Compile Include="entity\Pedidoprinterjob.cs" />
<Compile Include="controller\PedidoprinterjobController.cs" />

```

21.13 Matriz de comportamiento final

#	Flag	PrinterServices	Tabla pedidoprinterjob	PrintJobResults	Front muestra
1	OFF	N/A	Sin registros	null	Imprime localmente
2	ON	Vivo	Registros con SENT	[jr1, jr2...]	showSuccessRegister()
3	ON	Muerto	Sin registros	null	"PrinterServiceLCA no está activo"
4	ON	Parcial	Algunos registros	parcial	showSuccessRegister() (al menos 1 job)

21.14 Ejemplo completo — Comanda de BARRA

Escenario: Mozo envía pedido con 1 chicha, 2 limonadas, 4 naranjadas → 7 pedidos internos → 1 comanda para impresora BARRA.

Flag ON + PrinterServices vivo:

```
1. PedidoController genera impresionList con 1 Impresion (comanda BARRA)
   Impresion.Pedidos_ids_printer_asociados = "1772,1773,1774,1775,1776,1777,1778"

2. PrinterServiceClient.EnviarComandasAsync() → POST /api/print/comandas
   Body: [{ impresora_ip:"192.168.1.100", cadena:"...", pedido_ids:["1772",...,"1778"] }]

3. PrinterServices.PrintController.PostComandas()
   → _jobManager.Enqueue(job) → jobId = "714"
   → Response: { status:"OK", jobs:[{ job_id:"714", pedido_ids:["1772",...,"1778"] }] }

4. PrinterServiceClient.ParseSuccessResponse()
   → List<PrintJobResult> = [{ JobId="714", PedidoIds=["1772",...,"1778"] }]

5. PedidoController: respuestaPS.Tipo == SUCCESS
   → PedidoprinterjobController.guardarJobsPorComandas(jobResults)
   → Inserta 7 registros en tabla pedidoprinterjob:
     (1772, "714", "SENT"), (1773, "714", "SENT"), ..., (1778, "714", "SENT")
   → respuesta.PrintJobResults = jobResults

6. Front: impresionList vacía + FLAG ON
   → verificarPrintJobsPorComandas(respuesta)
   → PrintJobResults.Count > 0 → showSuccessRegister() 
```

Flag ON + PrinterServices muerto:

```
1-2. Igual que arriba

3. PrinterServiceClient.PostConReintentoAsync() → timeout × 2 intentos
   → Respuesta.Err("Servicio no disponible después de 2 intentos")

4. PedidoController: respuestaPS.Tipo == ERROR
   → NO guarda en pedidoprinterjob
   → mensajes.Add("Hubo un problema en la impresión: PrinterServiceLCA no está activo")
   → respuesta.PrintJobResults = null

5. Front: impresionList vacía + FLAG ON
   → verificarPrintJobsPorComandas(respuesta)
   → PrintJobResults == null → showErrorRegister("...PrinterServiceLCA no está activo") 
```

21.15 Presenters que requieren el mismo cambio

Presenter	Archivo	Método	Tipo
ListaPedidosTemporalesPresenter	Pedidos/ListaPedidosTemporalesPresenter.cs	enviarPedidos()	Salones
ListaPedidosTemporalesPresenter	Pedidos/ListaPedidosTemporalesPresenter.cs	enviarPedidosOld()	Salones (legacy)
ListaPedidosTemporalesPresenter	Pedidos/ListaPedidosTemporalesPresenter.cs	enviarPedidosSelfService()	Self-service
FragmentDetalleMesaPresenter	Pedidos/mesa/FragmentDetalleMesaPresenter.cs	Similar patrón	Detalle mesa
ListaPedidoTemporalesDeliveryPresenter	delivery/ListaPedidoTemporalesDeliveryPresenter.cs	Similar patrón	Delivery
ResumenDeliveryPresenter	delivery/ResumenDeliveryPresenter.cs	Similar patrón	Delivery resumen

Todos usan el mismo patrón: `if (impresionList.Count > 0) → imprimirComandas() / else → showSuccessRegister()`. El método `verificarPrintJobsPorComandas()` puede extraerse a una clase utilitaria compartida si se prefiere.

21.16 Orden de implementación

Paso	Archivo	Acción	Proyecto
1	entitybase/PedidoprinterjobBase.cs	CREAR	QuipuNetX
2	entity/Pedidoprinterjob.cs	CREAR	QuipuNetX
3	Sugar/Com/Orm/SugarContext.cs	AGREGAR CreateTable	QuipuNetX
4	controller/PedidoprinterjobController.cs	CREAR	QuipuNetX
5	Util/Respuesta.cs	AGREGAR PrintJobResults	QuipuNetX
6	controller/PedidoController.cs	MODIFICAR addLista (await + guardar)	QuipuNetX
7	QuipuNetX.csproj	AGREGAR 3 archivos	QuipuNetX
8	ListaPedidosTemporalesPresenter.cs	AGREGAR verificarPrintJobsPorComandas	QuipuNet (Front)
9	Otros Presenters	REPLICAR else-if + verificación	QuipuNet (Front)
—	PrinterServices	SIN CAMBIOS (ya retorna joblds)	PrinterServices

22. Fase 7B — Reimpresión segura: Cancelar job anterior + Crear nuevo ⌚ PENDIENTE DE IMPLEMENTAR

Estado: PENDIENTE — Esta sección documenta el plan de implementación. No se ha escrito código aún.

22.1 Problema

Cuando el usuario solicita **reimprimir** una comanda (botón "Reimprimir" en `FragmentDetallePedido`), el sistema actual genera un nuevo `ImpresionList` y lo envía al Front para impresión local.

Con `USAR_PRINTER_SERVICE` activo, este flujo delegaría la impresión a `PrinterServices`. **Pero el job original podría seguir vivo** en `PrinterServices` con estado `PENDING`, `WAITING` o incluso en `RETRY`. Si no se cancela el job anterior antes de crear uno nuevo:

Job original (PENDING/WAITING) → Worker lo procesa → IMPRIME ✓
Job nuevo (reimpresión) → Worker lo procesa → IMPRIME ✓
Resultado: DUPLICADO ✗

22.2 Solución: Cancelar antes de reimprimir

El flujo de reimpresión debe:

1. **Consultar** la tabla `pedidoprinterjob` para obtener los `jobId`s asociados al `pedido_id`
2. **Cancelar** esos jobs en `PrinterServices` vía API `POST /api/job/{jobId}/cancel`
3. **Enviar** las nuevas comandas a `PrinterServices` (flujo normal)
4. **Guardar** los nuevos `jobId`s en `pedidoprinterjob`

ANTES (sin cancelación):
Front → `reImprimirPedido` → genera `ImpresionList` → Front imprime local

DESPUÉS (con `PrinterServices` + cancelación):
Front → `reImprimirPedido`
→ Backend consulta `pedidoprinterjob` por `pedido_id` → obtiene `jobIds` anteriores
→ Backend llama `POST /api/job/{jobId}/cancel` para CADA `jobId` anterior
→ `PrinterServices` marca jobs como `CANCELLED` (Worker los ignora)
→ Backend envía nuevas comandas a `PrinterServices` → obtiene nuevos `jobIds`
→ Backend guarda nuevos `jobIds` en `pedidoprinterjob` (`status="PENDING"`)
→ Backend retorna respuesta con `PrintJobResults` al Front

22.3 Nuevo estado: CANCELLED en `PrintJobStatus`

Agregar estado `Cancelled` al enum `PrintJobStatus`:

Archivo: `PrinterServices/Queue/PrintJob.cs`

```
public enum PrintJobStatus
{
    Pending,
```

```

    Printing,
    Done,
    Failed,
    Waiting,
    Cancelled    // ← NUEVO: Job cancelado por reimpresión
}

```

22.4 Nuevo método: MarkCancelled en PrintJobManager

Archivo: PrinterServices/Queue/PrintJobManager.cs

```

public void MarkCancelled(PrintJob job, string reason)
{
    job.Estado = PrintJobStatus.Cancelled;
    job.ErrorMensaje = reason;
    UpdateJobInDb(job);
    Log.InfoFormat("[QUEUE] Job {0} CANCELADO → {1}", job.JobId, reason);
}

```

Importante: RecoverPending() ya filtra solo PENDING y WAITING, por lo que jobs CANCELLED **NO se recuperarán** al reiniciar PrinterServices.



22.5 PrintWorker: Verificar estado antes de procesar

El PrintWorker.WorkLoop debe verificar si el job fue cancelado entre el momento que se encoló y el momento que se desencola:

Archivo: PrinterServices/Workers/PrintWorker.cs – método WorkLoop

```

job = await _jobManager.DequeueAsync(ct);
if (job == null) continue;

// FASE 7B: Verificar si el job fue cancelado (por reimpresión) mientras esperaba en cola
// Re-verificar estado desde SQLite (puede haber sido cancelado mientras estaba en cola en memoria)
var freshJob = _jobManager.GetJobById(job.JobId);
if (freshJob != null && freshJob.Estado == PrintJobStatus.Cancelled)
{
    Log.InfoFormat("[WORKER] Job {0} CANCELADO (detectado por re-check SQLite)", job.JobId);
    LogPrint(job, "CANCELLED", "Job cancelado por reimpresión");
    continue;
}

job.Estado = PrintJobStatus.Printing;

```

Razón del re-check: El job podría estar en la cola en memoria (ConcurrentQueue) cuando llega la cancelación. La cancelación solo actualiza SQLite. Cuando el Worker desencola el job, el objeto en memoria aún tiene Estado = Pending. El re-check contra SQLite detecta la cancelación.

22.6 Nuevo endpoint: POST /api/job/{jobId}/cancel

Archivo: PrinterServices/Api/Controllers/JobController.cs

```

public ApiResult CancelJob(string jobId)
{
    if (string.IsNullOrEmpty(jobId))
        return ApiResult.BadRequest("jobId es requerido");

    var job = _jobManager.GetJobById(jobId);
    if (job == null)
        return ApiResult.NotFound();

    // Solo cancelar jobs que NO estén ya terminados
    if (job.Estado == PrintJobStatus.Done)
        return ApiResult.BadRequest("No se puede cancelar un job ya impreso (DONE)");

    if (job.Estado == PrintJobStatus.Cancelled)
        return ApiResult.Ok(JsonConvert.SerializeObject(new { status = "ALREADY_CANCELLED", jobId }));

    _jobManager.MarkCancelled(job, "Cancelado por reimpresión");

    return ApiResult.Ok(JsonConvert.SerializeObject(new {
        status = "CANCELLED",
        jobId = job.JobId,
    }));
}

```

```

        previousState = job.Estado.ToString()
    });
}

```

22.7 Ruta en ApiRouter

Archivo: PrinterServices/Api/ApiRouter.cs – método RouteAsync

```

// Agregar ANTES de /api/job/{jobId}/retry
if (method == "POST" && path.StartsWith("/api/job/") && path.EndsWith("/cancel"))
{
    string segment = path.Substring("/api/job/".Length);
    string jobId = segment.Substring(0, segment.Length - "/cancel".Length);
    return _jobController.CancelJob(jobId);
}

```

22.8 Nuevo método en PrinterServiceClient (QuipuNetX)

Archivo: QupuNetX/Services/Print/PrinterServiceClient.cs

```

/// Cancela un job en PrinterServices. Retorna true si fue cancelado exitosamente.
public async Task<bool> CancelJobAsync(string jobId)
{
    // POST /api/job/{jobId}/cancel
}

```

22.9 Modificar flujo de reimpresión en PedidoController

Archivo: QupuNetX/controller/PedidoController.cs – método reImprimirPedido

```

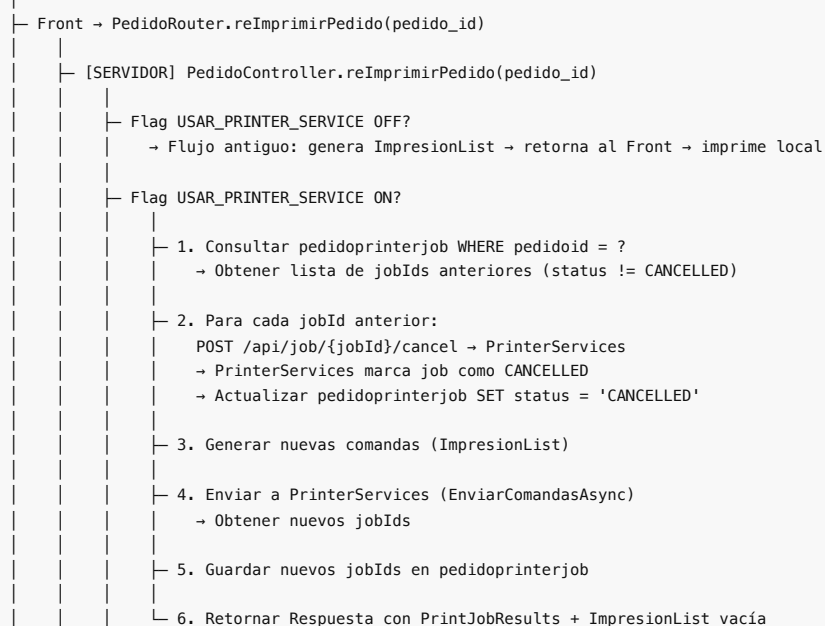
// ANTES de generar nuevas comandas (si flag USAR_PRINTER_SERVICE ON):
// 1. Consultar pedidoprinterjob por pedido_id → obtener jobIds anteriores
// 2. Para cada jobId: llamar PrinterServiceClient.CancelJobAsync(jobId)
// 3. Actualizar status en pedidoprinterjob a "CANCELLED"

// DESPUÉS de generar nuevas comandas (si flag ON):
// 4. Enviar a PrinterServices → obtener nuevos jobIds
// 5. Guardar nuevos jobIds en pedidoprinterjob (status="PENDING")
// 6. Retornar respuesta con PrintJobResults

```

22.10 Diagrama de flujo completo

Usuario → click "Reimprimir" (Front)



```

|   |─ [CLIENTE] REST → servidor ejecuta todo lo anterior
|   |
|   └─ Front recibe Respuesta
|       |
|       └─ ImpresionList con datos → imprimir local (flag OFF)
|           |
|           └─ ImpresionList vacía + flag ON → verificarPrintJobsPorComandas()
|               |
|               └─ PrintJobResults != null → showSuccessRegister ✅
|                   |
|                   └─ PrintJobResults == null → showErrorRegister ⚠️

```

22.11 Tabla de estados del job en el ciclo de vida

Estado	Quién lo asigna	Significado	Worker lo procesa?
PENDING	Enqueue()	Recién creado, esperando en cola	✅ Sí
PRINTING	PrintWorker	En proceso de impresión	— (transitorio)
DONE	MarkDone()	Impresión exitosa	❌ Ya terminó
FAILED	MarkFailed()	Todos los reintentos agotados	❌ Ya terminó
WAITING	MarkWaiting()	Impresora offline/sin papel	✅ Sí (via ReEnqueue)
CANCELLED	MarkCancelled()	Cancelado por reimpresión	❌ Worker lo omite

22.12 Métodos existentes que ya son seguros con CANCELLED

Método	Archivo	¿Seguro?	Razón
RecoverPending()	PrintJobManager.cs	✅	Solo recupera PENDING y WAITING
RequeueWaitingJobs()	StatusMonitor.cs	✅	Solo reencola WAITING
RetryJob()	JobController.cs	✅	Solo reintenta FAILED
WorkLoop()	PrintWorker.cs	⚠️ MODIFICAR	Agregar re-check SQLite antes de procesar

22.13 Orden de implementación

Paso	Archivo	Acción	Proyecto
1	Queue/PrintJob.cs	Agregar Cancelled al enum	PrinterServices
2	Queue/PrintJobManager.cs	Agregar MarkCancelled()	PrinterServices
3	Workers/PrintWorker.cs	Re-check SQLite antes de procesar	PrinterServices
4	Api/Controllers/JobController.cs	Agregar CancelJob()	PrinterServices
5	Api/ApiRouter.cs	Agregar ruta /api/job/{jobId}/cancel	PrinterServices
6	Services/Print/PrinterServiceClient.cs	Agregar CancelJobAsync()	QuipuNetX
7	controller/PedidoController.cs	Modificar reImprimirPedido	QuipuNetX
8	Presenters de reimpresión	Agregar verificación PrintJobResults	QuipuNet (Front)

22.14 Consideraciones de seguridad y edge-cases

- Job ya en estado PRINTING:** Si el Worker ya está enviando bytes por TCP, la cancelación no puede detener el envío en curso. El job se completará y se marcará DONE. La reimpresión generará un duplicado en este caso — aceptable porque es un window muy corto.
- Job DONE:** No se puede cancelar un job ya impreso. La API retorna error. El backend debe continuar con la reimpresión de todas formas (el usuario explícitamente quiere reimprimir).
- PrinterServices caído:** Si PrinterServices no responde a la cancelación, el backend debe capturar el timeout y continuar con la reimpresión. El riesgo de duplicado existe pero es preferible a no reimprimir.
- Múltiples jobs por pedido:** Un pedido puede tener múltiples jobs (una comanda por impresora/área). La cancelación debe iterar sobre TODOS los joblds del pedido.
- Concurrencia:** Dos usuarios reimprimen el mismo pedido simultáneamente. Ambos cancelan los mismos jobs, pero solo uno genera nuevos. La tabla `pedidoprinterjob` con timestamps permite auditar.

23. Notificaciones Push: PrinterServices → QuipuNetX → Front (PENDIENTE)

Estado:  PENDIENTE — Plan de implementación

Objetivo: Reemplazar el mecanismo local de `FailedQueueManager` / `PrinterQueueManager` cuando el feature flag `USAR_PRINTER_SERVICE` está activo, por un modelo push donde `PrinterServices` notifica cada cambio de estado de un job a `QuipuNetX`, y el `Front` reacciona en tiempo real desde RAM (sin polling a BD).

23.1 Problema actual

Cuando `USAR_PRINTER_SERVICE` está activo:

- `QuipuNetX` envía comandos a `PrinterServices` y guarda los joblds en `pedidoprinterjob` con `STATUS= SENT`
- Pero **NO hay mecanismo** para que `PrinterServices` notifique de vuelta cuando el job cambió de estado (`DONE`, `FAILED`, `WAITING`, etc.)
- El `Front` no tiene manera de saber si la impresión fue exitosa, falló, o está en espera
- `FailedQueueManager.RetryAllFailedAsync()` opera con impresión LOCAL, no con `PrinterServices`
- Los modales de impresión (`CustomModalImprimiendoNewView`, `CustomModalImpresionFallidaNew`) no reflejan el estado real de `PrinterServices`

23.2 Arquitectura propuesta: RAM-first + BD backup + HTTP callback



23.3 ¿Por qué RAM-first y NO polling a BD?

El patrón existente de `PrinterQueueManager` y `FailedQueueManager` ya probó que funciona:

- Listas en RAM + eventos `Action` → UI se suscribe → reacción instantánea (~0ms)
- `ObservableCollection` + eventos `CommandAdded` / `CommandRemoved` → UI reacciona inmediato
- Cero consultas a BD para actualizar modales

Polling a BD introduciría:

- ~50-100ms por query SQLite
- Timer polling cada 500ms → "saltitos" en la UI
- Percepción de impresión lenta para el usuario
- Inconsistente con el patrón existente

La BD (`pedidoprinterjob`) solo se toca en:

1. **Al crear** (ya existente: `PedidoprinterjobController.guardarJobsPorComandas()` con `STATUS=SENT`)
2. **Al recibir callback** → persist en background con `Task.Run()` , no bloquea UI
3. **Al arrancar** → recovery de jobs pendientes ante caída

23.4 Mapeo de estados: `PrinterServices` → `pedidoprinterjob`

Estado <code>PrinterServices</code>	STATUS en <code>pedidoprinterjob</code>	Quién lo asigna	Significado	Worker lo procesa?
(job encolado)	SENT	QuipuNetX (al crear)	Job enviado y aceptado por PS	—
Printing	PRINTING	Callback PS→QuipuNetX	Worker procesando el job	— (transitorio)
Done	DONE	Callback PS→QuipuNetX	Impresión física exitosa	✗ Terminal
Failed	FAILED	Callback PS→QuipuNetX	Reintentos agotados	✗ Terminal
Waiting	WAITING	Callback PS→QuipuNetX	Impresora offline/sin papel	✓ Sí (via ReEnqueue)
Cancelled	CANCELLED	Callback PS→QuipuNetX	Cancelado por reimpresión (Sec 22)	✗ Terminal
Expired	EXPIRED	PrinterServices	Job WAITING que superó tiempo límite	✗ Terminal

23.5 Nuevo estado: EXPIRED

23.5.1 Concepto

Un job en estado `WAITING` permanece indefinidamente esperando que la impresora vuelva online. Esto no es deseable porque:

- El usuario puede haber resuelto el problema de otra forma (reimprimir manualmente, cambiar de impresora)
- Jobs acumulados en `WAITING` consumen memoria y confunden al operador
- El Front muestra indefinidamente "Esperando impresora..." sin resolución

Solución: Si un job lleva más de X segundos en `WAITING` , marcarlo como `EXPIRED` automáticamente. `PrinterServices` deja de considerarlo.

23.5.2 Configuración: `EXPIRAR_IMPRESION_DESPUES_DE_X`

- **Origen:** Feature flag en QuipuNetX → configuración `EXPIRAR_IMPRESION_DESPUES_DE_X`
- **Valor:** Número entero expresado en **segundos** (ej: `300` = 5 minutos)
- **Flujo de sincronización:** QuipuNetX → `POST /api/printers/sync` → `PrinterServices` persiste en `config_settings`

Cuando QuipuNetX inicializa y sincroniza la lista de impresoras con `POST /api/printers/sync` , el body JSON **también incluirá** este valor:

```
{
  "printers": [ ... ],
  "config": {
    "ExpirarImpresionDespuesDe": 300
  }
}
```

O alternativamente, el endpoint `/api/printers/sync` actual recibe un array de impresoras. Se puede ampliar el body para incluir configuraciones adicionales:


```
[
  { "impresora_id": "1", "ip": "192.168.1.50", "nombre": "COCINA", ... },
  { "impresora_id": "2", "ip": "192.168.1.51", "nombre": "BARRA", ... }
]
```

→ Se crea un **nuevo endpoint** `POST /api/config/sync` que envía las configuraciones de QuipuNetX:

```
{
  "ExpirarImpresionDespuesDe": 300
}
```

PrinterServices lo persiste en su tabla `config_settings` :

KEY	VALUE	CATEGORY	VALUE_TYPE
ExpirarImpresionDespuesDe	300	queue	int

23.5.3 Quién marca EXPIRED y cuándo

StatusMonitor o **PrintWorker** en PrinterServices:

- En `StatusMonitor.RequeueWaitingJobs()` : antes de reencolar un job WAITING, verificar si `(DateTime.Now - job.FechaCreacion).TotalSeconds > ExpirarImpresionDespuesDe`
- Si expiró → `_jobManager.MarkExpired(job)` → NO reencolar
- Si no expiró → reencolar normalmente (flujo actual)

```
StatusMonitor.RequeueWaitingJobs():
foreach job in WAITING:
  if (ahora - job.FechaCreacion) > config.ExpirarImpresionDespuesDe:
    _jobManager.MarkExpired(job) ← nuevo método
    NotifyIfAvailable(n => n.NotifyPrintExpired(...)) ← nueva notificación
    QuipuNetXCallbackClient.NotifyStatusChange(job, "EXPIRED")
  else:
    _jobManager.ReEnqueue(job) ← flujo actual, sin cambios
```

23.5.4 Cambios necesarios para EXPIRED

#	Archivo	Cambio
1	Queue/PrintJob.cs	Agregar Expired al enum PrintJobStatus
2	Queue/PrintJobManager.cs	Agregar MarkExpired(PrintJob job)
3	Monitoring/StatusMonitor.cs	En RequeueWaitingJobs(): check expiración antes de reencolar
4	Config/ConfigManager.cs	Agregar default ExpirarImpresionDespuesDe = 300
5	Notifications/NotificationManager.cs	Agregar constante NotificationType.Expirado = "EXPIRADO" y método NotifyPrintExpired()
6	QuipuNetX: endpoint sync	Enviar EXPIRAR_IMPRESION_DESPUES_DE_X en sync de configuración

23.6 Flujo temporal completo (lo que siente el usuario)

```
t=0ms    Usuario click "Enviar pedido"
t=200ms  QuipuNetX envía a PrinterServices, recibe jobIds
         → PrintJobStatusManager.RegisterJobs() en RAM con STATUS=SENT
         → Task.Run → guardar en SQLite (background, no bloquea)
         → Respuesta al Front con PrintJobResults

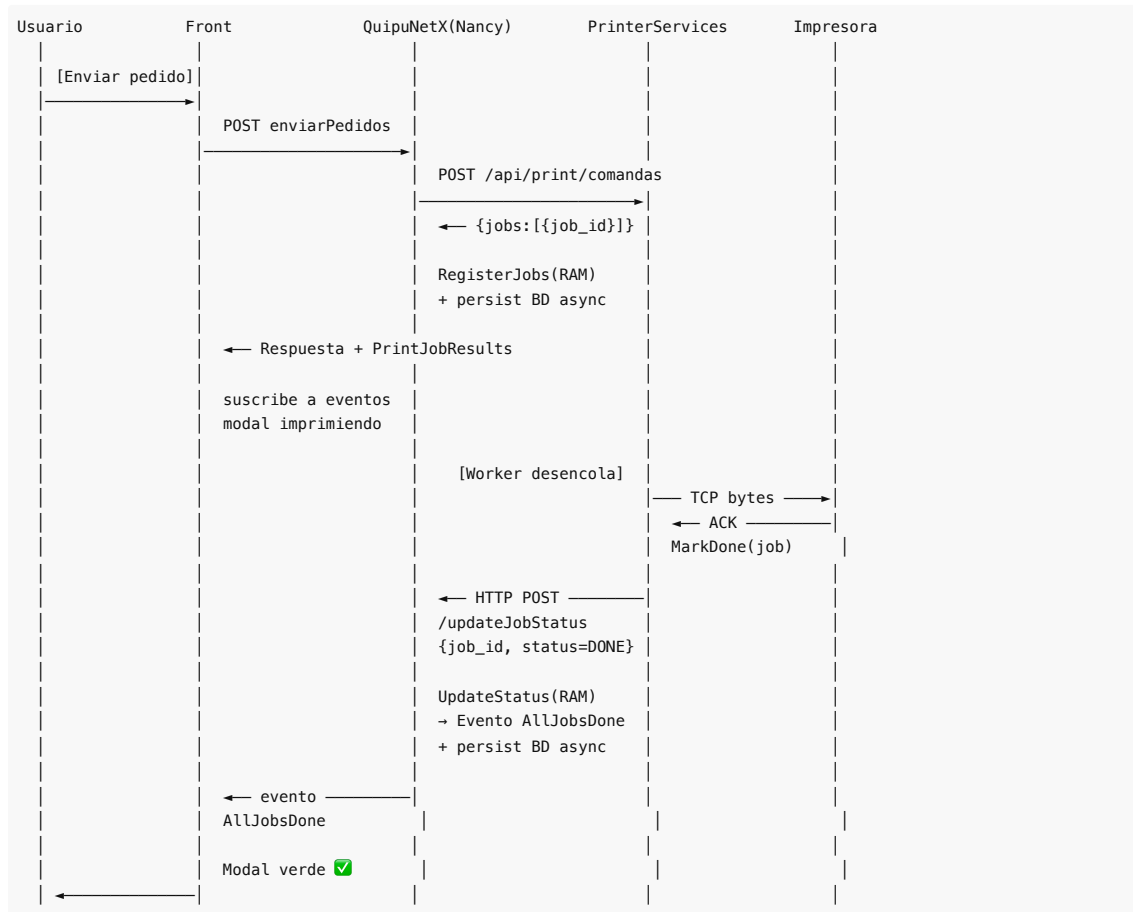
t=250ms  Front recibe respuesta
         → Se suscribe a PrintJobStatusManager.JobStatusChanged
         → showImprimiendoPrinterService() → modal abierto

t=800ms  PrinterServices imprimió OK → HTTP callback → QuipuNetX
         → PrintJobStatusManager.UpdateStatus("714", "DONE")
         → RAM actualizada INMEDIATAMENTE (~0ms)
         → Evento JobStatusChanged dispara
         → Evento AllJobsDone dispara (si todos DONE)
         → Task.Run → UPDATE SQLite (background, no bloquea)
```

t=800ms Front recibe evento AllJobsDone
 → Modal barra → 100% verde "¡Impresión exitosa!"
 → Auto-cierre en 2s

TOTAL PERCIBIDO POR USUARIO: ~800ms (solo el tiempo real de impresión TCP)

23.7 Secuencia de diagrama: Callback HTTP (ruta feliz)



23.8 Failover: Escenario 1 — QuipuNetX se cae y se reinicia

TIMELINE:

t=0 QuipuNetX envía jobs → pedidoprinterjob BD tiene STATUS=SENT

t=1 PrintJobStatusManager RAM tiene {714: SENT, 715: SENT}

t=2 ✗ QuipuNetX CRASH ✗ → RAM perdida

t=3 PrinterServices sigue trabajando:
 Job 714 → imprime OK → intenta callback → QuipuNetX DOWN → falla
 Job 715 → falla → intenta callback → QuipuNetX DOWN → falla
 PrinterServices encola callbacks pendientes en su ConcurrentQueue

t=4 QuipuNetX REINICIA
 → BD sigue intacta: {714: SENT, 715: SENT}
 → PrintJobStatusManager.Initialize():
 Paso 1 (BD → RAM): Cargar pedidoprinterjob
 WHERE STATUS NOT IN ('DONE', 'FAILED', 'CANCELLED', 'EXPIRED')
 → Pobla RAM con los pendientes
 Paso 2 (Sync con PrinterServices):
 POST /api/jobs/status → {job_ids: ["714", "715"]}
 → PS responde: {714: DONE, 715: FAILED}
 → Actualiza RAM + BD con estado real

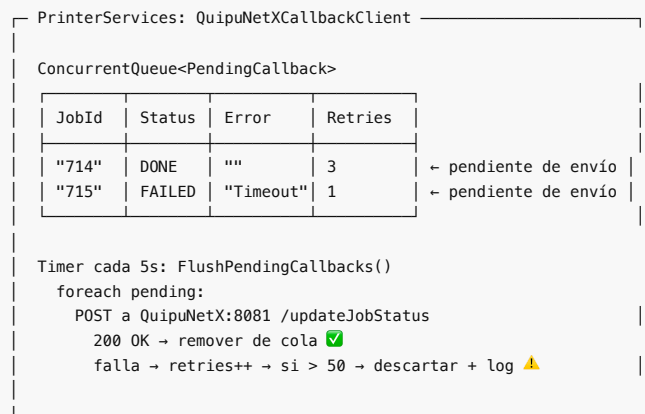
t=5 Timer de PrinterServices tmb intenta flush de callbacks pendientes
 → QuipuNetX ya levantó → recibe OK → doble-write pero IDEMPOTENTE

Clave: UpdateStatus() es **idempotente** — si llega dos veces "DONE" para el mismo job, no pasa nada.

23.9 Failover: Escenario 2 — PrinterServices intenta callback pero QuipuNetX está caído

QuipuNetXCallbackClient (en PrinterServices):

1. Intenta HTTP POST a IpOrigen:8081 → CONNECTION REFUSED
2. Guarda en ConcurrentQueue<PendingCallback>
3. Timer FlushPendingCallbacks() cada 5s reintenta pendientes
4. Cuando QuipuNetX responde 200 → remover de cola
5. Max retries: 50 (~4 minutos de cobertura)
6. Después de max retries: log warning + descartar
(QuipuNetX hará sync al reiniciar con Paso 2)



23.10 Failover: Escenario 3 — PrinterServices se cae y se reinicia

TIMELINE:

- t=0 QuipuNetX envió jobs → pedidoprinterjob BD tiene STATUS=SENT
- t=1 PrinterServices recibió y encoló jobs en su BD
- t=2 ✗ PrinterServices CRASH ✗
- t=3 PrinterServices REINICIA
- RecoverPending() carga PENDING/WAITING de su SQLite (ya existente)
 - Workers procesan normalmente
 - Callbacks se envían al terminar cada job
 - QuipuNetX recibe callback → actualiza RAM + BD

Este escenario ya está cubierto por `PrintJobManager.RecoverPending()`. Los callbacks se dispararán naturalmente.

23.11 Failover: Escenario 4 — AMBOS se caen (peor caso)

- t=0 Ambos caídos
- t=1 PrinterServices reinicia primero → RecoverPending() → procesa jobs
- intenta callback → QuipuNetX DOWN → encola en ConcurrentQueue
- t=2 QuipuNetX reinicia → Initialize():
- BD: carga SENT/WAITING/PRINTING
 - Sync POST /api/jobs/status → PS ya los marcó DONE/FAILED
 - Actualiza RAM + BD con estado real
- t=3 Timer de PS tmb intenta flush de callbacks pendientes
- QuipuNetX ya levantó → recibe OK → idempotente, sin problema

23.12 PrintJobStatusManager: Estructura completa

```
// Archivo: QuipuNetX/PrinterManager/PrintJobStatusManager.cs

public class PrintJobStatusManager
{
    // — Singleton (mismo patrón que PrinterQueueManager) —
    private static readonly Lazy<PrintJobStatusManager> _instance = ...;
    public static PrintJobStatusManager Instance => _instance.Value;

    // — Almacén en RAM (fuente de verdad para la UI) —
    private readonly ConcurrentDictionary<string, PrintJobStatusInfo> _jobs;
    private readonly object _lock = new object();
```

```

// — Eventos (mismo patrón que PrinterQueueManager/FailedQueueManager) —
public event Action<string, string> JobStatusChanged; // (jobId, newStatus)
public event Action<List<string>> AllJobsDone; // (pedidoIds) todos completados
public event Action<string, string> AnyJobFailed; // (jobId, errorMsg)
public event Action<string, string> AnyJobWaiting; // (jobId, reason)
public event Action<string> AnyJobExpired; // (jobId) expiró
public event Action OpenProgress; // Abrir modal imprimiendo
public event Action HideProgress; // Cerrar modales

// — Registro: llamado por PedidoController.addLista() —
public void RegisterJobs(List<PrintJobResult> jobResults)
{
    // 1. Agregar cada job al diccionario RAM con status=SENT
    // 2. Disparar OpenProgress
    // 3. Task.Run → guardar en SQLite (guardarJobsPorComandas ya existe)
}

// — Actualización: llamado por endpoint HTTP callback —
public void UpdateStatus(string jobId, string newStatus, string error = "")
{
    // 1. Actualizar RAM → INMEDIATO
    // 2. Disparar JobStatusChanged
    // 3. Evaluar: ¿todos DONE? → AllJobsDone
    //           ¿alguno FAILED? → AnyJobFailed
    //           ¿alguno WAITING? → AnyJobWaiting
    //           ¿alguno EXPIRED? → AnyJobExpired
    // 4. Si todos terminales (DONE/FAILED/EXPIRED/CANCELLED) → HideProgress
    // 5. Task.Run → UPDATE SQLite (background)
    // NOTA: Idempotente – si ya tiene ese status, no hace nada
}

// — Recovery ante caída —————
public void Initialize()
{
    // Paso 1: BD → RAM
    // SELECT * FROM pedidoprinterjob
    // WHERE STATUS NOT IN ('DONE','FAILED','CANCELLED','EXPIRED')
    // Poblar _jobs con los pendientes

    // Paso 2: Sync con PrinterServices (background, no bloquea arranque)
    // Task.Run → POST /api/jobs/status {job_ids: [...]}
    // Para cada job: actualizar RAM + BD con estado real de PS
}

// — Consultas desde Front (sin tocar BD, todo en RAM) —
public PrintJobStatusInfo GetJob(string jobId) { ... }
public List<PrintJobStatusInfo> GetJobsByPedido(string pedidoId) { ... }
public List<PrintJobStatusInfo> GetActiveJobs() { ... } // no-terminales
public List<PrintJobStatusInfo> GetFailedJobs() { ... }

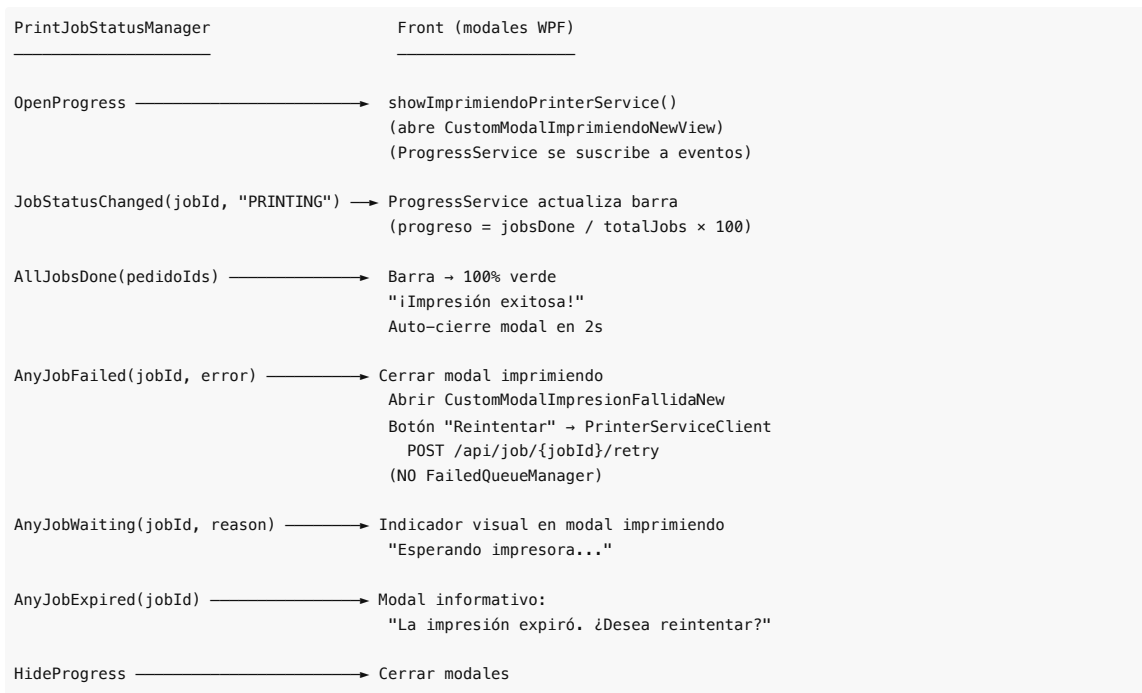
// — Limpieza —————
public void Clear() { ... } // Limpiar ciclo terminado
}

public class PrintJobStatusInfo
{
    public string JobId { get; set; }
    public string Status { get; set; } // SENT, PRINTING, DONE, FAILED, WAITING, CANCELLED, EXPIRED
    public List<string> PedidoIds { get; set; }
    public string Error { get; set; }
    public string ImpresoraNombre { get; set; }
    public string AreaImpresion { get; set; }
    public DateTime FechaRegistro { get; set; }
    public DateTime? FechaUltimaActualizacion { get; set; }
}

```

23.13 Cómo se conecta con los modales existentes del Front

Cuando `USAR_PRINTER_SERVICE` está ON, el Front NO usa `PrinterQueueManager / FailedQueueManager`. En su lugar, se suscribe a los eventos de `PrintJobStatusManager`:



23.14 Reintentar con PrinterServices (no FailedQueueManager)

ANTES (flujo local, flag OFF):
Modal Fallida → "Reintentar" → FailedQueueManager.RetryAllFailedAsync()
→ command.ExecutePrintAsync() → impresión local TCP

DESPUÉS (flujo PrinterServices, flag ON):
Modal Fallida → "Reintentar" → PrinterServiceClient.RetryJobAsync(jobId)
→ POST /api/job/{jobId}/retry → PS reencola el job
→ PS Worker imprime → callback → UpdateStatus("714", "DONE")
→ Evento AllJobsDone → Modal se cierra verde

23.15 Feature flag EXPIRAR_IMPRESION_DESPUES_DE_X: Flujo de sincronización



```

        .TotalSeconds;
    if (elapsed > maxSeconds):
        _jobManager.MarkExpired(job)      ← EXPIRED
        callback a QuipuNetX: EXPIRED
    else:
        _jobManager.ReEnqueue(job)        ← flujo actual

```

23.16 Enum PrintJobStatus actualizado (PrinterServices)

```

public enum PrintJobStatus
{
    Pending,      // Recién creado, en cola
    Printing,     // Worker procesando
    Done,         // Impresión exitosa
    Failed,       // Reintentos agotados
    Waiting,      // Impresora offline/sin papel
    Cancelled,    // Cancelado por reimpresión (Sección 22)
    Expired       // WAITING que superó tiempo límite (Sección 23)
}

```

23.17 Plan de implementación: Pasos pendientes

Fase A: PrinterServices — Callback HTTP a QuipuNetX

Paso	Archivo	Acción	Proyecto
A.1	Services/QuipuNetXCallbackClient.cs	CREAR — HTTP POST callback con cola de retry	PrinterServices
A.2	Workers/PrintWorker.cs	MODIFICAR — llamar callback después de cada NotifyIfAvailable()	PrinterServices
A.3	Monitoring/StatusMonitor.cs	MODIFICAR — llamar callback en RequeueWaitingJobs()	PrinterServices

Fase B: PrinterServices — Estado EXPIRED

Paso	Archivo	Acción	Proyecto
B.1	Queue/PrintJob.cs	MODIFICAR — agregar Expired al enum PrintJobStatus	PrinterServices
B.2	Queue/PrintJobManager.cs	MODIFICAR — agregar MarkExpired(PrintJob job)	PrinterServices
B.3	Monitoring/StatusMonitor.cs	MODIFICAR — check expiración en RequeueWaitingJobs()	PrinterServices
B.4	Config/ConfigManager.cs	MODIFICAR — agregar default ExpirarImpresionDespuesDe = 300	PrinterServices
B.5	Notifications/NotificationManager.cs	MODIFICAR — agregar NotificationType.Expirado + NotifyPrintExpired()	PrinterServices

Fase C: PrinterServices — Endpoint bulk status + config sync

Paso	Archivo	Acción	Proyecto
C.1	Api/Controllers/JobController.cs	MODIFICAR — agregar GetJobsStatus(body) para bulk query	PrinterServices
C.2	Api/Controllers/ConfigController.cs	CREAR — endpoint POST /api/config/sync	PrinterServices
C.3	Api/ApiRouter.cs	MODIFICAR — agregar rutas /api/jobs/status y /api/config/sync	PrinterServices

Fase D: QuipuNetX — Recibir callbacks + Manager en RAM

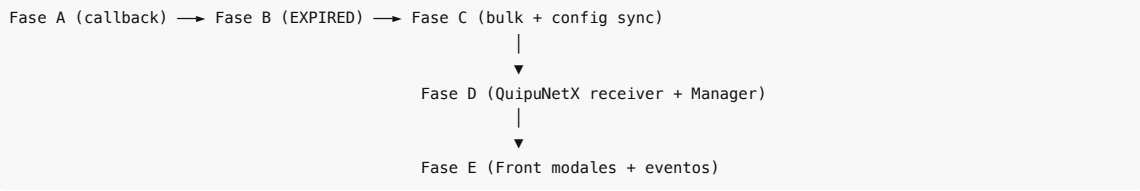
Paso	Archivo	Acción	Proyecto
D.1	PrinterManager/PrintJobStatusManager.cs	CREAR — Singleton en RAM con eventos	QuipuNetX
D.2	PrinterManager/PrintJobStatusInfo.cs	CREAR — clase de datos para cada job	QuipuNetX
D.3	ws/modules/PrinterServiceModule.cs	CREAR — Nancy module con endpoint /api/rest/printerservice/updateJobStatus	QuipuNetX
D.4	controller/PedidoPrinterJobController.cs	MODIFICAR — agregar actualizarEstadoJob()	QuipuNetX
D.5	controller/PedidoController.cs	MODIFICAR — llamar PrintJobStatusManager.RegisterJobs() después de guardar	QuipuNetX
D.6	Services/Print/PrinterServiceClient.cs	MODIFICAR — agregar GetJobsStatusBulk() y SyncConfigAsync()	QuipuNetX

D.7	Inicialización QuipuNetX	MODIFICAR — llamar <code>PrintJobStatusManager.Initialize()</code> al arrancar	QuipuNetX
D.8	Sincronización impresoras	MODIFICAR — enviar <code>EXPIRAR_IMPRESION_DESPUES_DE_X</code> en sync	QuipuNetX

Fase E: Front — Modales suscritos a eventos

Paso	Archivo	Acción	Proyecto
E.1	Presenters (ListaPedidosTemporales, etc.)	MODIFICAR — suscribirse a eventos del manager cuando flag ON	QuipuNet (Front)
E.2	ProgressService (o nuevo)	CREAR/MODIFICAR — adaptar para leer de <code>PrintJobStatusManager</code>	QuipuNet (Front)
E.3	Modal Fallida	MODIFICAR — "Reintentar" llama <code>PrinterServiceClient.RetryJobAsync()</code> cuando flag ON	QuipuNet (Front)
E.4	Modal Expirado	CREAR — informar expiración, opción de reimprimir	QuipuNet (Front)

23.18 Orden de implementación recomendado



Dependencias:

- Fase B depende de Fase A (EXPIRED necesita callback para notificar a QuipuNetX)
- Fase C es independiente, puede hacerse en paralelo con B
- Fase D depende de A+C (necesita el callback y el endpoint bulk)
- Fase E depende de D (necesita el manager en RAM funcionando)

23.19 Resumen de garantías

Garantía	Cómo se logra
Tiempo real para UI	RAM-first + eventos C# (mismo patrón que <code>PrinterQueueManager</code>)
Persistencia ante caída	BD <code>pedidoprinterjob</code> como backup, escrita async
Recovery QuipuNetX	<code>Initialize()</code> : BD→RAM + sync bulk con <code>PrinterServices</code>
Recovery PrinterServices	<code>RecoverPending()</code> ya existente + callbacks al completar
Callback resiliente	Cola de retry en <code>PrinterServices</code> + sync bulk como fallback
Idempotencia	<code>UpdateStatus()</code> ignora si el estado ya es igual
Jobs expirados	EXPIRED automático por <code>StatusMonitor</code> según config <code>ExpirarImpresionDespuesDe</code>
Config centralizada	Feature flag de QuipuNetX sincronizado a <code>config_settings</code> de <code>PrinterServices</code>

23.20 REGLA DE ORO: Feature flags separan flujos — NUNCA mezclar

Principio fundamental

El feature flag `USAR_PRINTER_SERVICE` actúa como **interruptor de flujo completo**:

```
if (flag OFF) → flujo LOCAL intacto (PrinterQueueManager, FailedQueueManager, ProgressService)
if (flag ON) → flujo PRINTER_SERVICES nuevo (PrintJobStatusManager, PSProgressService, PSRetryService)
```

NUNCA meter lógica de `PrinterServices` dentro de las clases del flujo local. **NUNCA** meter lógica local dentro de las clases del flujo `PrinterServices`. Son dos mundos paralelos que coexisten, activados por un solo flag.

Patrón de bifurcación en Presenters (ya implementado)

```
// CORRECTO: El presenter bifurca UNA VEZ al inicio, delega a métodos separados
if (respuesta.ImpresionList != null && respuesta.ImpresionList.Count > 0)
{
    // Flujo LOCAL → intacto, no se toca
```

```

        this.imprimirComandas(respuesta.ImpresionList);
    }
    else if (FeatureFlagManager.Instance.IsEnabled("USAR_PRINTER_SERVICE"))
    {
        // Flujo PRINTER_SERVICES → clase dedicada maneja todo
        this.verificarPrintJobsPorComandas(respuesta, view);
    }
}

```

```

// INCORRECTO: Mezclar lógica de ambos flujos en el mismo método
if (flag)
{
    // ... 50 líneas de lógica PrinterServices ...
    PrinterQueueManager.Instance.Enqueue(...); // ❌ Mezclando flujos
    PrintJobStatusManager.Instance.RegisterJobs(...);
}

```

Regla para CADA clase nueva

Toda clase nueva del flujo PrinterServices debe:

1. **Existir en paralelo** a su equivalente local (no reemplazarla)
2. **No importar** ni referenciar clases del flujo local (ni viceversa)
3. **Activarse solo** cuando el flag está ON
4. **No modificar** el comportamiento del flujo local cuando el flag está OFF

23.21 Inventario: Clases del flujo LOCAL que NO se modifican

Estas clases son del flujo de impresión local. **NO SE TOCAN** en esta implementación. Solo se les agrega un `if (flag)` en el punto de entrada del presenter para desviar al flujo nuevo.

Clase	Archivo	Responsabilidad	Se modifica?
PrinterQueueManager	QuipuNetX/PrinterManager/PrinterQueueManager.cs	Cola de impresión local + eventos ProgressChanged/OpenProgress/HideProgress	❌ NO TOCAR
FailedQueueManager	QuipuNetX/PrinterManager/FailedQueueManager.cs	Cola de comandos fallidas + RetryAllFailedAsync() + eventos CommandAdded/CommandRemoved	❌ NO TOCAR
ProgressService	Front/MainApp/Services/ProgressService.cs	Timer WPF que lee PrinterQueueManager para barra de progreso del modal imprimiendo	❌ NO TOCAR
ModaImpresionFallidaViewModel	Front/MainApp/.../ModaImpresionFallidaViewModel.cs	ViewModel del modal fallida, llama FailedQueueManager.RetryAllFailedAsync()	❌ NO TOCAR
ImpresionService	Front/Services/Print/ImpresionService.cs	Singleton de impresión con strategy pattern	❌ NO TOCAR
PrintComandaCommand	QuipuNetX/PrinterManager/...	Comando que ejecuta impresión TCP local	❌ NO TOCAR
IPrinterCommand	QuipuNetX/PrinterManager/Interfaces/...	Interfaz de comando de impresión	❌ NO TOCAR

23.22 Buenas prácticas de código limpio para la implementación

22.1 Métodos pequeños con responsabilidad única (SRP)

Cada método hace UNA cosa. Si necesita hacer dos, se divide en dos.

```

// ❌ INCORRECTO: Método gigante que hace todo
public void UpdateStatus(string jobId, string newStatus, string error)
{
    // ... 80 líneas: buscar en RAM, actualizar, evaluar todos, evaluar fallidos,
    // disparar eventos, persistir en BD, loggear ...
}

// ✅ CORRECTO: Método orquestador que delega a métodos pequeños
public void UpdateStatus(string jobId, string newStatus, string error = "")
{
    if (!ActualizarEnRam(jobId, newStatus, error)) return; // Idempotente: si ya tiene ese status, sale
    DispararEventosCambio(jobId, newStatus, error);        // Evalúa y dispara eventos según estado
}

```



```

        PersistirEnBackground(jobId, newStatus);                // Task.Run → SQLite async
    }

    private bool ActualizarEnRam(string jobId, string newStatus, string error) { ... } // 10 líneas
    private void DispararEventosCambio(string jobId, string newStatus, string error) { ... } // 15 líneas
    private void PersistirEnBackground(string jobId, string newStatus) { ... } // 5 líneas

```

22.2 Clases con responsabilidad limitada

Clase	Responsabilidad ÚNICA	NO hace
PrintJobStatusManager	Almacén RAM + eventos	No persiste en BD directamente, no hace HTTP
PrintJobStatusPersister	Escribe/lee BD pedidoprinterjob	No dispara eventos, no conoce la UI
PrintJobStatusRecovery	Recovery al arrancar (BD→RAM + sync PS)	Solo se ejecuta una vez al iniciar
QuipuNetXCallbackClient	HTTP POST fire-and-forget + cola retry	No modifica BD, no conoce el Manager
PSProgressService	Lee del Manager, expone propiedades para WPF binding	No modifica estados, no hace HTTP
PSRetryService	Llama POST /api/job/{id}/retry	No modifica estados localmente

22.3 Nomenclatura: Prefijo PS para clases del flujo PrinterServices

Para distinguir visualmente las clases del flujo PrinterServices de las del flujo local:

Flujo LOCAL:	Flujo PRINTER_SERVICES:
ProgressService	PSProgressService
ModaImpresionFallidaViewModel	PSModaImpresionFallidaViewModel
PrinterQueueManager	PrintJobStatusManager (ya tiene nombre propio)
FailedQueueManager	(no tiene equivalente: el retry va a PS API)

22.4 Eventos con firma simple

Los eventos del PrintJobStatusManager usan firmas simples (Action<string, string>) por consistencia con PrinterQueueManager :

```

// Mismo patrón que PrinterQueueManager.ProgressChanged (Action sin params)
public event Action<string, string> JobStatusChanged; // (jobId, newStatus)
public event Action<List<string>> AllJobsDone;        // (pedidoIds)
public event Action<string, string> AnyJobFailed;     // (jobId, error)

```

22.5 Cada archivo nuevo ≤ 150 líneas

Si un archivo supera ~150 líneas, es señal de que tiene demasiada responsabilidad. Dividir en archivos más pequeños:

PrintJobStatusManager.cs	→ RAM + eventos (~120 líneas)
PrintJobStatusPersister.cs	→ BD async (~60 líneas)
PrintJobStatusRecovery.cs	→ Recovery al arrancar (~80 líneas)
PrintJobStatusInfo.cs	→ Clase de datos (~30 líneas)

22.6 Comentarios explicativos en cada línea

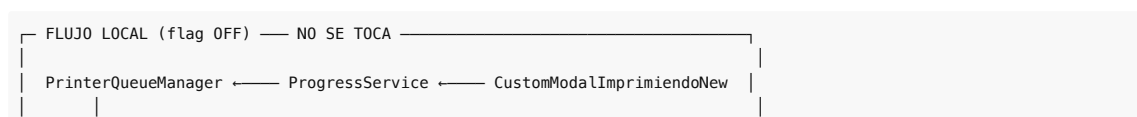
Según la convención del proyecto, todo código nuevo incluye comentarios // en cada línea explicando qué hace:

```

public bool ActualizarEnRam(string jobId, string newStatus, string error)
{
    PrintJobStatusInfo info;                // Variable para almacenar el job actual
    if (!_jobs.TryGetValue(jobId, out info)) return false; // Si el job no existe en RAM, salir
    if (info.Status == newStatus) return false; // Idempotente: si ya tiene ese status, no hacer nada
    info.Status = newStatus;                // Actualizar estado en la referencia en RAM
    info.Error = error;                    // Actualizar mensaje de error (puede ser vacío)
    info.FechaUltimaActualizacion = DateTime.Now; // Registrar timestamp de última actualización
    return true;                          // Retornar true indicando que hubo cambio
}

```

23.23 Mapa de clases: Existentes vs Nuevas por fase



```

└─ FailedQueueManager ← ModaImpresionFallidaViewModel
    │
    └─ ReintentarCommand
    └─ CancelarCommand

ImpresionService → ImpresionStrategyFactory → ComandasPrintStrategy

```

FLUJO PRINTER_SERVICES (flag ON) — TODO NUEVO

Fase D: QuipuNetX

PrintJobStatusManager (RAM + eventos)	← D.1 CREAR
PrintJobStatusInfo (datos)	← D.2 CREAR
PrintJobStatusPersister (BD async)	← D.1 CREAR (inner)
PrintJobStatusRecovery (arranque)	← D.1 CREAR (inner)
PrinterServiceModule (Nancy endpoint)	← D.3 CREAR
PedidoPrinterJobController.actualizarEstado()	← D.4 MODIFICAR

Fase E: Front

PSProgressService (lee de PrintJobStatusManager)	← E.2 CREAR
PSRetryService (llama PS API /retry)	← E.2 CREAR
Presenters: bifurcación if(flag) → delegar	← E.1 MODIFICAR
Modal Fallida: if(flag) → PSRetryService	← E.3 MODIFICAR
Modal Expirado (nuevo)	← E.4 CREAR

Fase A+B+C: PrinterServices

QuipuNetXCallbackClient (HTTP + cola retry)	← A.1 CREAR
PrintWorker → callback después de notify	← A.2 MODIFICAR
PrintJobStatus.Expired (enum)	← B.1 MODIFICAR
PrintJobManager.MarkExpired()	← B.2 MODIFICAR
StatusMonitor: check expiración	← B.3 MODIFICAR
ConfigController (POST /api/config/sync)	← C.2 CREAR
JobController.GetJobsStatus (bulk)	← C.1 MODIFICAR

PUNTOS DE BIFURCACIÓN (únicos lugares donde se modifica código existente)

```

ListaPedidosTemporalesPresenter:
  enviarPedidos()      → if (flag ON) → verificarPrintJobsPorComandas()
  enviarPedidosOld()   → if (flag ON) → verificarPrintJobsPorComandas()
  enviarPedidosSelfSrv → if (flag ON) → verificarPrintJobsPorComandas()
  † YA IMPLEMENTADO en Paso 8

FragmentDetallePedidoPresenter:
  reimprimirPedido()   → if (flag ON) → PSRetryService (Fase E)
  † PENDIENTE

ModaImpresionFallidaViewModel:
  ReintentarCommand    → if (flag ON) → PSRetryService.RetryAllAsync()
  † PENDIENTE — agregar bifurcación, NO modificar lógica local

ProgressService / CustomModalImprimiendoNewView:
  constructor          → if (flag ON) → usar PSProgressService en su lugar
  † PENDIENTE — la VIEW decide qué service instanciar, no el service

```

23.24 Reglas de modificación para archivos existentes del Front

Cuando sea necesario agregar bifurcación en archivos existentes:

Regla 1: La bifurcación se hace en el **punto de entrada** (el método que decide), NO en lo profundo de la lógica.

```

// ✅ CORRECTO: Bifurcación al inicio del flujo
if (FeatureFlagManager.Instance.IsEnabled("USAR_PRINTER_SERVICE"))

```

```

{
    this.manejarConPrinterServices(respuesta, view); // Método nuevo, dedicado
}
else
{
    this.manejarConImpresionLocal(respuesta, view); // Método existente, sin cambios
}

```

```

// ✗ INCORRECTO: Bifurcación enterrada en la lógica profunda
public void OnTimerTick(...)
{
    if (FeatureFlagManager.Instance.IsEnabled("USAR_PRINTER_SERVICE"))
    {
        // 40 líneas de lógica PS mezclada con lógica local
    }
    else
    {
        // lógica local
    }
}

```

Regla 2: Máximo 1 **check de flag** por método. Si necesitas más, el método es demasiado grande.

Regla 3: La línea de código que lee el flag se encapsula:

```

// Helper reutilizable en todos los presenters
private bool usaPrinterServices()
{
    return FeatureFlagManager.Instance.IsEnabled("USAR_PRINTER_SERVICE");
}

```

Regla 4: Los métodos nuevos que manejan el flujo PrinterServices se nombran descriptivamente:

```

// ✔ Nombres que indican el flujo
private void manejarImpresionConPrinterServices(Respuesta r, IView v) { ... }
private void reintentarConPrinterServices(string jobId) { ... }

// ✗ Nombres ambiguos
private void manejarImpresion2(Respuesta r, IView v) { ... }
private void reintentarNuevo(string jobId) { ... }

```

23.25 Regla crítica: Dispatcher en métodos show*PrinterService (WPF Threading)

Contexto: Los métodos `verificarPrintJobsPorComandas`, `enviarPedido`, etc. en los Presenters se ejecutan en **hilos de fondo** (async/await, Task.Run). Los métodos de la vista (`showFailedPrinterService`, `showImprimiendoPrinterService`, `showSuccessPrinterService`) crean ventanas WPF (`Window.Show()`), que **SOLO pueden ejecutarse en el UI thread**.

Regla: Todo método `show*` en `ListaPedidosTemporales.cs` (y cualquier vista) que cree ventanas WPF **DEBE** envolver su contenido en `Application.Current.Dispatcher.BeginInvoke(...)` .

```

// ✔ CORRECTO – envuelve en Dispatcher (funciona desde hilo de fondo)
public void showFailedPrinterService()
{
    Application.Current.Dispatcher.BeginInvoke(new Action(() =>
    {
        var viewModel = new PSModaImpresionFallidaViewModel();
        var modal = new CustomModalImpresionFallidaNew(viewModel);
        modal.Show(); // UI thread ✓
    }));
}

// ✗ INCORRECTO – sin Dispatcher (falla silenciosamente desde hilo de fondo)
public void showFailedPrinterService()
{
    var viewModel = new PSModaImpresionFallidaViewModel();
    var modal = new CustomModalImpresionFallidaNew(viewModel);
    modal.Show(); // InvalidOperationException o fallo silencioso
}

```

Referencia: `showErrorRegister` y `showSuccessRegister` ya siguen este patrón correctamente. Cualquier método nuevo que muestre UI desde un presenter DEBE hacer lo mismo.

Síntoma si se omite: El toast/modal simplemente no aparece. No hay excepción visible en la UI — falla silenciosamente porque WPF no puede crear controles fuera del UI thread.

23.27 Fixes implementados: Expiración de jobs + UI de progreso (Feb 2026)

23.27.1 Bug: Jobs WAITING no expiraban cuando impresora permanecía offline

Síntoma: Con `ExpirarImpresionDespuesDe=10`, los jobs WAITING nunca expiraban si la impresora permanecía offline.

Causa raíz: La expiración estaba en `StatusMonitor.RequeueWaitingJobs()`, que solo se ejecutaba cuando una impresora transitaba de offline a online. Si permanecía offline, nunca se llamaba.

Solución: Loop periódico `ExpirationLoop` en `PrintWorker` (cada 5s) que consulta BD buscando jobs WAITING que superaron el tiempo configurado → `MarkExpired` + `LogPrint("EXPIRED")` + callback HTTP + gRPC.

Archivos: `PrinterServices/Workers/PrintWorker.cs` — nuevo `ExpirationLoop()` + segundo `Task.Factory.StartNew` en `Start()`.

23.27.2 Bug: Retry de jobs EXPIRED rechazado por PrinterServices

Síntoma: Al clicar Reintentar en un job EXPIRED, PS respondía 400 BadRequest.

Causa raíz: `JobController.RetryJob()` solo aceptaba `PrintJobStatus.Failed`.

Solución:

- `JobController.cs` : acepta `Failed` y `Expired`
- `PrintJobManager.Retry()` : resetea `FechaCreacion = DateTime.Now` (evita re-expiración inmediata)
- `PrintJobManager.Retry()` : escribe `RETRIED` en `print_log` con estado previo

23.27.3 Bug: "Error al reintentar" cuando PS respondía "RETRIED"

Causa raíz: `ParseSuccessResponse()` esperaba `status="OK"` pero PS respondía `"RETRIED"`. Además, la rama `else` en `RetryAllFailedJobsAsync()` sobrescribía `DONE` con `FAILED`.

Solución:

- `PrinterServiceClient.ParseSuccessResponse()` : acepta cualquier status no nulo si HTTP 200
- `RetryAllFailedJobsAsync()` rama `else`: solo revierte a `FAILED` si job sigue `PENDING`

23.27.4 Bug: pedidoprinterjob no se actualizaba al reintentar

Solución: `RetryAllFailedJobsAsync()` llama `PedidoprinterjobController.actualizarEstadoJob()` al marcar `PENDING` y al revertir a `FAILED`. Agregado `using QuipuNetX.controller`.

23.27.5 Bug: Barra de progreso verde "Impresión exitosa" cuando había errores

Síntoma: Modal "Imprimiendo X documento(s)" mostraba barra verde al 100% incluso cuando todos los jobs expiraron.

Causa raíz (principal): `CustomModalImprimiendoNewView.xaml` bindeaba `HayErroresEnCola` a `TieneError` (propiedad legacy sin efecto en triggers) en vez de `EsErrorProgress` (la que controlan los triggers XAML de color).

```
ProgressBar control tiene 2 propiedades:
├─ TieneError (legacy)      ← NO conectada a triggers XAML
└─ EsErrorProgress         ← controla triggers: rojo si true, verde si Value=100+false
```

Causa raíz (secundaria): Race condition temporal — `OnJobStatusChanged` actualizaba `Progreso` a 100% sin marcar `HayErroresEnCola=true`, generando un gap donde el trigger verde se activaba.

Solución:

- `CustomModalImprimiendoNewView.xaml` : cambiar `TieneError` → `EsErrorProgress`
- `PSProgressService.OnJobStatusChanged()` : al detectar `FAILED/EXPIRED`, marcar `HayErroresEnCola=true` ANTES de actualizar `Progreso`
- `PSModaImpresionFallidaViewModel.OnJobStatusChanged()` : mismo fix + actualizar `CantidadErrores` y `MotivoError`

Triggers XAML del ProgressBar (referencia):

```
<!-- Rojo: EsErrorProgress=True -->
<DataTrigger Binding="{Binding EsErrorProgress}" Value="True">
  <Setter Property="Background" Value="#F45B69"/>
</DataTrigger>
<!-- Verde: Value=100 AND EsErrorProgress=False -->
<MultiDataTrigger>
  <Condition Binding="{Binding Value}" Value="100"/>
  <Condition Binding="{Binding EsErrorProgress}" Value="False"/>
```

```
<Setter Property="Background" Value="#9AD791"/>
</MultiDataTrigger>
```

23.27.6 Resumen de archivos modificados

Proyecto	Archivo	Cambios
PrinterServices	Workers/PrintWorker.cs	ExpirationLoop() + LogPrint("EXPIRED")
PrinterServices	Api/Controllers/JobController.cs	RetryJob() acepta EXPIRED
PrinterServices	Queue/PrintJobManager.cs	Retry() resetea FechaCreacion + escribe RETRIED en print_log
QuipuNetX	Services/Print/PrinterServiceClient.cs	ParseSuccessResponse() acepta status no nulo + RetryAllFailedJobsAsync() protege else branch + actualiza pedidoprinterjob
Front	Xaml/.../CustomModalImprimiendoNewView.xaml	TieneError → EsErrorProgress
Front	MainApp/Services/PSProgressService.cs	OnJobStatusChanged() marca error antes de progreso
Front	MainApp/Services/PSModaImpresionFallidaViewModel.cs	OnJobStatusChanged() marca error + recalcula motivos

23.28 Feature: Badge visual "Impr. Fallida" en lista de pedidos

23.28.1 Contexto y problema

La lista de pedidos (`ListaPedidos`) tiene una feature visual **preparada pero desconectada**: cuando `Pedido.IsPrinterComandaFailed = true` , el XAML muestra un badge animado rojo con texto "Impr. Fallida" y un ícono de impresora pulsante.

Pedido: Lomo Saltado

Impr. Fallida ← badge

Pedido: Ceviche

(pulsa rojo ↔ gris cada 1.5s)

(sin badge = impresión OK o pendiente)

Estado actual:

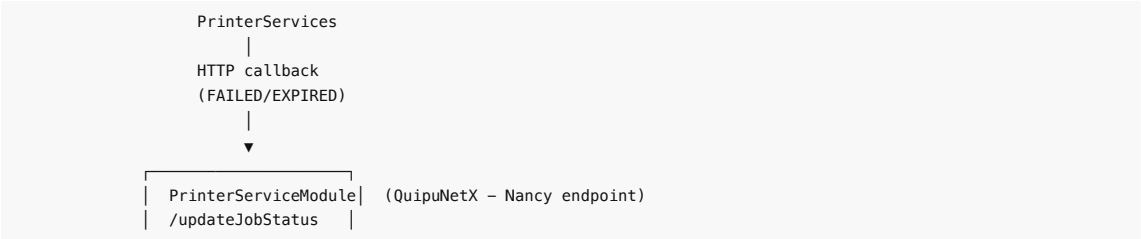
- `PedidoDB.IsPrinterComandaFailed` : propiedad [Ignore] (RAM) con `OnPropertyChanged` ✓
- `ListItemPedido.xaml` : DataTriggers con animaciones `RepeatBehavior="Forever"` ✓
- `ListaPedidos.cs` : loop que pone `IsPrinterComandaFailed = false` al cargar ✗ (siempre false)
- **Nadie pone `IsPrinterComandaFailed = true` → badge nunca aparece**

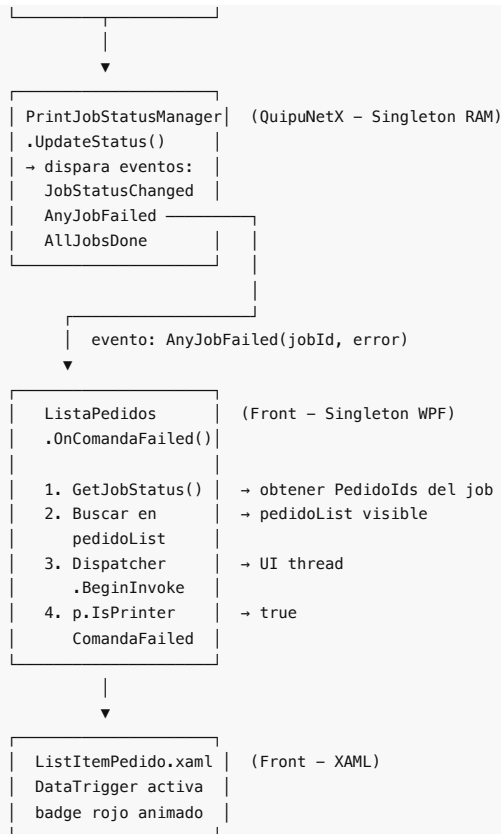
Problema: Los callbacks FAILED/EXPIRED llegan a `QuipuNetX` via `PrinterServiceModule` → `PrintJobStatusManager` , pero nadie conecta ese evento con los pedidos visibles en la lista.

23.28.2 Componentes existentes (no requieren cambios)

Componente	Ubicación	Rol
<code>PedidoDB.IsPrinterComandaFailed</code>	<code>QuipuNetX/entitydb/PedidoDB.cs</code>	Propiedad bool [Ignore] + <code>OnPropertyChanged</code>
<code>ListItemPedido.xaml</code>	<code>Front/Xaml/ListItemPedido.xaml</code>	3 DataTriggers: Border visible, ícono rojo, texto rojo — todos con animación pulsante
<code>PrintJobStatusManager.AnyJobFailed</code>	<code>QuipuNetX/Services/Print/PrintJobStatusManager.cs</code>	Evento <code>Action<string, string></code> que se dispara cuando un job es FAILED/EXPIRED
<code>PrintJobStatusInfo.PedidoIds</code>	<code>QuipuNetX/Services/Print/PrintJobStatusInfo.cs</code>	Lista de <code>pedido_id</code> asociados a cada job (en RAM)
<code>PedidoprinterjobController</code>	<code>QuipuNetX/controller/PedidoprinterjobController.cs</code>	CRUD para tabla <code>pedidoprinterjob</code> (<code>jobId</code> ↔ <code>pedidoId</code>)

23.28.3 Arquitectura de la conexión





23.28.4 Flujo temporal: Callback → Badge visible

```

t=0ms    PrinterServices envía callback: { job_id: "abc123", status: "FAILED", error: "Timeout" }
|
t=5ms    PrinterServiceModule.ProcessUpdateJobStatus()
|
|   PrintJobStatusManager.UpdateStatus("abc123", "FAILED", "Timeout")
|   |   Actualiza RAM: _jobs["abc123"].Status = "FAILED"
|   |   Dispara JobStatusChanged("abc123", "FAILED")
|   |   Dispara AnyJobFailed("abc123", "Timeout") ← NUEVO listener
|   |   PedidoprinterjobController.actualizarEstadoJob("abc123", "FAILED")
|   |   UPDATE pedidoprinterjob SET status='FAILED' WHERE jobid='abc123'
|
t=10ms   ListaPedidos.OnComandaFailed("abc123", "Timeout")
|   |   PrintJobStatusManager.GetJobStatus("abc123") → PedidoIds: ["1772", "1773"]
|   |   HashSet<string> pedidoIdsFallidos = {"1772", "1773"}
|   |   Dispatcher.BeginInvoke:
|   |   |   foreach pedido in pedidoList:
|   |   |   |   if pedidoIdsFallidos.Contains(pedido.Pedido_id):
|   |   |   |   |   pedido.IsPrinterComandaFailed = true ← OnPropertyChanged
|   |
t=15ms   WPF DataTrigger detecta IsPrinterComandaFailed = true
|   |   Border brComandaFailed → Visibility.Visible
|   |   Storyboard sbComandaFailedBg → Background pulsa #FFF5F5 ↔ #F4F4F4
|   |   Storyboard sbIconFg → Ícono pulsa #EF4444 ↔ #646464
|   |   Storyboard sbTxtFg → Texto "Impr. Fallida" pulsa #EF4444 ↔ #646464
  
```

23.28.5 Flujo: Recarga de lista (cambio de mesa/vista)

ANTES (bug): showPedidosListView ponía IsPrinterComandaFailed = false para todos → badge desaparecía al cambiar de mesa y volver.

AHORA: Al recargar la lista, consultar el estado real de cada pedido en pedidoprinterjob :

```

showPedidosListView() → ActualizarPedidosObservable()
|
▼
MarcarPedidosConFalloDeImpresion(pedidoList):
  foreach pedido in pedidoList:
  
```

```

var registro = Pedidoprinterjob.findByPedidoId(pedido.Pedido_id)
if registro != null AND (registro.Status == "FAILED" OR registro.Status == "EXPIRED"):
    pedido.IsPrinterComandaFailed = true      ← badge rojo persiste
else:
    pedido.IsPrinterComandaFailed = false      ← sin badge (DONE, PENDING, etc.)

```

Casos:

- Pedido con job DONE → `false` (impresión exitosa, sin badge)
- Pedido con job FAILED → `true` (badge rojo visible)
- Pedido con job EXPIRED → `true` (badge rojo visible)
- Pedido con job PENDING → `false` (reintento en progreso, sin badge)
- Pedido sin job → `false` (flujo local sin PrinterServices, sin badge)

23.28.6 Flujo: Retry exitoso → Badge desaparece

```

Usuario clicka "Reintentar" en modal de impresión fallida
|
▼
PrinterServiceClient.RetryAllFailedJobsAsync()
├─ PrintJobStatusManager.UpdateStatus(jobId, "PENDING")      ← ya no es FAILED
└─ PedidoprinterjobController.actualizarEstadoJob(jobId, "PENDING")
|
▼ (PrinterServices reimprime exitosamente)
|
PrinterServiceModule recibe callback: status="DONE"
├─ PrintJobStatusManager.UpdateStatus(jobId, "DONE")
├─ Dispara JobStatusChanged, AllJobsDone
└─ PedidoprinterjobController.actualizarEstadoJob(jobId, "DONE")
|
▼ (próxima recarga de la lista)
|
MarcarPedidosConFalloDeImpresion():
    registro.Status = "DONE" → pedido.IsPrinterComandaFailed = false
    → Badge desaparece ✅

```

NOTA: El badge desaparece en la **próxima recarga** de la lista (cambio de mesa y vuelta). Para desaparición **en tiempo real** al recibir callback `DONE`, se podría suscribir también a `JobStatusChanged` y limpiar el flag cuando el status sea `DONE/PENDING`. Esto es opcional y se puede agregar como mejora posterior.

23.28.7 Cambios necesarios

#	Proyecto	Archivo	Cambio	Líneas aprox
1	Front	MainApp/venta/Pedidos/ListaPedidos.cs	Suscribirse a <code>PrintJobStatusManager.Instance.AnyJobFailed</code> en constructor	+1
2	Front	MainApp/venta/Pedidos/ListaPedidos.cs	Nuevo método <code>OnComandaFailed(jobId, error)</code> : obtiene <code>Pedidoids</code> del job, busca en <code>pedidoList</code> , setea <code>IsPrinterComandaFailed = true</code> via <code>Dispatcher</code>	+15
3	Front	MainApp/venta/Pedidos/ListaPedidos.cs	Reemplazar <code>loop p.IsPrinterComandaFailed = false</code> por llamada a <code>MarcarPedidosConFalloDeImpresion(pedidoList)</code>	~5
4	Front	MainApp/venta/Pedidos/ListaPedidos.cs	Nuevo método <code>MarcarPedidosConFalloDeImpresion(pedidoList)</code> : por cada pedido, consulta <code>pedidoprinterjob</code> y setea <code>true</code> si <code>FAILED/EXPIRED</code> , <code>false</code> si no	+15
5	QuipuNetX	controller/PedidoprinterjobController.cs	Nuevo método <code>tieneJobFallido(pedidoId) → bool</code> : consulta si el pedido tiene job en <code>FAILED/EXPIRED</code>	+10

Total: ~46 líneas nuevas/modificadas, 2 archivos.

23.28.8 Consideraciones de rendimiento

- **OnComandaFailed** : $O(N)$ donde N = pedidos visibles en la lista (típicamente < 50). Despreciable.
- **MarcarPedidosConFalloDeImpresion** : Hace 1 query SQLite por pedido. Para 50 pedidos = 50 queries simples de índice primario (~1ms c/u = 50ms total). Aceptable ya que ocurre al recargar la lista.
- **Optimización futura:** Si N crece, se puede hacer un solo `SELECT` con `IN` clause para todos los `pedidoids`.

23.28.9 Reglas de negocio del badge

Estado del job	Badge visible	Color	Razón
----------------	---------------	-------	-------

FAILED	✅ Sí	Rojo pulsante	Impresión falló, necesita atención
EXPIRED	✅ Sí	Rojo pulsante	Impresión expiró, necesita atención
DONE	❌ No	—	Impresión exitosa
PENDING	❌ No	—	Reintento en progreso
WAITING	❌ No	—	Esperando impresora (aún no es error)
SENT	❌ No	—	Recién enviado
Sin job	❌ No	—	Flujo local (sin PrinterServices)

23.26 Checklist pre-implementación por fase

Antes de implementar cada fase, verificar:

- ☐ ¿Las clases nuevas están en archivos separados? (no agregar a archivos existentes)
- ☐ ¿Cada clase tiene UNA responsabilidad?
- ☐ ¿Ningún método supera ~30 líneas?
- ☐ ¿Los feature flags se verifican SOLO en puntos de entrada (presenters/views)?
- ☐ ¿Las clases del flujo local siguen funcionando idéntico con flag OFF?
- ☐ ¿Hay comentarios `//` en cada línea de código nuevo?
- ☐ ¿Los archivos nuevos están agregados al `.csproj` ?
- ☐ ¿No se importaron namespaces del flujo local en clases del flujo PrinterServices?

24. Evolución de tabla `pedidoprinterjob` : Soporte multi-tipo de impresión

24.1 Problema

La tabla `pedidoprinterjob` actual solo soporta **comandas** (relación `pedido_id ↔ job_id`). Pero con la Fase 7B (comprobantes) y 7C (secundarios), PrinterServices manejará **6 tipos de impresión** diferentes, cada uno asociado a entidades distintas:

TIPO ACTUAL (solo comandas):

ID(PK)	PEDIDOID	JOBID	STATUS	FECHACREACION
--------	----------	-------	--------	---------------

↑ solo `pedido_id` – no puede distinguir tipos ni asociar a ventas

Limitaciones:

- No se puede saber si un job es una comanda, un comprobante o una encuesta
- Comprobantes, encuestas, promociones y precuentas están asociados a una **venta** (`venta_id`), no a un pedido
- Delivery tiene su propia entidad y flujo
- Sin campo TIPO, no hay forma de filtrar/consultar por tipo de impresión

24.2 Solución: 2 columnas nuevas — TIPO + VENTAID

Columna **TIPO** — Clasificar el tipo de impresión

Valor	Qué imprime	Entidad origen	Strategy backend
COMANDA	Comanda de cocina/bar	pedido_id(s)	ServerComandasStrategy
COMPROBANTE	Boleta/Factura	venta_id	ServerVentaRapidaStrategy / ServerVentaSalonStrategy
PROMOCION	Ticket de sorteo/promoción	venta_id	(dentro de VentaRapida/Delivery)
ENCUESTA	Ticket de encuesta	venta_id	(dentro de VentaRapida/Delivery)
PRECUENTA	Pre-cuenta	venta_id o pedido_ids	(dentro de VentaRapida/Delivery)
DELIVERY	Ticket de delivery	venta_id	ServerDeliveryStrategy

Columna **VENTAID** — Asociar a la venta cuando aplica

Las comandas ya tienen `PEDIDOID`. Pero comprobantes, encuestas, promociones, precuentas y delivery están asociados a una **venta**, no a un pedido individual.

24.3 Esquema nuevo

ID(PK)	PEDIDOID	VENTAID	TIPO	JOBID	STATUS	FECHACREACION
--------	----------	---------	------	-------	--------	---------------

142	"713177"	NULL	"COMANDA"	"49ed.."	"DONE"	"2026-02-27 13:03"
143	"713178"	NULL	"COMANDA"	"49ed.."	"DONE"	"2026-02-27 13:03"
150	NULL	"5001"	"COMPROBANTE"	"a1b2.."	"DONE"	"2026-02-27 13:04"
151	NULL	"5001"	"PROMOCION"	"c3d4.."	"DONE"	"2026-02-27 13:04"
152	NULL	"5001"	"ENCUESTA"	"e5f6.."	"SENT"	"2026-02-27 13:04"
153	NULL	"5001"	"PRECUENTA"	"g7h8.."	"FAILED"	"2026-02-27 13:04"
154	NULL	"5001"	"DELIVERY"	"i9j0.."	"DONE"	"2026-02-27 13:04"

24.4 Reglas de uso por tipo

Tipo	PEDIDOID	VENTAID	Cuándo se usa
COMANDA	✅ pedido_id	❌ NULL	addLista, reImprimirPedido
COMPROBANTE	❌ NULL	✅ venta_id	VentaController al cobrar
PROMOCION	❌ NULL	✅ venta_id	Sorteo dentro del cobro
ENCUESTA	❌ NULL	✅ venta_id	Encuesta dentro del cobro
PRECUENTA	❌ NULL	✅ venta_id	Precuenta asociada a venta
PRECUENTA	✅ pedido_id	❌ NULL	Precuenta pre-cobro (sin venta aún)
DELIVERY	❌ NULL	✅ venta_id	Ticket delivery dentro del cobro

Regla general: Si hay `venta_id` → usar `VENTAID` . Si no hay venta (comandas, precuenta pre-cobro) → usar `PEDIDOID` .

24.5 Cambios en `PedidoprinterjobBase.cs`

Nuevos campos

```
// Tipo de impresión: COMANDA, COMPROBANTE, PROMOCION, ENCUESTA, PRECUENTA, DELIVERY
protected internal string tipo;

// ID de la venta asociada (null para comandas, obligatorio para comprobantes/promoción/encuesta/delivery)
protected internal string ventaaid;

// Columna TIPO - clasifica el tipo de documento impreso
[Column("TIPO")]
public virtual string Tipo { get => tipo; set => tipo = value; }

// Columna VENTAID - referencia lógica a tabla venta (null para comandas)
[Column("VENTAID")]
public virtual string Ventaaid { get => ventaaid; set => ventaaid = value; }
```

Actualizar `fromJSON()`

```
case "tipo": Tipo = v; break;
case "ventaaid": Ventaaid = v; break;
```

Actualizar `toJSON()`

```
obj["tipo"] = Tipo;
obj["ventaaid"] = Ventaaid;
```

Nuevos métodos de consulta

```
// Buscar todos los registros asociados a una venta
public static IList<Pedidoprinterjob> findByVentaaid(string ventaaid)
{
    return Find<Pedidoprinterjob>(typeof(Pedidoprinterjob), "VENTAID=?",
        new string[] { ventaaid }, null, null, null);
}

// Buscar por tipo de impresión
public static IList<Pedidoprinterjob> findByTipo(string tipo)
{
    return Find<Pedidoprinterjob>(typeof(Pedidoprinterjob), "TIPO=?",
```

```

        new string[] { tipo }, null, null, null);
    }

    // Buscar por venta + tipo (ej: todos los jobs COMPROBANTE de la venta 5001)
    public static IList<Pedidoprinterjob> findByVentaIdAndTipo(string ventaId, string tipo)
    {
        return FindWithQuery<Pedidoprinterjob>(
            "SELECT * FROM pedidoprinterjob WHERE VENTAID=? AND TIPO=?", ventaId, tipo);
    }

    // Verificar si una venta tiene algún job en estado FAILED o EXPIRED
    public static bool tieneJobFallidoPorVenta(string ventaId)
    {
        var list = FindWithQuery<Pedidoprinterjob>(
            "SELECT * FROM pedidoprinterjob WHERE VENTAID=? AND (STATUS='FAILED' OR STATUS='EXPIRED') LIMIT 1",
            ventaId);
        return list != null && list.Count > 0;
    }
}

```

24.6 Cambios en PedidoprinterjobController.cs

Nuevo método genérico para guardar jobs de cualquier tipo

```

/// <summary>
/// Guarda jobs en pedidoprinterjob para cualquier tipo de impresión (no solo comandas).
/// Para COMANDA: usa pedidoIds del PrintJobResult, ventaId = null.
/// Para COMPROBANTE/PROMOCION/ENCUESTA/PRECUESTA/DELIVERY: usa ventaId, pedidoId = null.
/// </summary>
public static void guardarJobsPorTipo(List<PrintJobResult> jobResults, string tipo, string ventaId = null)
{
    // Fecha/hora actual para todos los registros del batch
    string fechaActual = DateTime.Now.ToString("yyyy-MM-dd HH:mm:ss");

    foreach (var jr in jobResults)
    {
        // Si el tipo es COMANDA, crear un registro por cada pedido_id asociado al job
        if (tipo == "COMANDA" && jr.PedidoIds != null && jr.PedidoIds.Count > 0)
        {
            foreach (var pid in jr.PedidoIds)
            {
                var reg = new Pedidoprinterjob(); // Nuevo registro en tabla
                reg.Pedidoid = pid; // pedido_id del pedido original
                reg.VentaId = null; // Comandas no tienen venta asociada
                reg.JobId = jr.JobId; // job_id de PrinterServices
                reg.Tipo = tipo; // "COMANDA"
                reg.Status = "SENT"; // Estado inicial: enviado OK
                reg.Fechacreacion = fechaActual; // Timestamp de creación
                reg.save(); // INSERT en SQLite
            }
        }
        else
        {
            // Para otros tipos (COMPROBANTE, PROMOCION, etc.): 1 registro con ventaId
            var reg = new Pedidoprinterjob(); // Nuevo registro en tabla
            reg.Pedidoid = null; // No aplica para tipos de venta
            reg.VentaId = ventaId; // venta_id asociada
            reg.JobId = jr.JobId; // job_id de PrinterServices
            reg.Tipo = tipo; // Tipo de impresión
            reg.Status = "SENT"; // Estado inicial: enviado OK
            reg.Fechacreacion = fechaActual; // Timestamp de creación
            reg.save(); // INSERT en SQLite
        }
    }
}

```

24.7 Compatibilidad hacia atrás

- **Registros existentes** (sin TIPO ni VENTAID): Se asumen como "COMANDA" con VENTAID = NULL
- **Método actual** guardarJobsPorComandas : Sigue funcionando — internamente debe setear Tipo = "COMANDA" en los registros nuevos
- **SQLite:** ALTER TABLE agrega columnas con DEFAULT NULL , los registros existentes no se pierden

- **EnsureTable()** : Al llamar `CreateTable<Pedidoprinterjob>()` , SQLite-net agrega columnas faltantes automáticamente (no recrea la tabla)

24.8 Tabla de decisión: Qué tipo usar por Strategy

Strategy backend	Tipo(s) que genera	PEDIDOID	VENTAID	Método a usar
ServerComandasStrategy	COMANDA	✔ pedido_ids	✘ NULL	guardarJobsPorTipo(jobs, "COMANDA")
ServerVentaRapidaStrategy	COMPROBANTE, PROMOCION, ENCUESTA, PRECUESTA	✘ NULL	✔ venta_id	guardarJobsPorTipo(jobs, "COMPROBANTE", ventaId)
ServerVentaSalonStrategy	COMPROBANTE, PROMOCION, ENCUESTA	✘ NULL	✔ venta_id	guardarJobsPorTipo(jobs, "COMPROBANTE", ventaId)
ServerDeliveryStrategy	COMPROBANTE, PROMOCION, ENCUESTA, DELIVERY	✘ NULL	✔ venta_id	guardarJobsPorTipo(jobs, "DELIVERY", ventaId)
ServerVentaProveedorStrategy	COMPROBANTE	✘ NULL	✔ venta_id	guardarJobsPorTipo(jobs, "COMPROBANTE", ventaId)
ServerAnularCorregirStrategy	COMPROBANTE	✘ NULL	✔ venta_id	guardarJobsPorTipo(jobs, "COMPROBANTE", ventaId)

24.9 Impacto en consultas existentes

Consulta	Cambio necesario
findByPedidoid(pid)	✔ Sin cambios — sigue filtrando por PEDIDOID (solo devuelve comandas)
findByJobid(jobId)	✔ Sin cambios — devuelve todos los registros del job (cualquier tipo)
tieneJobFallido(pedidoId)	✔ Sin cambios — busca por PEDIDOID (para badges de comandas)
NUEVO findByVentaId(ventaId)	Buscar todos los jobs de una venta (comprobante + promoción + encuesta + etc.)
NUEVO findByVentaIdAndTipo(ventaId, tipo)	Buscar jobs específicos de una venta por tipo
NUEVO tieneJobFallidoPorVenta(ventaId)	Badge de impresión fallida para ventas

24.10 Impacto en PrintJobStatusManager (RAM)

PrintJobStatusInfo necesita un campo nuevo para distinguir el tipo:

```

public class PrintJobStatusInfo
{
    public string JobId { get; set; }
    public string Status { get; set; }
    public List<string> PedidoIds { get; set; }
    public string VentaId { get; set; }           // ← NUEVO: venta asociada (null para comandas)
    public string Tipo { get; set; }             // ← NUEVO: COMANDA, COMPROBANTE, PROMOCION, etc.
    public string Error { get; set; }
    public string ImpresoraNombre { get; set; }
    public string AreaImpresion { get; set; }
    public DateTime FechaRegistro { get; set; }
    public DateTime? FechaUltimaActualizacion { get; set; }
}

```

24.11 Ejemplo completo — Cobro de venta rápida con PS

Escenario: Cajero cobra venta #5001 (1 lomo, 2 cervezas).
VentaController genera 4 tipos de impresión.

- VentaController → ServerVentaRapidaStrategy:
 - Comprobante → EnviarVentaAsync() → job_id "a1b2" → guardarJobsPorTipo([a1b2], "COMPROBANTE", "5001")
 - Promoción → EnviarPromocionAsync() → job_id "c3d4" → guardarJobsPorTipo([c3d4], "PROMOCION", "5001")
 - Encuesta → EnviarEncuestaAsync() → job_id "e5f6" → guardarJobsPorTipo([e5f6], "ENCUESTA", "5001")
 - Delivery → EnviarPrecuentaAsync() → job_id "g7h8" → guardarJobsPorTipo([g7h8], "DELIVERY", "5001")

- Tabla pedidoprinterjob después del cobro:

ID	PEDIDOID	VENTAID	TIPO	JOBID	STATUS

150	NULL	"5001"	"COMPROBANTE"	"a1b2.."	"SENT"
151	NULL	"5001"	"PROMOCION"	"c3d4.."	"SENT"
152	NULL	"5001"	"ENCUESTA"	"e5f6.."	"SENT"
153	NULL	"5001"	"DELIVERY"	"g7h8.."	"SENT"

3. Front: ImpresionList vacía + flag ON
→ verificarPrintJobsPorVenta(respuesta)
→ PrintJobResults.Count > 0 → showImprimiendoPrinterService(onSuccess, onFail)
→ PSProgressService monitorea 4 jobs en tiempo real

24.12 Ejemplo completo — Venta de delivery con PS

Escenario: Delivery de 2 pizzas + 1 gaseosa.
Pedido_ids internos: [801, 802, 803]. Venta generada al cobrar: #6001.

===== FASE 1: Al enviar pedido (addLista) =====

PedidoController.addLista() → ServerComandasStrategy:
└ Comandas cocina → EnviarComandasAsync() → job_id "x1y2"
→ guardarJobsPorTipo([{"JobId":"x1y2", PedidoIds:["801","802","803"]}], "COMANDA")

Tabla pedidoprinterjob después de addLista:

ID	PEDIDOID	VENTAID	TIPO	JOBID	STATUS	
200	"801"	NULL	"COMANDA"	"x1y2.."	"SENT"	← pizza 1
201	"802"	NULL	"COMANDA"	"x1y2.."	"SENT"	← pizza 2
202	"803"	NULL	"COMANDA"	"x1y2.."	"SENT"	← gaseosa

Front: showImprimiendoPrinterService → PSProgressService monitorea 1 job (x1y2)
→ DONE → showSuccessRegister 

===== FASE 2: Al cobrar venta (VentaController) =====

VentaController → ServerDeliveryStrategy:
└ Ticket repartidor → EnviarDeliveryAsync() → job_id "d3e4"
| → guardarJobsPorTipo([d3e4], "DELIVERY", "6001")
└ Boleta/Factura → EnviarVentaAsync() → job_id "f5g6"
| → guardarJobsPorTipo([f5g6], "COMPROBANTE", "6001")
└ Sorteo → EnviarPromocionAsync() → job_id "h7i8"
| → guardarJobsPorTipo([h7i8], "PROMOCION", "6001")
└ Encuesta → EnviarEncuestaAsync() → job_id "j9k0"
→ guardarJobsPorTipo([j9k0], "ENCUESTA", "6001")

Tabla pedidoprinterjob después del cobro (registros nuevos se suman):

ID	PEDIDOID	VENTAID	TIPO	JOBID	STATUS	
200	"801"	NULL	"COMANDA"	"x1y2.."	"DONE"	← ya impreso
201	"802"	NULL	"COMANDA"	"x1y2.."	"DONE"	← ya impreso
202	"803"	NULL	"COMANDA"	"x1y2.."	"DONE"	← ya impreso
203	NULL	"6001"	"DELIVERY"	"d3e4.."	"SENT"	← ticket repartidor
204	NULL	"6001"	"COMPROBANTE"	"f5g6.."	"SENT"	← boleta
205	NULL	"6001"	"PROMOCION"	"h7i8.."	"SENT"	← sorteo
206	NULL	"6001"	"ENCUESTA"	"j9k0.."	"SENT"	← encuesta

Front: showImprimiendoPrinterService → PSProgressService monitorea 4 jobs
→ Todos DONE → showSuccessRegister 
→ Si "d3e4" FAILED → showFailedPrinterService (ticket repartidor falló)

===== CONSULTAS ÚTILES PARA DELIVERY =====

```
// ¿El ticket del repartidor se imprimió OK?
var ticketDelivery = Pedidoprinterjob.findByVentaIdAndTipo("6001", "DELIVERY");
// → [{ID:203, VENTAID:"6001", TIPO:"DELIVERY", JOBID:"d3e4", STATUS:"DONE"}]
```

```
// ¿Alguna impresión de la venta 6001 falló? (cualquier tipo)
bool fallido = Pedidoprinterjob.tieneJobFallidoPorVenta("6001");
```

```
// → false (todos DONE)

// Todos los jobs de la venta 6001 (delivery + comprobante + promoción + encuesta)
var todosJobsVenta = Pedidoprinterjob.findByVentaid("6001");
// → 4 registros (IDs 203-206)

// Las comandas del pedido 801 (fase 1, separadas de la venta)
var comandasPedido = Pedidoprinterjob.findByPedidoid("801");
// → [{ID:200, PEDIDOID:"801", TIPO:"COMANDA", JOBID:"x1y2", STATUS:"DONE"}]
```

Clave: El delivery genera registros en **2 momentos distintos**:

1. **addLista** → comandas (COMANDA con PEDIDOID , sin VENTAID)
2. **Cobro** → ticket repartidor + comprobante + extras (DELIVERY / COMPROBANTE /etc. con VENTAID , sin PEDIDOID)

24.13 Orden de implementación

Paso	Archivo	Acción
1	entitybase/PedidoprinterjobBase.cs	Agregar campos tipo + ventaid + columnas + fromJSON/toJSON + queries nuevos
2	entity/Pedidoprinterjob.cs	Sin cambios (hereda de Base)
3	controller/PedidoprinterjobController.cs	Agregar guardarJobsPorTipo()
4	Services/Print/PrintJobStatusInfo.cs	Agregar VentaId + Tipo
5	controller/PedidoController.cs	En guardarJobsPorComandas existente: setear Tipo = "COMANDA"
6	Futuro: VentaController.cs	Usar guardarJobsPorTipo(jobs, tipo, ventaId) en Fase 7B

24.14 Estado de implementación (Marzo 2026)

Paso	Archivo	Estado	Detalle
1	entitybase/PedidoprinterjobBase.cs	✅ HECHO	Campos ventaid + tipo, columnas [Column("VENTAID")] + [Column("TIPO")], fromJSON/toJSON actualizados, 4 queries nuevos: findByVentaid, findByTipo, findByVentaidAndTipo, tieneJobFallidoPorVenta
2	entity/Pedidoprinterjob.cs	✅ SIN CAMBIOS	Hereda de Base — las columnas nuevas se heredan automáticamente
3	controller/PedidoprinterjobController.cs	✅ HECHO	Nuevo método guardarJobsPorTipo(jobResults, tipo, ventaId) + guardarJobsPorComandas ahora setea Tipo = "COMANDA"
4	Services/Print/PrintJobStatusInfo.cs	✅ HECHO	Nuevas propiedades VentaId + Tipo para tracking en RAM
5	controller/PedidoController.cs	✅ YA IMPLÍCITO	guardarJobsPorComandas ya setea Tipo = "COMANDA" (paso 3)
6	VentaController.cs	🕒 PENDIENTE	Usar guardarJobsPorTipo(jobs, tipo, ventaId) cuando se implemente Fase 7B (comprobantes)

Build: Compilación exitosa (0 errores) — `dotnet build QuipuNetX.sln`

Compatibilidad hacia atrás: SQLite-net agrega columnas faltantes con `DEFAULT NULL` al llamar `EnsureTable()` . Los registros existentes (sin TIPO ni VENTAID) siguen funcionando — se asumen como COMANDA con VENTAID=NULL.

Próximo paso: Cuando se implemente una Strategy de venta (Fase 7B), usar:

```
// Ejemplo para comprobante de venta rápida
PedidoprinterjobController.guardarJobsPorTipo(jobResults, "COMPROBANTE", ventaId);

// Ejemplo para ticket de delivery
PedidoprinterjobController.guardarJobsPorTipo(jobResults, "DELIVERY", ventaId);
```

25. Fase 7B: Métodos de `PrinterServiceClient` para documentos individuales

25.1 Contexto

`PrinterServiceClient` solo tenía `EnviarComandasAsync()` (POST array a `/api/print/comandas`). Para la Fase 7B (comprobantes, promociones, encuestas, precuentas, delivery), se necesitan métodos que envíen **documentos individuales** (un solo JObject, no un array).

25.2 Métodos implementados (Marzo 2026)

Método	Endpoint PS	Equivale a (Front)	Tipo pedidoprinterjob
EnviarVentaAsync(impresion, ventaId)	POST /api/print/venta	imprimirVenta()	COMPROBANTE
EnviarPromocionAsync(impresion, ventaId)	POST /api/print/comanda	imprimirPromociones()	PROMOCION
EnviarEncuestaAsync(impresion, ventaId)	POST /api/print/comanda	imprimirEncuesta()	ENCUESTA
EnviarPrecuentaAsync(impresion, ventaId)	POST /api/print/precuenta	imprimirPrecuenta()	PRECUESTA / DELIVERY
IsAvailableAsync()	GET /api/health	Health check	—

25.3 Arquitectura: Helper `EnviarDocumentoSingleAsync`

Todos los métodos de documento individual delegan a un **helper privado** que encapsula el patrón común:

```
EnviarVentaAsync(impresion, ventaId)
EnviarPromocionAsync(impresion, ventaId)
EnviarEncuestaAsync(impresion, ventaId, caller)
EnviarPrecuentaAsync(impresion, ventaId)
```

└─

└─

└─

└─

└─

└─

`EnviarDocumentoSingleAsync(impresion, endpoint, ventaId, msg,`
`caller)`

- 1. Validar Impresion no null
- 2. EnriquecerImpresion() (misma función que comandas)
- 3. Inyectar ventaId como pedido_ids (para callbacks PS)
- 4. PostConReintentoAsync(url, body) – con retry
- 5. Si falla → RegisterPendingImpresiones (para retry)
- 6. Retornar Respuesta con PrintJobResult

25.4 Diferencia vs `EnviarComandasAsync`

Aspecto	<code>EnviarComandasAsync</code>	<code>EnviarDocumentoSingleAsync</code>
Body JSON	JArray (múltiples items)	JsonObject (un solo item)
Endpoint	/api/print/comandas	Varía: /api/print/venta, /api/print/comanda, /api/print/precuenta
IDs en JSON	pedido_ids (lista de pedidos)	pedido_ids = [ventaId] (para trazabilidad en callbacks)
PS Controller	PostComandas() (loop + dedup)	PostComanda() / PostVenta() / PostPrecuenta() (single)

25.5 Uso desde Strategies (futuro)

```
// ServerVentaRapidaStrategy – dentro del cobro
var respComprobante = await PrinterServiceClient.Instance.EnviarVentaAsync(impresionComprobante, ventaId);
if (respComprobante.Tipo == Util.SUCCESS && respComprobante.Data is List<PrintJobResult> jobsComp)
{
    PedidoprinterjobController.guardarJobsPorTipo(jobsComp, "COMPROBANTE", ventaId);
    PrintJobStatusManager.Instance.RegisterJobs(jobsComp);
}

var respSorteo = await PrinterServiceClient.Instance.EnviarPromocionAsync(impresionSorteo, ventaId);
if (respSorteo.Tipo == Util.SUCCESS && respSorteo.Data is List<PrintJobResult> jobsSorteo)
{
    PedidoprinterjobController.guardarJobsPorTipo(jobsSorteo, "PROMOCION", ventaId);
    PrintJobStatusManager.Instance.RegisterJobs(jobsSorteo);
}

var respEncuesta = await PrinterServiceClient.Instance.EnviarEncuestaAsync(impresionEncuesta, ventaId);
if (respEncuesta.Tipo == Util.SUCCESS && respEncuesta.Data is List<PrintJobResult> jobsEnc)
{
    PedidoprinterjobController.guardarJobsPorTipo(jobsEnc, "ENCUESTA", ventaId);
    PrintJobStatusManager.Instance.RegisterJobs(jobsEnc);
}
```

25.6 `IsAvailableAsync` — Health check

```
// Verificar si PrinterServices está activo antes de enviar
bool psActivo = await PrinterServiceClient.Instance.IsAvailableAsync();
if (!psActivo)
{
    // PS no disponible – fallback o mostrar error
}
```

```
mensajes.Add("PrinterServices no está activo");
}
```

- **Timeout:** 3 segundos (corto, para no bloquear el flujo)
- **Retorna:** `true` si PS responde HTTP 200 con `status: "OK"` , `false` si no responde o error
- **No lanza excepciones:** Captura todo y retorna `false`

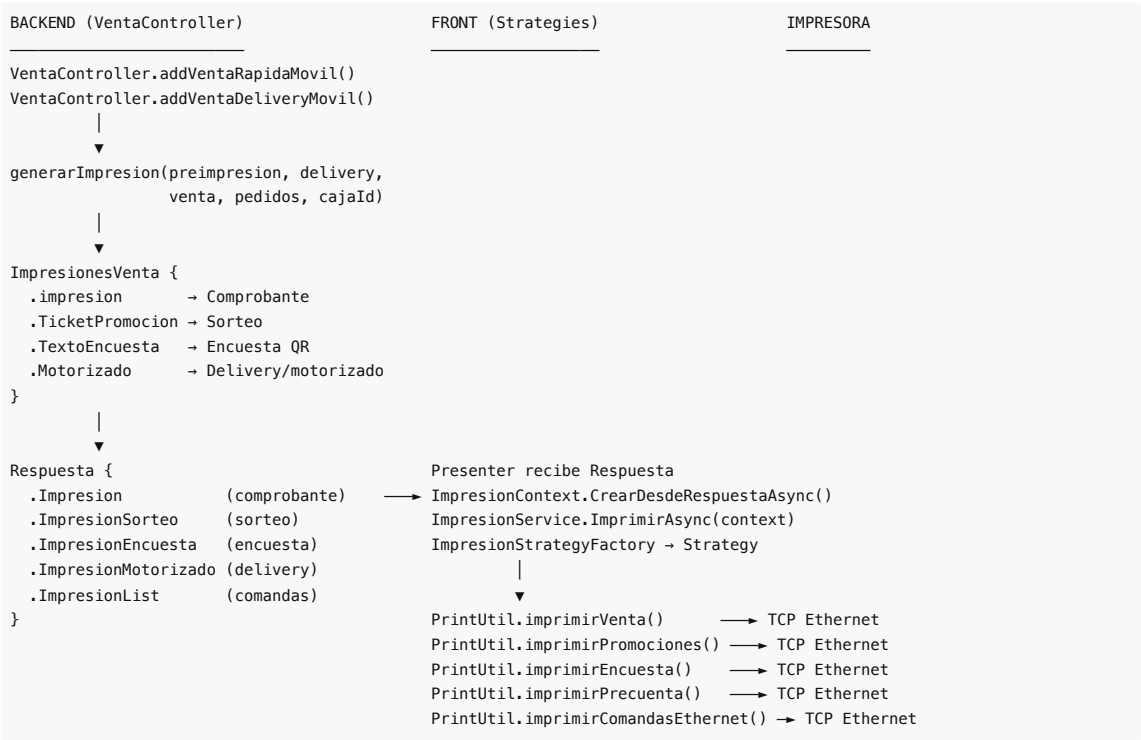
25.7 Estado de implementación

Componente	Estado
EnviarVentaAsync	✅ HECHO — PrinterServiceClient.cs
EnviarPromocionAsync	✅ HECHO — PrinterServiceClient.cs
EnviarEncuestaAsync	✅ HECHO — PrinterServiceClient.cs
EnviarPrecuentaAsync	✅ HECHO — PrinterServiceClient.cs
IsAvailableAsync	✅ HECHO — PrinterServiceClient.cs
EnviarDocumentoSingleAsync (helper)	✅ HECHO — PrinterServiceClient.cs
Endpoints en PrinterServices	✅ YA EXISTÍAN — PostVenta, PostPrecuenta, PostComanda en PrintController.cs
Rutas en ApiRouter.cs	✅ YA EXISTÍAN — /api/print/venta, /api/print/precuenta, /api/print/comanda, /api/health
Build	✅ 0 errores

Próximo paso: Crear las Strategies backend (`ServerVentaRapidaStrategy` , etc.) que usen estos métodos.

26. Mapa del flujo actual de impresión y corrección de gaps (Fase 7B)

26.1 Flujo actual (sin PrinterServices)



26.2 Clase contenedora: ImpresionesVenta

Archivo: `QuipuNetX/entity/extras/ImpresionesVenta.cs`

Propiedad	Tipo documento	Generado por
<code>.impresion</code>	Comprobante (boleto/factura)	<code>ComprobanteController.getImpresionVenta()</code> O <code>ventaRapida.getImpresion()</code>

.TicketPromocion	Sorteo/promoción	Promocionsorteo.getFormatoParaImpresionPromocionSorteoVigente()
.TextoEncuesta	Encuesta QR	ComprobanteController.getImpresionTextoQrEncuesta()
.Motorizado	Ticket delivery/motorizado	CadenaImpresionDespacho.buildCadenaDespacho() o ComprobanteController.getImpresionVenta()

26.3 Strategies del Front (ya existen)

Strategy	Qué imprime	PrintUtil
VentaRapidaPrintStrategy	Comprobante + comandas + sorteo + encuesta + motorizado	imprimirVenta + imprimirComandasEthernet + imprimirPromociones + imprimirEncuesta + imprimirPrecuenta
VentaSalonPrintStrategy	Comprobante + sorteo + encuesta (NO comandas, NO motorizado)	imprimirVenta + imprimirPromociones + imprimirEncuesta
DeliveryPrintStrategy	Comprobante + sorteo + encuesta + motorizado (NO comandas)	imprimirVenta + imprimirPromociones + imprimirEncuesta + imprimirPrecuenta

26.4 Gaps encontrados y corregidos

Problema: EnriquecerImpresion fue diseñado solo para comandas. Cada PrintUtil.imprimirXxx() tiene lógica ESC/POS específica que PS no replicaba.

PrintUtil (Front)	Lógica ESC/POS especial	Campo faltante en EnriquecerImpresion	Campo faltante en PS
imprimirVenta	Split cadena en ##FE##, genera QR ESC/POS inline	qrData, facturacionElectronica, impresora_tamanoqr	FacturacionElectronica, TamanoQr en PrintJob
imprimirEncuesta	BigWeightLetter (DoubleWidthHeight) + QR centrado	qrEncuesta	QrEncuesta en PrintJob, BuildPayload sin BigWeightLetter
imprimirPromociones	N copias via Promocionsorteo_cantidadimpresiones	promocionsorteo_cantidadimpresiones	ParsePrintJob no mapeaba a Copias
imprimirPrecuenta	Texto simple + feed + cut	Ninguno (ya compatible)	Ninguno (ya compatible)

26.5 Correcciones implementadas (Marzo 2026)

QuipuNetX — EnriquecerImpresion (PrinterServiceClient.cs)

Campos agregados al JSON que se envía a PS:

```
json["facturacionElectronica"] = impresion.FacturacionElectronica;
json["qrData"] = impresion.QrData ?? "";
json["impresora_tamanoqr"] = impresion.Impresora_tamanoqr ?? ...;
json["qrEncuesta"] = impresion.QrEncuesta ?? "";
json["promocionsorteo_cantidadimpresiones"] = impresion.Promocionsorteo_cantidadimpresiones ?? "1";
```

PrinterServices — ParsePrintJob (PrintController.cs)

Nuevos campos parseados:

- facturacionElectronica → job.FacturacionElectronica (bool)
- impresora_tamanoqr → job.TamanoQr (string)
- qrEncuesta → job.QrEncuesta (string)
- promocionsorteo_cantidadimpresiones → job.Copias (si > actual)

PrinterServices — PrintJob.cs + PrintJobEntity.cs

3 campos nuevos:

- FacturacionElectronica (bool/int en BD facturacion_electronica)
- TamanoQr (string/ tamano_qr)
- QrEncuesta (string/ qr_encuesta)

PrinterServices — BuildPayload (PrintWorker.cs)

3 ramas de renderizado:

- Encuesta** (job.QrEncuesta != null): DoubleWidthHeight + Text centrado + QR centrado + Reset
- Venta FE** (job.FacturacionElectronica && ##FE## en cadena): Split en marker, texto antes, QR centrado, texto después
- Simple** (comandas, precuentas): Texto plano (sin cambios)

26.6 Build

Proyecto	Errores
PrinterServices.sln	✔ 0 errores
QuipuNetX.sln	✔ 0 errores

Próximo paso: Crear las Strategies backend que usen los métodos corregidos de `PrinterServiceClient`.

27. Server Strategies para impresión vía PrinterServices (Fase 7B)

27.1 Infraestructura pre-existente (Fase 7A)

Componente	Archivo	Descripción
IPrintStrategy	Services/Print/IPrintStrategy.cs	Interfaz: NombreEstrategia + EjecutarAsync(context)
ImpresionContext	Services/Print/ImpresionContext.cs	Contexto con ImpresionPrincipal, Sorteo, Encuesta, Motorizado, Comandas, flags
ImpresionStrategyFactory	Services/Print/ImpresionStrategyFactory.cs	Switch por TipoImpresionOrigen
ServerImpresionService	Services/Print/ServerImpresionService.cs	Singleton orquestador: ImprimirAsync(context) → Factory → Strategy
ServerComandasStrategy	Services/Print/Strategies/ServerComandasStrategy.cs	Solo comandas (Fase 7A)
PrintResultDto	Services/Print/PrintResultDto.cs	Resultado: Success, ErrorMessage, ImpresionErrorList

27.2 Strategies creadas (Fase 7B)

Strategy	Archivo	Qué imprime	Endpoint PS
ServerVentaRapidaStrategy	Strategies/ServerVentaRapidaStrategy.cs	Comprobante + Comandas + Sorteo + Encuesta + Motorizado (5 tipos)	/api/print/venta + /api/print/comandas + /api/print/venta x3
ServerVentaSalonStrategy	Strategies/ServerVentaSalonStrategy.cs	Comprobante + Sorteo + Encuesta (3 tipos)	/api/print/venta x3
ServerDeliveryStrategy	Strategies/ServerDeliveryStrategy.cs	Comprobante + Sorteo + Encuesta + Motorizado (4 tipos)	/api/print/venta x3 + /api/print/precuenta

27.3 Patrón de implementación

Todas las strategies siguen el mismo patrón **best-effort**:

1. Intentar cada tipo de impresión secuencialmente
2. Si uno falla, acumular error pero NO detener el flujo
3. Al final, si hay errores → `PrintResultDto.CreateError` con todos concatenados
4. Si todo OK → `PrintResultDto.CreateSuccess`

27.4 Factory methods en ImpresionContext

```
// Fase 7A (ya existía)
ImpresionContext.CrearParaComandas(impresionComandas)

// Fase 7B (nuevos)
ImpresionContext.CrearParaVentaRapida(ventaId, impresionesVenta, comandas, imprimirComandas)
ImpresionContext.CrearParaVentaSalon(ventaId, impresionesVenta)
ImpresionContext.CrearParaVentaDelivery(ventaId, impresionesVenta)
```

27.5 Factory habilitado

```
switch (context.TipoImpresion)
{
    case Comandas:      → ServerComandasStrategy      // Fase 7A
    case VentaRapida:   → ServerVentaRapidaStrategy    // Fase 7B ✔
    case VentaSalon:    → ServerVentaSalonStrategy     // Fase 7B ✔
    case VentaDelivery: → ServerDeliveryStrategy       // Fase 7B ✔
}
```

```
// Fase 7C (pendiente): AnularCorregir, VentaProveedoror
}
```

27.6 Uso desde VentaController (ejemplo)

```
// Después de generarImpresion():
if (FeatureFlagManager.Instance.IsEnabled("USAR_PRINTER_SERVICE"))
{
    var ctx = ImpresionContext.CrearParaVentaRapida(venta_id, impresiones, impresionList, imprimirComandas);
    var resultado = await ServerImpresionService.Instance.ImprimirAsync(ctx);
    // resultado.Success → OK, resultado.ErrorMessage → errores parciales
}
```

27.7 Build

Proyecto	Errores CS
QuipuNetX.sln	✔ 0 errores CS
PrinterServices.sln	✔ 0 errores

Próximo paso completado — ver sección 28.

28. Integración de Strategies en VentaController (Fase 7B — continuación)

28.1 Objetivo

Integrar `ServerVentaRapidaStrategy` y `ServerDeliveryStrategy` en los métodos `addVentaRapidaMovil` y `addVentaDeliveryMovil` del `VentaController.cs`, utilizando el feature flag `USAR_PRINTER_SERVICE` para bifurcar entre impresión local (Front) e impresión delegada (`PrinterServices`).

28.2 Patrón de integración

Se replicó el patrón ya implementado en `PedidoController.addLista` (líneas 614-718), adaptándolo para ventas:

- Feature flag check:** `Util.esModoServidor() && SQLite.FeatureFlagConfigReader.IsEnabled("USAR_PRINTER_SERVICE")`
- Crear ImpresionContext** via factory method (`CrearParaVentaRapida` / `CrearParaVentaDelivery`)
- Clear PrintJobStatusManager** para evitar contaminación de sesiones previas
- Ejecutar strategy** via `ServerImpresionService.Instance.ImprimirAsync(ctx)` dentro de `Task.Run().GetAwaiter().GetResult()`
- Evaluar resultado** (`PrintResultDto.Success`):
 - Si OK → vaciar todas las impresiones locales (comprobante, sorteo, encuesta, motorizado, comandas)
 - Si falla → mantener impresiones para fallback local + agregar mensaje de aviso
- Injectar PrintJobResults** en `respuesta_.PrintJobResults` para el frontend

28.3 Cambios en VentaController.cs

28.3.1 Import agregado

```
using QipuNetX.Services.Print;
```

28.3.2 addVentaRapidaMovil — Variables

```
// Declaración al inicio del método (después de declaración de objVenta)
List<PrintJobResult> printJobResultsParaRespuesta = null;
```

28.3.3 addVentaRapidaMovil — Bloque PS

Insertado después de `impresionList = objVenta.ImpresionListComandas;` y antes del emit a cocina:

```
if (Util.esModoServidor()
    && SQLite.FeatureFlagConfigReader.IsEnabled("USAR_PRINTER_SERVICE"))
{
    var impresionesVenta = new ImpresionesVenta();
    impresionesVenta.impresion = impresion;
    impresionesVenta.TicketPromocion = impresionTicketPromocion;
    impresionesVenta.TextoEncuesta = impresionTextoEncuesta;
    impresionesVenta.Motorizado = impresionMotorizado;
```

```

var ctxPS = ImpresionContext.CrearParaVentaRapida(
    venta_id, impresionesVenta, impresionList, imprimirComandas
);
PrintJobStatusManager.Instance.Clear();

PrintResultDto resultadoPS = null;
try {
    resultadoPS = Task.Run(async () =>
        await ServerImpresionService.Instance.ImprimirAsync(ctxPS)
    ).GetAwaiter().GetResult();
} catch (Exception exPS) {
    Util.Capture(exPS, "VentaController", "addVentaRapidaMovil.PrinterService", ...);
}

if (resultadoPS != null && resultadoPS.Success) {
    impresion = null;
    impresionTicketPromocion = null;
    impresionTextoEncuesta = null;
    impresionMotorizado = null;
    impresionList = new List<Impresion>();
} else {
    mensajes.Add("Aviso PS: " + errorMsg);
}
}

```

28.3.4 addVentaRapidaMovil — Respuesta

```

respuesta_.PrintJobResults = printJobResultsParaRespuesta;

```

28.3.5 addVentaDeliveryMovil — Variables

```

// Declaración al inicio del método (después de preimpresionMotorizado)
List<PrintJobResult> printJobResultsParaRespuesta = null;

```

28.3.6 addVentaDeliveryMovil — Bloque PS

Insertado después de `impresionMotorizado = impresiones.Motorizado;` y antes de `} // ./ if :`

```

if (Util.esModoServidor()
    && SQLite.FeatureFlagConfigReader.IsEnabled("USAR_PRINTER_SERVICE"))
{
    var ctxPS = ImpresionContext.CrearParaVentaDelivery(
        venta_id, impresiones
    );
    PrintJobStatusManager.Instance.Clear();

    PrintResultDto resultadoPS = null;
    try {
        resultadoPS = Task.Run(async () =>
            await ServerImpresionService.Instance.ImprimirAsync(ctxPS)
        ).GetAwaiter().GetResult();
    } catch (Exception exPS) {
        Util.Capture(exPS, "VentaController", "addVentaDeliveryMovil.PrinterService", ...);
    }

    if (resultadoPS != null && resultadoPS.Success) {
        impresion = null;
        impresionTicketPromocion = null;
        impresionTextoEncuesta = null;
        impresionMotorizado = null;
    } else {
        mensajes.Add("Aviso PS delivery: " + errorMsg);
    }
}

```

28.3.7 addVentaDeliveryMovil — Respuesta

```

respuesta_.PrintJobResults = printJobResultsParaRespuesta;

```

28.4 Diferencias clave entre ambos métodos

Aspecto	addVentaRapidaMovil	addVentaDeliveryMovil
Factory method	CrearParaVentaRapida	CrearParaVentaDelivery
Comandas	Sí (vía impresionList + imprimirComandas)	No
Strategy seleccionada	ServerVentaRapidaStrategy	ServerDeliveryStrategy
impresionList vaciada	Sí (new List<Impresion>())	No aplica

28.5 Diferencia con PedidoController.addLista

En `PedidoController.addLista`, el bloque PS guarda los `PrintJobResult` en la tabla `pedidoprinterjob` (`PedidoprinterjobController.guardarJobsPorComandas`) y registra los jobs en `PrintJobStatusManager.Instance.RegisterJobs`. En `VentaController`, esta lógica de persistencia **no se incluye aún** porque las strategies (`ServerVentaRapidaStrategy` / `ServerDeliveryStrategy`) retornan `PrintResultDto` (éxito/fallo) en lugar de `List<PrintJobResult>`. La persistencia de jobs por tipo de venta se implementará en una fase posterior cuando se extienda `ServerImpresionService` para retornar `PrintJobResult` por cada documento enviado.

28.6 Build

Proyecto	Errores CS
QuipuNetX.sln	✔ 0 errores CS

28.7 Próximo paso

Extender las strategies para que retornen `List<PrintJobResult>` por cada documento enviado, y agregar la persistencia en `pedidoprinterjob` + el registro en `PrintJobStatusManager` — similar al patrón ya implementado en `PedidoController.addLista`.

Próximo paso completado — ver sección 29.

29. Trazabilidad completa: Strategies extraen, persisten y retornan PrintJobResults (Fase 7B — cierre)

29.1 Objetivo

Completar el circuito de trazabilidad `job_id ↔ tipo ↔ venta_id` para que:

1. Cada strategy extraiga los `PrintJobResult` de cada `Respuesta.Data` de `PrinterServiceClient`
2. Les asigne el Tipo correcto (`COMPROBANTE`, `COMANDA`, `PROMOCION`, `ENCUESTA`, `PRECUENTA`, `DELIVERY`)
3. Los persista en `pedidoprinterjob` vía `PedidoprinterjobController.guardarJobsPorTipo()`
4. Los registre en `PrintJobStatusManager.Instance.RegisterJobs()` para monitoreo en RAM
5. Maneje race conditions con callbacks tempranos de gRPC
6. Los retorne consolidados en `PrintResultDto.PrintJobResults` al controller
7. El controller los inyecte en `respuesta_.PrintJobResults` para el frontend

29.2 Cambios en PrintJobResult.cs

Nueva propiedad Tipo :

```
/// Tipo de documento: COMANDA, COMPROBANTE, PROMOCION, ENCUESTA, PRECUENTA, DELIVERY
public string Tipo { get; set; }
```

29.3 Cambios en PrintResultDto.cs

Nueva propiedad + factory methods actualizados:

```
/// Lista consolidada de PrintJobResult de todos los documentos enviados a PS
public List<PrintJobResult> PrintJobResults { get; set; }

// Factory methods ahora aceptan la lista:
public static PrintResultDto CreateSuccess(string ventaId, List<PrintJobResult> printJobResults = null)
public static PrintResultDto CreateError(string ventaId, string errorMessage, PrintErrorType errorType,
List<PrintJobResult> printJobResults = null)
```

Nota: `CreateError` también recibe `printJobResults` porque en best-effort pueden haber jobs exitosos + errores parciales (ej: comprobante OK pero encuesta falló).

29.4 Patrón helper `ExtraerYRegistrarJobs` (en cada strategy)

```
private static void ExtraerYRegistrarJobs(Respuesta resp, string tipo, string ventaId, List<PrintJobResult> todosLosJobs)
{
    // 1. Extraer de resp.Data: puede ser List<PrintJobResult> o PrintJobResult individual
    // 2. Asignar Tipo a cada job
    // 3. PedirPrinterJobController.guardarJobsPorTipo(jobs, tipo, ventaId)
    // 4. PrintJobStatusManager.Instance.RegisterJobs(jobs)
    // 5. Sincronizar race condition (si callback gRPC ya llegó con FAILED/EXPIRED)
    // 6. Acumular en todosLosJobs
}
```

29.5 Mapeo Tipo por sección en cada strategy

ServerVentaRapidaStrategy (5 tipos)

Sección	Método PrinterServiceClient	Tipo en BD	ventaId
1) Comprobante	EnviarVentaAsync	COMPROBANTE	✅ venta_id
2) Comandas	EnviarComandasAsync	COMANDA	❌ null (usa pedidoIds)
3) Sorteo	EnviarPromocionAsync	PROMOCION	✅ venta_id
4) Encuesta	EnviarEncuestaAsync	ENCUESTA	✅ venta_id
5) Motorizado	EnviarPrecuentaAsync	PRECUENTA	✅ venta_id

ServerDeliveryStrategy (4 tipos)

Sección	Método PrinterServiceClient	Tipo en BD	ventaId
1) Comprobante	EnviarVentaAsync	COMPROBANTE	✅ venta_id
2) Sorteo	EnviarPromocionAsync	PROMOCION	✅ venta_id
3) Encuesta	EnviarEncuestaAsync	ENCUESTA	✅ venta_id
4) Motorizado	EnviarPrecuentaAsync	DELIVERY	✅ venta_id

Nota: En `ServerDeliveryStrategy`, el motorizado se clasifica como `DELIVERY` (sección 24.8 del vibe). En `ServerVentaRapidaStrategy`, como `PRECUENTA`.

29.6 Cambios en `VentaController.cs`

Los bloques PS en `addVentaRapidaMovil` y `addVentaDeliveryMovil` ahora extraen los jobs del `resultadoPS`:

```
// Dentro del bloque if (resultadoPS.Success):
if (resultadoPS.PrintJobResults != null && resultadoPS.PrintJobResults.Count > 0)
{
    printJobResultsParaRespuesta = resultadoPS.PrintJobResults;
}
```

La persistencia (`guardarJobsPorTipo` + `RegisterJobs` + race condition sync) ya **NO se hace en el controller** — se hace dentro de la strategy vía `ExtraerYRegistrarJobs`. El controller solo extrae la lista consolidada para inyectarla en `respuesta_.PrintJobResults`.

29.7 Flujo completo (ejemplo: venta rápida #5001)

```
VentaController.addVentaRapidaMovil()
→ ImpresionContext.CrearParaVentaRapida(5001, impresiones, comandas, true)
→ ServerImpresionService.ImprimirAsync(ctx)
→ ServerVentaRapidaStrategy.EjecutarAsync(ctx)
|
|— EnviarVentaAsync() → resp.Data = PrintJobResult{JobId:"a1b2"}
|   → ExtraerYRegistrarJobs(resp, "COMPROBANTE", "5001", todosLosJobs)
|       → guardarJobsPorTipo([a1b2, COMPROBANTE]), "COMPROBANTE", "5001")
|       → RegisterJobs([a1b2])
|       → todosLosJobs = [a1b2, COMPROBANTE]
|
|— EnviarComandasAsync() → resp.Data = List<PrintJobResult>[{JobId:"b3c4"}]
|   → ExtraerYRegistrarJobs(resp, "COMANDA", null, todosLosJobs)
|       → guardarJobsPorTipo([b3c4, COMANDA], "COMANDA", null)
```

```

        → RegisterJobs([b3c4])
        → todosLosJobs = [a1b2, COMPROBANTE], {b3c4, COMANDA}]

    EnviarPromocionAsync() → resp.Data = PrintJobResult{JobId:"d5e6"}
    → ExtraerYRegistrarJobs(resp, "PROMOCION", "5001", todosLosJobs)

    EnviarEncuestaAsync() → resp.Data = PrintJobResult{JobId:"f7g8"}
    → ExtraerYRegistrarJobs(resp, "ENCUESTA", "5001", todosLosJobs)

    EnviarPrecuentaAsync() → resp.Data = PrintJobResult{JobId:"h9i0"}
    → ExtraerYRegistrarJobs(resp, "PRECUESTA", "5001", todosLosJobs)

    PrintResultDto { Success=true, PrintJobResults=[a1b2,b3c4,d5e6,f7g8,h9i0] }

    resultadoPS.PrintJobResults → printJobResultsParaRespuesta
    → respuesta_.PrintJobResults = printJobResultsParaRespuesta
    → return respuesta_ // Frontend recibe 5 jobs para monitorear

```

29.8 Build

Proyecto	Errores CS
QuipuNetX.sln	✔ 0 errores CS

29.9 Archivos modificados

Archivo	Cambio
Services/Print/PrintJobResult.cs	+ propiedad Tipo
Services/Print/PrintResultDto.cs	+ propiedad PrintJobResults + factory methods con parámetro opcional
Services/Print/Strategies/ServerVentaRapidaStrategy.cs	+ using controller + ExtraerYRegistrarJobs en 5 secciones + helper method + todosLosJobs en CreateSuccess/CreateError
Services/Print/Strategies/ServerDeliveryStrategy.cs	+ using controller + ExtraerYRegistrarJobs en 4 secciones + helper method + todosLosJobs en CreateSuccess/CreateError
controller/VentaController.cs	+ extracción de printJobResultsParaRespuesta del resultadoPS.PrintJobResults en ambos métodos

29.10 Próximo paso

La integración de PrinterServices en VentaController está completa para Fase 7B. Los próximos pasos son:

- **Frontend:** Implementar `verificarPrintJobsPorVenta(respuesta)` en los Presenters de venta para monitorear los jobs via `PSProgressService`
- **ServerVentaSalonStrategy:** Integrar en `addVentaSalonMovil` (si existe) con el mismo patrón
- **Fase 7C:** `ServerAnularCorregirStrategy` , `ServerVentaProveedorStrategy`

Próximo paso completado — ver sección 30.

30. Integración PS en confirmación de delivery: Backend + Frontend (Fase 7B)

30.1 Contexto

Al confirmar un delivery pendiente (botón azul ✓ en la pantalla "PEDIDOS POR CONFIRMAR"), el backend genera comandas de cocina y las envía al frontend para impresión local. Este flujo necesita integrarse con PrinterServices.

30.2 Flujo original (sin PS)

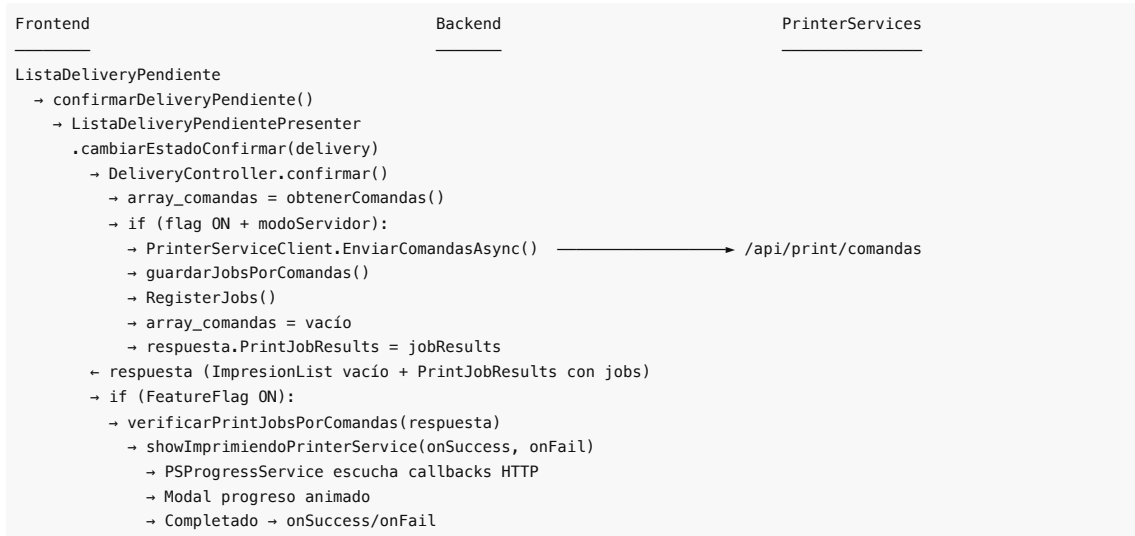
Frontend	Backend	Impresora
ListaDeliveryPendiente → confirmarDeliveryPendiente() → ListaDeliveryPendientePresenter .cambiarEstadoConfirmar(delivery) → DeliveryIterator .cambiarEstadoConfirmar(id) → DeliveryRouter.confirmar(id, cajaId) → DeliveryController .confirmar(id, cajaId)		objDelivery.obtenerComandas() → array_comandas return Respuesta(tipo, null,

```

                                mensajes, array_comandas,
                                objDelivery)
← respuesta.ImpresionList
PrintUtil.imprimirComandasEthernet() → TCP Ethernet

```

30.3 Flujo con PS (flag ON)



30.4 Cambios — Backend

DeliveryController.cs

- **Import:** using QuipuNetX.Services.Print;
- **Bloque PS** insertado antes del return en confirmar(): mismo patrón que PedidoController.addLista
 - Verifica flag + modo servidor
 - Envía comandas a PS via EnviarComandasAsync
 - Guarda jobs en pedidoprinterjob via guardarJobsPorComandas
 - Registra en PrintJobStatusManager
 - Vacía array_comandas
- **Return:** Construcción explícita de Respuesta con PrintJobResults = printJobResultsParaRespuesta

30.5 Cambios — Frontend

ListaDeliveryPendienteIView.cs

- using System;
- 3 métodos nuevos: showImprimiendoPrinterService(Action, Action), showFailedPrinterService(), showSuccessPrinterService()

ListaDeliveryPendiente.cs

- using QuipuNet.Xaml.NuevoDiseño.OpcionesGenerales.Notificaciones_Impresiones;
- using QuipuNet.MainApp.NuevoDiseñoQuipunet.Impresoras.ViewModels;
- using QuipuNet.MainApp.Services;
- 3 métodos show* implementados (mismo patrón que ListaPedidosTemporales.cs):
 - showImprimiendoPrinterService : PSProgressService + CustomModalImprimiendoNewView + callbacks Completado
 - showFailedPrinterService : PSModalImpresionFallidaViewModel + CustomModalImpresionFallidaNew + auto-retry
 - showSuccessPrinterService : CustomModalImpresionExitosaNew + auto-close 3s

ListaDeliveryPendientePresenter.cs

- using QuipuNet.MainApp.servicios; (FeatureFlagManager)
- using QuipuNetX.Services.Print; (PrintJobResult)
- Bifurcación en cambiarEstadoConfirmar: if (FeatureFlagManager.Instance.IsEnabled("USAR_PRINTER_SERVICE")) → verificarPrintJobsPorComandas(respuesta) / else if → impresión local
- Método nuevo verificarPrintJobsPorComandas(Respuesta): verifica PrintJobResults, muestra modal progreso con callbacks success/fail

30.6 Build

Proyecto	Errores CS
QuipuNetX.sln	✅ 0 errores

QuipuNet (Front)	✔ 0 errores
------------------	-------------

30.7 Próximo paso

- Integrar PS en otros puntos de la pantalla de delivery (reimpresión de comandas, precuenta)
- Integrar `ServerVentaSalonStrategy` en `addVentaSalonMovil`
- Fase 7C: strategies adicionales

Próximo paso completado — ver secciones 31 y 32.

31. Fixes: Timeout PSProgressService + Migración BD + Presenter correcto delivery (Marzo 2026)

31.1 Fix: Modal "Imprimiendo" se quedaba permanente

Síntoma: El modal de progreso PS se quedaba abierto indefinidamente cuando `PrinterServices` imprimía pero los callbacks HTTP no llegaban a `QuipuNetX`.

Causa raíz: `PSProgressService` esperaba `AllJobsDone` sin timeout. Si PS no enviaba callbacks, el evento nunca se disparaba.

Fix: Agregar timeout de 30 segundos en `PSProgressService.Timer_Tick()`. Si `AllJobsDone` no llega en 30s, forzar `Completado(huboErrores: true)`.

Archivo: `Front/MainApp/Services/PSProgressService.cs` — `Timer_Tick()`

31.2 Fix: NullReferenceException en EnsureTable al arrancar

Síntoma: La app crashaba al iniciar con `NullReferenceException` en `SugarContext.getSugarContext()`.

Causa raíz: `EnsureTable()` usaba `getSugarDataBase()` (estático, requiere `instance` inicializado), pero se llamaba dentro de `createTables()` cuando `instance` aún no existía.

Fix: Mover los `ALTER TABLE` directamente a `SugarContext.createTables()` usando `getSugarDb()` (método de instancia, accesible dentro del constructor).

Archivo: `QuipuNetX/Sugar/Com/Orm/SugarContext.cs` — `createTables()`

31.3 Fix: Presenter correcto para "PEDIDOS POR CONFIRMAR"

Síntoma: Los modales PS no aparecían al confirmar delivery desde la pantalla principal.

Causa raíz: La pantalla "PEDIDOS POR CONFIRMAR" usa `DeliverysConfirmadosNew` → `DeliverysConfirmadosNewPresenter`, NO `ListaDeliveryPendiente` → `ListaDeliveryPendientePresenter`. La integración PS se había hecho en el presenter equivocado.

Fix: Integrar PS en `DeliverysConfirmadosNewPresenter`:

- `DeliverysConfirmadosNewIView.cs` : + `using System`; + 3 métodos `show*`
- `DeliverysConfirmadosNew.cs` : + imports PS + 3 métodos `show*` implementados (sin auto-retry)
- `DeliverysConfirmadosNewPresenter.cs` : + `using servicios`; + bifurcación `FeatureFlagManager` + `verificarPrintJobsPorComandas`

31.4 Fix: Modal error desaparecía por auto-retry

Síntoma: El modal de error PS aparecía y desaparecía inmediatamente.

Causa raíz: `showFailedPrinterService()` ejecutaba `viewModel.ReintentarCommand.Execute(null)` (auto-retry) que causaba cierre rápido.

Fix: Eliminar `ReintentarCommand.Execute(null)` de `showFailedPrinterService()`. El usuario decide cuándo reintentar. Mismo patrón que `ListaPedidosTemporales.cs`.

31.5 Build

Proyecto	Errores CS
QuipuNetX.sln	✔ 0 errores
QuipuNet (Front)	✔ 0 errores

32. Evolución tabla pedidoprinterjob: Columnas MODALIDAD y DELIVERYID (Marzo 2026)

32.1 Problema

La tabla `pedidoprinterjob` no permitía saber:

1. Desde qué flujo se originó una impresión (venta rápida, salón, delivery, selfservice)
2. A qué delivery estaba asociada una comanda o comprobante

32.2 Solución: 2 columnas nuevas

Columna MODALIDAD

Valor	Flujo de origen	Caller
VENTA_RAPIDA	Venta rápida (salón/llevar)	VentaController.addVentaRapidaMovil
VENTA_SALON	Venta de salón	PedidoController.addLista (via modalidadVenta)
DELIVERY	Delivery	DeliveryController.confirmar / VentaController.addVentaDeliveryMovil
SELFSERVICE	Self-service	PedidoController.addLista (via modalidadVenta)
(otros)	Según modalidadVenta del caller	Propagado desde el frontend

Columna DELIVERYID

ID del delivery asociado. Permite asociar tanto la COMANDA como el COMPROBANTE de un delivery al mismo `delivery_id`.

- `null` si no es delivery
- `"1547"` si es delivery #1547

32.3 Esquema final

ID (PK)	PEDIDOID	VENTAID	TIPO	JOBID	STATUS	FECHACREACION	MODALIDAD	DELIVERYID
390	"45036"	NULL	"COMANDA"	"09d3..."	"DONE"	"2026-03-04 10:40"	"DELIVERY"	"1547"
391	NULL	"5001"	"COMPROBANTE"	"a1b2..."	"SENT"	"2026-03-05 13:30"	"VENTA_RAPIDA"	NULL
392	NULL	"6001"	"DELIVERY"	"d3e4..."	"SENT"	"2026-03-05 13:31"	"DELIVERY"	"1548"

32.4 Cambios implementados

Archivo	Cambio
PedidoprinterjobBase.cs	+ fields modalidad, deliveryid + [Column] + fromJSON + toJSON
SugarContext.cs	+ 2 ALTER TABLE para MODALIDAD y DELIVERYID
PedidoprinterjobController.cs	+ params modalidad, deliveryId en guardarJobsPorComandas y guardarJobsPorTipo
ImpresionContext.cs	+ props Modalidad, DeliveryId + params en 4 factory methods
ServerVentaRapidaStrategy.cs	+ propagación context.Modalidad/DeliveryId en ExtraerYRegistrarJobs
ServerDeliveryStrategy.cs	+ propagación context.Modalidad/DeliveryId en ExtraerYRegistrarJobs
VentaController.cs	+ "VENTA_RAPIDA" y "DELIVERY" + delivery_id en factory calls
PedidoController.cs	+ modalidadVenta, delivery_id en guardarJobsPorComandas
DeliveryController.cs	+ "DELIVERY", delivery_id en guardarJobsPorComandas

32.5 Migración de BD existente

SQLite no soporta `IF NOT EXISTS` en `ALTER TABLE`. La migración se hace con try/catch en `SugarContext.createTables()`:

```
try { getSugarDb().Execute("ALTER TABLE pedidoprinterjob ADD COLUMN MODALIDAD TEXT DEFAULT NULL"); } catch (Exception) { }
try { getSugarDb().Execute("ALTER TABLE pedidoprinterjob ADD COLUMN DELIVERYID TEXT DEFAULT NULL"); } catch (Exception) { }
```

Registros existentes quedan con `MODALIDAD = NULL` y `DELIVERYID = NULL` (compatibles).

32.6 Compatibilidad hacia atrás

- Todos los params nuevos son opcionales (`= null`) → callers existentes compilan sin cambios
- Registros antiguos (sin MODALIDAD/DELIVERYID) siguen funcionando
- `EnsureTable()` en `PedidoprinterjobBase` también tiene los `ALTER TABLE` como respaldo

32.7 Build

Proyecto	Errores CS
QuipuNetX.sln	✔ 0 errores

32.8 Próximo paso

- Verificar que las columnas se crean correctamente al reiniciar la app
- Implementar queries por MODALIDAD y DELIVERYID (`findByModalidad` , `findByDeliveryid`)
- Fase 7C: estrategias adicionales

33. Regla de unicidad: PSPProgressService es el ÚNICO disparador de modales de resultado de impresión

33.1 Regla

Cuando PrinterServices está activo (`USAR_PRINTER_SERVICE=true` en servidor, o `FEATURE_FLAG_FROM_SERVER_PRINTER_SERVICES=true` en cliente), **PSPProgressService.Completado** es el **ÚNICO** evento que debe desencadenar la apertura de modales de resultado de impresión:

- **CustomModalImpresionExitosaNew** → se abre desde `onSuccess` del callback de `Completado`
- **CustomModalImpresionFallidaNew** → se abre desde `onFail` del callback de `Completado`

Ningún otro componente debe abrir estos modales cuando PrinterServices está activo.

33.2 Flujo único autorizado

```
Presenter.verificarPrintJobsPorComandas(respuesta)
→ view.showImprimiendoPrinterService(onSuccess, onFail, cantidadJobs)
→ PSPProgressService (se suscribe a PrintJobStatusManager)
→ Escucha callbacks reales de PrinterServices
→ Completado(huboErrores) se dispara cuando:
  a) Todos los jobs terminaron (OnAllJobsDone) → huboErrores = true/false
  b) Timeout de seguridad (30s servidor, 300s cliente) → huboErrores = true
→ Callback del caller:
  if (huboErrores) → onFail() → showFailedPrinterService()
                                → new PSModalImpresionFallidaViewModel()
                                → new CustomModalImpresionFallidaNew(viewModel)
  else                → onSuccess() → showSuccessPrinterService()
                                → new CustomModalImpresionExitosaNew()
```

33.3 Componentes que NO deben abrir modales de resultado

Componente	Rol cuando PS activo	¿Abre modal?
UtilesCabeceraViewModel (OcultarProgresoImpresion)	Solo iniciar animación del badge de cabecera	✗ NO
ListaPedidos (OnComandaFailed)	Solo actualizar badge pulsante en pedidos	✗ NO
PrintJobStatusManager (eventos directos)	Solo fuente de datos RAM y eventos	✗ NO

33.4 Guard en UtilesCabeceraViewModel

En `UtilesCabeceraViewModel.OcultarProgresoImpresion()` , el guard **debe cubrir ambos flags** (servidor local y cliente remoto):

```
// UtilesCabeceraViewModel.cs - línea 626
bool usarPS = FeatureFlagManager.Instance.IsEnabled("USAR_PRINTER_SERVICE");
bool servidorTienePS = Util.esModoCliente() && UtilFront.FEATURE_FLAG_FROM_SERVER_PRINTER_SERVICES;

if (usarPS || servidorTienePS)
{
    // Solo iniciar animación del badge de cabecera (sin modal).
    // El modal lo abre exclusivamente el Presenter vía PSPProgressService.Completado
    iniciarAnimacion();
    return;
}
```

33.5 Razón del diseño

1. **Evita modales duplicados:** Sin esta regla, `UtilesCabeceraViewModel` (vía `AllJobsDone` → `OcultarProgresoImpresion`) y el `Presenter` (vía `PSPProgressService.Completado`) podían abrir dos modales simultáneos, causando confusión al usuario.

2. **ViewModel correcto:** `UtilesCabeceraViewModel` usaba `ModaImpresionFallidaViewModel` (legacy, lee de `FailedQueueManager`), mientras que el Presenter usa `PSModaImpresionFallidaViewModel` (reactivo, lee de `PrintJobStatusManager`). Solo el segundo es compatible con `PrinterServices`.
3. **Auto-cierre destructivo:** El modal de cabecera se auto-cerraba a los 5 segundos (`Task.Delay(5000)`), destruyendo la UI antes de que el usuario pudiera interactuar con los botones Reintentar/Cancelar.
4. **Responsabilidad clara:** El Presenter sabe qué pedido/venta se estaba procesando. La cabecera es un componente global que no tiene ese contexto.

33.6 Callers autorizados (Presenters)

Cada Presenter implementa el flujo vía su View (`showImprimiendoPrinterService` → `showFailedPrinterService`):

Presenter	View	Pantalla
<code>ListaPedidosTemporalesPresenter</code>	<code>ListaPedidosTemporales</code>	Pedidos salón
<code>FragmentDetallePedidoPresenter</code>	<code>FragmentDetallePedido</code>	Reimpresión pedido
<code>DeliverysConfirmadosNewPresenter</code>	<code>DeliverysConfirmadosNew</code>	Delivery confirmado
<code>ListaDeliveryPendientePresenter</code>	<code>ListaDeliveryPendiente</code>	Delivery pendiente
<code>ResumenDeliveryPresenter</code>	<code>ResumenDelivery</code>	Resumen delivery

Todos siguen el mismo patrón: `showImprimiendoPrinterService(onSuccess, onFail, cantidadJobs)` → `PSProgressService` → `Completado` → `onFail()` → `showFailedPrinterService()` .

33.7 Bug corregido

Archivo: `QuipuNet/ViewModels/General/UtilesCabeceraViewModel.cs` **Línea:** 626 **Cambio:** `if (usarPS)` → `if (usarPS || servidorTienePS)` **Efecto:** En modo cliente, la cabecera ya no abre un modal propio cuando `PrinterServices` está activo en el servidor remoto.